机器学习

Machine Learning

严绍恒 (*Shaoheng Yan*)

Homepage: PhotonYan.fun/about

Instructor: 张牧涵（*Muhan Zhang*）老师

Course：2025-2026 秋季学期「机器学习」

January 2026

Contents

Chapter 1 Linear Regression **2**
1.1 Basic Knowledge of Linear Regression . . . 2
1.2 Closed-Form of Linear Regression . . . 3
1.3 Geomeric View of LR . . . 6

Chapter 2 Logistic Regression **8**
2.1 Classification . . . 8
2.1.1 Binary Classification Problem . . . 8
2.1.2 Multi-Class Classificaiton (Softmax Regression) . . . 12
2.2 Rethink of Linear Regression . . . 13
2.2.1 MLE Explanation for Linear Regression . . . 13
2.2.2 Maximum a Posteriori (MAP) . . . 14

Chapter 3 Support Vector Machine **15**
3.1 Constrained Optimization . . . 15
3.2 Inequality optimization . . . 17
3.3 General Case . . . 18
3.4 Support Vector Machine . . . 19
3.4.1 Hard Margin . . . 19
3.5 Dual Form Of SVM . . . 22
3.5.1 Preliminaries from Convex Analysis* . . . 22
3.5.2 Dual Theory in the SVM Problem . . . 25
3.6 Kernel . . . 29

Chapter 4 Representer Theorem **34**
4.1 Revisit the Preliminaries . . . 34
4.2 Representer Theorem . . . 36
4.3 Dual Form of Linear Regression . . . 40

Chapter 5 Learning Theory **42**
5.1 Learning Theory . . . 42
5.2 VC-Dim . . . 48

Chapter 6 Gaussian Process **52**
6.1 Stochastic Process . . . 52
6.2 Gaussian Process Regression (GPR) . . . 57

Chapter 7 Trees and Ensemble Learning **60**
7.1 Decision Trees and Feature Selection . . . 62
7.1.1 Decision Trees and Axis-Aligned Splits . . . 62
7.1.2 Entropy, Information Gain, and Mutual Information . . . 63
7.1.3 Information Gain Ratio . . . 67
7.1.4 Gini Index . . . 68
7.1.5 Building Decision Trees: Greedy Algorithm . . . 69
7.1.6 Continuous Features in Decision Trees . . . 70
7.2 From Single Trees to Ensembles . . . 70

7.2.1 Bias–Variance Decomposition . . . 71
7.2.2 Bagging: Bootstrap Aggregating . . . 72
7.2.3 Random Forests . . . 73
7.2.4 Boosting . . . 74

Chapter 8 Unsupervised Learning **78**
8.1 Dimensionality reduction . . . 78
8.1.1 Principal Component Analysis (PCA) . . . 78
8.1.2 Singular Value Decomposition (SVD) . . . 81
8.1.3 t-distributed Stochastic Neighbor Embedding (t-SNE)* . . . 82
8.2 Clustering . . . 83
8.2.1 K-means Clustering Algorithm . . . 83
8.2.2 Mixture of Gaussians (MoG) . . . 85
8.3 Latent Variable Models and the EM Algorithm . . . 86

Chapter 9 Variational Autoencoder (VAE) **89**
9.1 Autoencoders (AE) . . . 89
9.2 Latent-Variable Generative Modeling . . . 90
9.3 Reparameterization Trick . . . 94

Chapter 10 Diffusion Model **96**
10.1 From a Physical Picture to Diffusion Models* . . . 96
10.1.1 Langevin Dynamics . . . 96
10.1.2 Score Matching . . . 97
10.1.3 Diffusion Models . . . 99
10.1.4 Discrete DDPM . . . 101
10.2 Denoising Diffusion Probabilistic Models (DDPM) . . . 103
10.3 Practical Implementation of Diffusion Models . . . 108

Preface

This lecture note is written to address the absence of a well-formatted, illustrated reference for this course. It began as spontaneous notes taken during class, and has gradually evolved through the careful revisions of the instructor and teaching assistants, as well as polishing assisted by large language models.

Some parts may remain incomplete or insufficiently elaborated. I sincerely ask for your understanding. If you find any typos, inaccuracies, or suggestions for improvement, you are warmly welcome to open an issue on GitHub:

https://github.com/PhotonYan/MachineLearningCourse2025

Shaoheng Yan

Tong Class 2024, Yuanpei College

Mμ Lab, Institute for Artificial Intelligence (IAI)

Peking University, November 2025

Environments in This Note

This lecture note adopts a unified visual style for mathematical structures and remarks. Below are the environments used throughout the text.

Definition 0.1

♣

Theorem 0.1

♡

Proposition 0.1

♡

Remark

Remark represents supplementary comments made by the instructor during class.

Note

Note indicates additional reflections by the author, which the reader may skip if desired.

Example 0.1

Proof

■

Solution

Chapter 1 Linear Regression

Introduction

- *ERM*
- *Gradient Descent*
- *Ridge Regression (L_2 regularization)*
- *Lasso Regression (L_1 regularization)*

1.1 Basic Knowledge of Linear Regression

Settings.

- Dataset: $D = \{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$. Here, y_i denotes the regression target, while x_i represents the input features used to predict y_i.
- Linear Model: $f(x) = w^\top x + b$, with weight $w \in \mathbb{R}^d$ and bias $b \in \mathbb{R}$.

Note

This definition is equivalent to an inner product: $\hat{y} = w^\top x + b$.

Definition 1.1 (Learnable / Trainable Parameters)

Learnable parameters are those that can be updated during the training process. ♣

Quiz. How to determine whether a parameter is learnable? How to determine w and b?

Ans: **ERM** (Empirical Risk Minimization)

- Loss function. Squared Loss (SE) is commonly used during optimization. The training objective can be written as:

$$\operatorname*{argmin}_{w,b} \frac{1}{n} \sum_{i \in [n]} \left(y_i - (w^\top x_i + b)\right)^2 \tag{1.1}$$

  The blue factor $1/n$ can be omitted in theoretical analysis, but is often kept in practice to stabilize the loss function during implementation.

Quiz: How to optimize the parameters?

Ans: **Gradient Descent** (as a traditional ML method). In the case of linear regression:

$$\frac{\partial \mathcal{L}}{\partial b} = -2 \sum_{i \in [n]} (y_i - w^\top x_i - b) \tag{1.2}$$

$$\frac{\partial \mathcal{L}}{\partial w} = -2 \sum_{i \in [n]} (y_i - w^\top x_i - b) x_i \tag{1.3}$$

Note

In the field of machine learning, the gradient of a scalar with respect to a vector is itself a vector ***(not a covector)****.*

This means:

$$\frac{\partial \mathcal{L}}{\partial w} = \begin{pmatrix} \frac{\partial \mathcal{L}}{\partial w_1} \\ \frac{\partial \mathcal{L}}{\partial w_2} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial w_d} \end{pmatrix} = \left(\frac{\partial \mathcal{L}}{\partial w_1}, \frac{\partial \mathcal{L}}{\partial w_2}, \cdots, \frac{\partial \mathcal{L}}{\partial w_d} \right)^\top \tag{1.4}$$

See the definition of matix derivatives (assuming no special structure in the matrix) in Matrix Cookbook Chapter 2.

Note

Here are some commonly used derivative formulas:

$$\frac{\partial x^\top x}{\partial x} = 2x \tag{1.5}$$

$$\frac{\partial a^\top x}{\partial x} = a, \quad \frac{\partial Ax}{\partial x} = A^\top \tag{1.6}$$

$$\frac{\partial x^\top Ax}{\partial x} = (A + A^\top)x \tag{1.7}$$

Remark

Both sides of an equation must have the same dimension. This principle can be used as a consistency check.

We optimize the parameters by subtracting a scalar multiple of the gradient from the parameters, considering the physical meaning of the gradient: the direction of the steepest **increase**.

Definition 1.2 (Hyperparameter)

A parameter that is fixed during optimization and specified before the training process. ♣

That is:

$$w' = w - \alpha \frac{\partial \mathcal{L}}{\partial w}, \quad b' = b - \alpha \frac{\partial \mathcal{L}}{\partial b} \tag{1.8}$$

Optimization will stop when the norm of the parameter update becomes smaller than a given hyperparameter.

1.2 Closed-Form of Linear Regression

Proposition 1.1

*Linear Regression has **Closed-Form** solution.* ♡

Settings.

- Matrix $X_0 := (x_1, \cdots, x_n)^\top$;
- Matrix $X := (X_0, \mathbb{1}) \in \mathbb{R}^{n \times (d+1)}$;
- $y = (y_1, \cdots, y_n)^\top \in \mathbb{R}^n$;
- $\hat{w} = (w^\top, b)^\top \in \mathbb{R}^{d+1}$.

Then the loss function of $\hat{w}$ can be written as:

$$\mathcal{L}(\hat{w}) = (y - X\hat{w})^\top (y - X\hat{w}) = \|y - X\hat{w}\|_2^2 \tag{1.9}$$

Here, $\|\cdot\|_p$ denotes the *p-norm* of a vector.

Note

Vectors can sometimes be treated as scalars, since linearity ensures that the validity of a proposition can be extended to any finite dimension.

Notice that the optimization stops when $\partial\mathcal{L}(\hat{w})/\partial\hat{w} = 0$. Under this condition, the parameters can be solved from the above constraint by following steps:

$$\frac{\partial\mathcal{L}(\hat{w})}{\partial\hat{w}} = -2X^\top(y - X\hat{w}) \tag{1.10}$$

Note

Both dimensional analysis and calculation using Leibniz's rule lead to the same result as the formula above:

$$\begin{aligned}\mathcal{L}(\hat{w}) &= y^\top y - 2y^\top X\hat{w} + \hat{w}^\top X^\top X\hat{w}\\ \partial_{\hat{w}}\mathcal{L}(\hat{w}) &= -2X^\top y + 2X^\top X\hat{w}\\ &= -2X^\top(y - X\hat{w})\end{aligned}$$

Remark

More matix formulas are available in Matrix Cookbook.

Thus, the target of the optimization satisfied:

$$X^\top y = X^\top X\hat{w} \tag{1.11}$$

That is:

$$\hat{w} = (X^\top X)^{-1}X^\top y \tag{1.12}$$

when $X^\top X$ invertible (non-singular / full-rank).

Example 1.1

When does $X^\top X$ not invertible?

Solution

$X \in \mathbb{R}^{n\times(d+1)}$:

- $d + 1 > n$. *Brief Proof:* $\operatorname{rank}(X^\top X) = \operatorname{rank}(X) \le \min(n, d+1) = n < d + 1$.
- *X has repeated columns. Proof is trivial.*

When $X^\top X$ isn't invertible:

1. If $\operatorname{rank}(X^\top X, X^\top y) > \operatorname{rank}(X^\top X)$, $\hat{w}$ has no solution;
2. $\hat{w}$ has infinity solution o.w.

Situation 1 is **impossible** because both $X^\top X$ and $X^\top y$ can be represented in the column space of $X^\top$. Therefore, the optimization problem must have a solution, which may be either unique or infinite.

As an infinite set of solutions makes it difficult to determine which estimate of $\hat{w}$ to choose, we apply L_2 **regularization** to linear regression, which is commonly referred to as **Ridge Regression**. That is:

$$\mathcal{L}_{L_2} := \mathcal{L}(\hat{w}) + \boxed{\lambda\|\hat{w}\|_2^2}, \tag{1.13}$$

where $\lambda > 0$ is a hyperparameter. Notice that $\|\hat{w}\|_2^2 = \sum_{i=1}^{d+1} \hat{w}_i^2$, L_2 regularization prevents any single dimension from being assigned an excessively large weight, and encourages the model to make use of more dimensions during training.

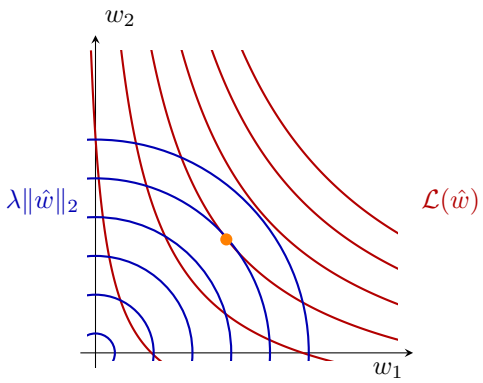


Figure 1.1: Illustration of L_2 regularization. The contours represent level sets of the regularized loss $\mathcal{L}(\hat{w}) + \lambda\|\hat{w}\|_2^2$, which take the form of concentric ellipses (circle in the plot).

During ridge regression, we minimize the $\mathcal{L}_{L_2}$:

$$\underset{\hat{w}}{\operatorname{argmin}}\, (y - X\hat{w})^\top (y - X\hat{w}) + \lambda \hat{w}^\top \hat{w} \tag{1.14}$$

The optimization stops when:

$$\frac{\partial \mathcal{L}_{L_2}}{\partial \hat{w}} = -2X^\top y + 2X^\top X\hat{w} + 2\lambda I\hat{w} = 0 \tag{1.15}$$

$$\Rightarrow (X^\top X + \lambda I)\hat{w} = X^\top y \tag{1.16}$$

Proposition 1.2

$X^\top X + \lambda I$ always invertible.

Proof

Since $X^\top X$ is a real symmetric matrix, we have the eigen-decomposition $X^\top X = U\Lambda U^\top$, where $\Lambda = \operatorname{diag}(\lambda_1, \cdots, \lambda_{d+1})$. Moreover, as $X^\top X \succeq 0$ (positive semi-definite), it follows that $\forall i \in [d+1]$, $\lambda_i \geq 0$. Note that:

$$\lambda I = \lambda U U^\top \tag{1.17}$$

since U is an orthogonal matrix. Hence:

$$X^\top X + \lambda I = U(\Lambda + \lambda I)U^\top \tag{1.18}$$

For all $i \in [d+1]$, we have:

$$\lambda_i + \lambda > \lambda_i \geq 0 \tag{1.19}$$

Thus, $X^\top X + \lambda I$ is a full-rank matrix.

Remark

Numerical issues may still occur even if $X^\top X$ is full rank (e.g., when eigenvalues λ_k are close to zero). The L_2 regularization factor λ mitigates this issue by shifting the eigenvalues upward, thereby improving numerical stability during training.

Another regularization method often used is L_1 regularization, where the loss function is defined as:

$$\mathcal{L}_{L_1} := \mathcal{L}(\hat{w}) + \boxed{\lambda\|\hat{w}\|_1} \tag{1.20}$$

L_1 regularization can induce sparsity in $\hat{w}$, which works in contrast to L_2 regularization. Specifically, L_1 regularization encourages the model to rely on only a small subset of input features, effectively performing **feature selection**.

Linear regression with L_1 regularization is called **Lasso Regression** (Least Absolute Shrinkage and Selection Operator).

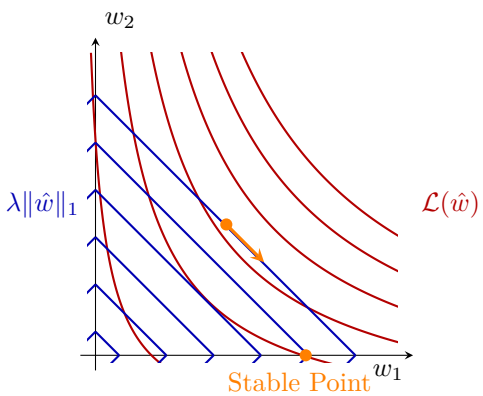


Figure 1.2: Illustration of L_1 regularization. The contours represent level sets of the regularized loss $\mathcal{L}(\hat{w}) + \lambda\|\hat{w}\|_1$, which take the form of nested diamonds (squares rotated by 45° in the plot).

1.3 Geomeric View of LR

Ideally, we would like to solve $X\hat{w} = y$. If y lies on the hypersurface

$$\mathcal{M}(X) := \mathrm{Span}(X) = \{Xw : w \in \mathbb{R}^{d+1}\} \subset \mathbb{R}^n \tag{1.21}$$

then the equation admits an exact solution. In most cases, however, $y \notin \mathcal{M}(X)$, so no exact solution exists. Nevertheless, we can always find an estimator $\hat{w}$ such that $\mathcal{P}_{\mathcal{M}(X)}y = X\hat{w}$, where $\mathcal{P}_{\mathcal{M}(X)}$ denotes the orthogonal projection onto the hypersurface $\mathcal{M}(X)$.

Proposition 1.3

$$\hat{y} = X\hat{w} \quad \Rightarrow \quad \hat{w} \textit{ is solution to LR.} \tag{1.22}$$

Proof

$$\begin{aligned} y - \hat{y} \perp \mathcal{M}(X) \quad &\Rightarrow \quad y - X\hat{w} \perp \mathcal{M}(X) \\ &\Rightarrow \quad X^\top(y - X\hat{w}) = 0 \quad \Rightarrow \quad \hat{w} = (X^\top X)^{-1}X^\top y \end{aligned}$$

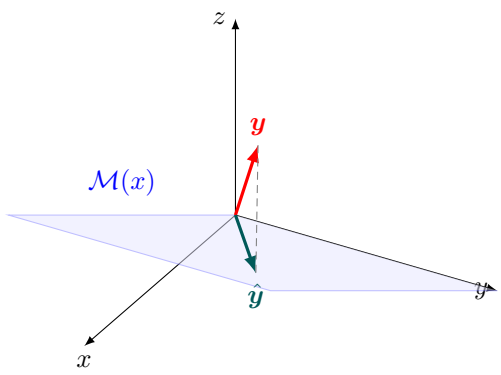


Figure 1.3: Orthogonal projection interpretation of linear regression. The predicted vector $X\hat{w}$ is obtained as the projection of y onto the hypersurface $\mathcal{M}(X) = \{Xw : w \in \mathbb{R}^{d+1}\}$, which is a linear subspace in the classical case.

Chapter 2 Logistic Regression

Introduction

- *Binary Classification Problem*
- *Cross Entropy*
- *Sigmoid Regression*
- *Maximum a posteriori*

2.1 Classification

2.1.1 Binary Classification Problem

Settings.

- Dataset: $D = \{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^d$ and $y_i \in \{0, 1\}$. Here, y_i denotes the classification target, while x_i represents the input features used to predict y_i.
- Model in Logistic Regression: In logistic regression, we start with a linear model $f(x) = w^\top x + b$. Unlike in ordinary regression, where the target variable lies in $\mathbb{R}$, here the target space collapses to $\{0, 1\}$. Thus, we need a function that maps real-valued outputs into this discrete set. Moreover, in many applications it is desirable to obtain not only a hard classification decision (0 or 1), but also a *soft* prediction: the probability of each class. Such a probabilistic interpretation provides both the likelihood estimate and the corresponding classification outcome.

Can we directly use a linear model to fit $p(y = 1 \mid x = x_i)$, as we did in the previous chapter? The answer is *no*. This is because there is a mismatch between the range of a linear model output (which lies in $\mathbb{R}$) and the valid domain of probabilities, $[0, 1]$.

To resolve this issue, we introduce a transformation function called the **sigmoid** function. The sigmoid maps any real-valued input into the interval $[0, 1]$, making it suitable for modeling probabilities. It is defined as:

Definition 2.1 (Sigmoid Function)

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{2.1}$$

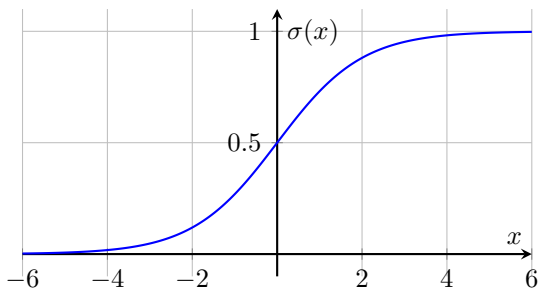


Figure 2.1: The sigmoid function $\sigma(z)$ over the interval $z \in [-6, 6]$.

The sigmoid function enjoys several elegant properties.

Theorem 2.1

$$1 - \sigma(z) = \sigma(-z)$$

Proof

This follows directly from the definition of $\sigma(z)$, or equivalently, by observing the symmetry of its graph. ■

To convert soft prediction results into binary outputs $\{0, 1\}$, we introduce a threshold: when $\sigma(z) = 0.5$, the model makes a hard prediction.

Definition 2.2 (Separating Hyperplane)

The condition $\sigma(z) = 0.5$ defines the separating hyperplane. *It partitions the input space $\mathbb{R}^d$ into two regions, thereby transforming probabilistic predictions into binary classification outcomes.*

The normal vector w is perpendicular to this hyperplane and points towards the region where the model predicts class 1 (i.e., where $p(y = 1 \mid x) > 0.5$). We ensure this property by choosing the orientation of w accordingly.

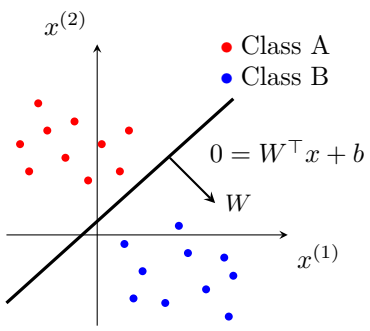


Figure 2.2: Classification by heperplane.

Model definition is clear, and now we turn our attention to parameter optimization. The question is: how can we find the proper w, b that achieve the best performance? The key problem here is to identify a suitable loss function that can be optimized via gradient descent.

Notice that:

$$P(y = 1 \mid x = x_i) = \sigma(f(x_i)) = \frac{1}{1 + \exp(-w^\top x + b)}. \tag{2.2}$$

We introduce a new method rather than continuing with ERM, by using **Maximum Likelihood Estimation (MLE)**. MLE is naturally designed to address probability modeling problems.

Definition 2.3 (Maximum Likelihood Estimation (MLE))

MLE aims to find parameters such that the likelihood of $P(y = y_i \mid x = x_i)$ is maximized.

Remark

For brevity, we write $P(y = y_i \mid x = x_i)$ as $P(y_i \mid x_i)$.

Definition 2.4 (Likelihood)

The likelihood on the entire training data is defined as

$$\prod_{i\in[n]} P(y_i \mid x_i; w, b) \tag{2.3}$$

assuming the samples are independent. ♣

According to the discussion above, in logistic regression we :

$$P(y_i \mid x_i) = \begin{cases} \sigma(w^\top x_i + b), & y_i = 1, \\ 1 - \sigma(w^\top x_i + b), & y_i = 0. \end{cases} \tag{2.4}$$

Therefore, the likelihood function can be expanded as:

$$\prod_{i\in[n]} \sigma(w^\top x_i + b)^{y_i} \left(1 - \sigma(w^\top x_i + b)\right)^{1-y_i} \tag{2.5}$$

The above form is intuitive once we recall that $x^0 = 1$.

To ensure floating-point precision, given the large amount of data and the monotonicity of the logarithm function, we transform the likelihood into the maximization of the log-likelihood:

$$\operatorname*{argmax}_{w,b} \sum_{i\in[n]} \left[y_i \log \sigma(w^\top x_i + b) + (1 - y_i) \log \left(1 - \sigma(w^\top x_i + b)\right) \right] \tag{2.6}$$

This is the final objective in MLE.

MLE can be transformed into ERM by applying argmin to the negative log-likelihood. We define the **Cross-entropy Loss**:

$$\mathcal{L}(w, b) := -\sum_{i\in[n]} \left[y_i \log \sigma(w^\top x_i + b) + (1 - y_i) \log \left(1 - \sigma(w^\top x_i + b)\right) \right] \tag{2.7}$$

Thus, minimizing the cross-entropy loss is equivalent to maximizing the log-likelihood, making the equivalence between MLE and ERM immediate.

Note

Why is the above loss called the Cross-entropy loss? The name originates from information theory. Entropy is defined as:

$$H(P) = \sum_y P(y) \log \frac{1}{P(y)} = -\sum_y P(y) \log P(y). \tag{2.8}$$

In other words, rarer events (with smaller probability) carry more information, and entropy measures the expected amount of information. In our context, we can evaluate the information content of the prediction for $y = \hat{y}_i$ given $x = x_i$ as:

$$\begin{aligned} H(P) &= -\sum_{\hat{y}_i \in \{0,1\}} P(y = \hat{y}_i \mid x_i) \log P(y = \hat{y}_i \mid x_i) \\ &= -[P(y = 1 \mid x_i) \log P(y = 1 \mid x_i) + P(y = 0 \mid x_i) \log P(y = 0 \mid x_i)] \end{aligned} \tag{2.9}$$

Notice that under our estimation, $P(y = 1 \mid x_i) = \sigma(f(x_i; w, b))$ and $P(y = 0 \mid x_i) = 1 - \sigma(f(x_i; w, b))$. In practice, we substitute the empirical distribution of samples for the true distribution when comparing the negative log-likelihood with entropy. This is why the terminology of entropy from information theory is carried over to name this loss term.

Note

The formal definition of cross entropy between two propbability distributions q and p $H(q,p)$ is defined by:

$$H(q,p) = -\sum_y q(y)\log p(y) = -\mathbb{E}_q \log p(y) \tag{2.10}$$

Definition 2.5 (KL-Divergence)

The KL-Divergence between two distributions p and q is defined by:

$$\mathrm{KL}(q\|p) = \sum_y q(y)\log\frac{q(y)}{p(y)} := \mathbb{E}_q \log\frac{q(y)}{p(y)} \tag{2.11}$$

♣

KL-Divergence measures the difference between two given distributions. In particular, $\mathrm{KL}(q\|p)$ differs from the cross-entropy by only a constant term $H(q)$:

$$\mathrm{KL}(q\|p) = H(q,p) - H(q). \tag{2.12}$$

In other words, KL-Divergence quantifies the extra number of bits required when we use p to approximate the ground-truth distribution q.

Back to the main content. Recall that a closed-form solution can be derived for the linear regression problem, as mentioned in the previous chapter. Here, we would like to investigate whether logistic regression also admits a closed-form solution.

Following the same steps we applied in linear case, we define $\hat{x} = (x^\top, 1)^\top \in \mathbb{R}^{d+1}$ and $\hat{w} = (w^\top, b)^\top \in \mathbb{R}^{d+1}$, thus $f(x) = \hat{w}^\top\hat{x}$:

$$\begin{aligned}
\mathcal{L}(\hat{w}) &= -\sum_{i\in[n]} y_i \log\sigma(\hat{w}\hat{x}_i) + (1-y_i)\log(1-\sigma(\hat{w}^\top\hat{x}_i)) \\
&= -\sum_{i\in[n]} y_i \log\frac{1+\exp(\hat{w}^\top\hat{x}_i)}{1+\exp(-\hat{w}^\top\hat{x}_i)} - \log(1+\exp(\hat{w}^\top\hat{x}_i)) \\
&= -\sum_{i\in[n]} y_i(\hat{w}^\top\hat{x}_i) - \log(1+\exp(\hat{w}^\top\hat{x}_i)) \qquad (2.13)
\end{aligned}$$

Take the derivative:

$$\frac{\partial\mathcal{L}(\hat{w})}{\partial\hat{w}} = -\sum_{i\in[n]}\left[y_i\hat{x}_i - \frac{\exp(\hat{w}^\top\hat{x}_i)}{1+\exp(\hat{w}^\top\hat{x}_i)}\hat{x}_i\right] \tag{2.14}$$

$$= -\sum_{i\in[n]}\left[y_i - P(y=1\mid x_i)\right]\hat{x}_i \tag{2.15}$$

By gradient descent, the parameter is updated as $\hat{w} \leftarrow \hat{w} + \alpha\sum_{i\in[n]}\Big(y_i - P(y_i=1\mid x_i)\Big)\hat{x}_i$. This makes sense, since the term $y_i - P(y_i=1\mid x_i)$ directly measures the prediction error on sample i, and the update moves $\hat{w}$ a small step along the direction of the input $\hat{x}_i$ to reduce this error.

If for all i, we have $y_i = P(y=1\mid x_i)$, then the model predicts every label y_i perfectly. At this point, optimization reaches a stationary solution. If the training data admits such a perfect solution:

Definition 2.6 (linearly separable)

If all points can be separated by a linear model without error, we say the dataset is linearly separable.

♣

Figure 2.2 illustrates a linearly separable setting, and the final state leads to $\|W\| \to \infty, \quad \|b\| \to \infty$. However, this situation is not desirable in practice, since it implies poor robustness. Hence a natural question arises: under the condition of linear separability, how can we find a well-chosen separating hyperplane that maximizes robustness? The answer will

be presented in the next chapter, where we introduce the Support Vector Machine (SVM). The SVM optimizes $\hat{w}, \hat{b}$ by maximizing the margin (the distance between data points and the separating hyperplane), instead of simply minimizing the cross-entropy loss.

Although logistic regression may suffer from divergence of parameters under separable data, it often achieves better performance than SVM in practice, due to the following reasons:

1. In most real-world problems, the data are not linearly separable;
2. Applying L_2 regularization can effectively prevent parameter divergence.

Remark

Why can't we use squared loss for classification? The reason is that in classification tasks such as logistic regression, the label $y \in \{0, 1\}$ should be interpreted as a categorical outcome rather than a numerical quantity.

2.1.2 Multi-Class Classificaiton (Softmax Regression)

We can combine k sub-classifiers to solve a k-class classification task. Specifically, we apply a sigmoid-like transformation to each sub-linear model $f_k(x) = w_k^\top x + b_k$, and obtain the probability of class k using a normalized expression:

$$P(y = k \mid x) = \frac{\exp(w_k^\top x + b_k)}{\sum_{j=1}^{k} \exp(w_j^\top x + b_j)}. \tag{2.16}$$

Note

*There is a close analogy between **softmax regression** in machine learning and the **partition function** in statistical physics.*

In softmax regression, the probability of assigning an input x to class k is

$$P(y = k \mid x) = \frac{\exp(\theta_k^\top x)}{\sum_{j=1}^{K} \exp(\theta_j^\top x)}. \tag{2.17}$$

Here the denominator

$$Z(x) = \sum_{j=1}^{K} \exp(\theta_j^\top x) \tag{2.18}$$

serves as a normalizing constant*, ensuring that the probabilities over all classes sum to* 1.

In statistical physics, for a system with possible states s of energy $E(s)$, the probability of observing state s under the Boltzmann distribution is

$$P(s) = \frac{\exp(-\beta E(s))}{Z} = -\frac{1}{\beta}\frac{\partial \log Z}{\partial E(s)}, \quad Z = \sum_{s} \exp(-\beta E(s)), \tag{2.19}$$

where Z is the partition function*. It normalizes the distribution and encodes all thermodynamic properties of the system.*

Thus, the role of $Z(x)$ in softmax regression is mathematically analogous to the role of the partition function Z in statistical physics: both are log-sum-exp normalizers that transform unnormalized scores (energies or logits) into proper probability distributions.

The softmax regression method has several advantages:

1. It is unified and normalized: $\sum_{j \in [j]} P(y = k \mid x) = 1$;
2. The amplify effect of exp function: when $f_k(x) \gg f_j(x), \forall j \neq k$, then $P(y = k \mid x) = 1$.

We apply the same MLE procedure to the multi-class classification task, which leads to the following optimization problem:

$$\underset{\{w_k, b_k\}}{\operatorname{argmax}} \sum_{i \in [n]} \log \frac{\exp(w_k^\top x_i + b_k)}{\sum_{j=1}^{k} \exp(w_j^\top x_i + b_j)} \tag{2.20}$$

Note

In modern neural networks, softmax regression is widely used as the standard classification method.

Remark

When $k = 2$, softmax regression reduces to logistic regression via the reparameterization $w = w_1 - w_2$ and $b = b_1 - b_2$.

2.2 Rethink of Linear Regression

2.2.1 MLE Explanation for Linear Regression

Definition 2.7 (Gaussian/Normal Distribution)

$$x \sim \mathcal{N}(\mu, \sigma^2) \quad \Leftrightarrow \quad P(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{2.21}$$

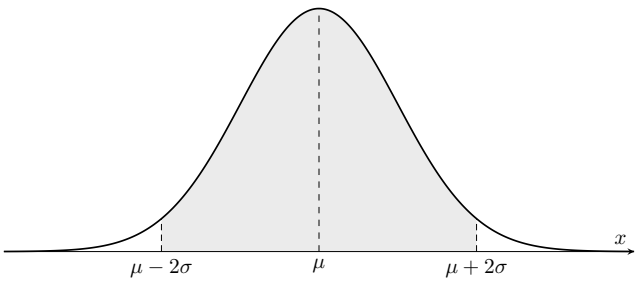


Figure 2.3: Normal distribution (95%).

While the Central Limit Theorem (CLT) does not imply that most datasets are normally distributed, it motivates modeling additive noise as Gaussian. We assume:

$$y = \underbrace{w^\top x + b}_{\text{latent model}} + \underbrace{\varepsilon}_{\text{noise}}, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2) \tag{2.22}$$

where σ^2 is a hyperparameter characterizing the noise scale. Then:

$$P(y \mid x; w, b, \sigma^2) = \mathcal{N}\big(y \,\big|\, w^\top x + b,\ \sigma^2\big) \tag{2.23}$$

The log-likelihood takes the form:

$$\begin{aligned} \underset{w,b}{\operatorname{argmax}} \sum_{i \in [n]} \log \mathcal{N}\big(y_i \,\big|\, w^\top x_i + b,\ \sigma^2\big) &= \underset{w,b}{\operatorname{argmax}} \sum_{i \in [n]} \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{\big(y_i - (w^\top x_i + b)\big)^2}{2\sigma^2} \right] \\ &\Leftrightarrow \underset{w,b}{\operatorname{argmin}} \sum_{i \in [n]} \big(y_i - (w^\top x_i + b)\big)^2, \end{aligned} \tag{2.24}$$

where the equivalence follows by dropping constants and positive scalings. Equation (2.24) recovers ERM with the squared-loss objective.

2.2.2 Maximum a Posteriori (MAP)

In the MLE perspective, w and b are treated as unknown fixed constants. In the Bayesian framework, however, even w, b are considered as random variables (R.V.). A "fixed constant" can be seen as a random variable with an extremely sharp distribution (near δ-distribution).

Suppose we place a Gaussian prior $P(\hat{w}) = \mathcal{N}(\hat{w} \mid 0,\ \sigma_w^2 \mathbb{I})$. The likelihood is given by

$$P(y \mid x, \hat{w}; \sigma^2, \sigma_w^2) = \mathcal{N}\big(y \mid \hat{w}^\top \hat{x},\ \sigma^2\big) \tag{2.25}$$

We want to compute the posterior distribution of $\hat{w}$. By Bayes' rule:

$$\begin{aligned} P(\hat{w} \mid y, x) &= \frac{P(y \mid x, \hat{w})\, P(\hat{w})}{P(y \mid x)} \\ &\propto \left(\prod_{i \in [n]} P(y_i \mid x_i, \hat{w}) \right) P(\hat{w}) \end{aligned}$$

Expanding this expression:

$$P(\hat{w} \mid y, x) = \frac{1}{Z} \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right)^n \exp\left(-\frac{1}{2\sigma^2} \sum_{i \in [n]} \big(y_i - \hat{w}^\top \hat{x}_i\big)^2 \right) \left(\frac{1}{\sqrt{2\pi\sigma_w^2}} \right)^{d+1} \exp\left(-\frac{\hat{w}^\top \hat{w}}{2\sigma_w^2} \right)$$

Taking the negative logarithm of the posterior, note that only the terms involving $\hat{w}$ are subject to optimization:

$$-\log P(\hat{w} \mid y, x) = \sum_{i \in [n]} \big(y_i - \hat{w}^\top \hat{x}_i\big)^2 + \frac{\sigma^2}{\sigma_w^2} \|\hat{w}\|^2 + \text{Const.}$$

Letting $\lambda = \sigma^2 / \sigma_w^2$, the MAP estimator is obtained by:

$$\operatorname*{argmin}_{\hat{w}} \sum_{i \in [n]} \big(y_i - \hat{w}^\top \hat{x}_i\big)^2 + \lambda \|\hat{w}\|^2, \tag{2.26}$$

which is exactly ridge regression. Therefore, the internal consistency of the theory is demonstrated.

Chapter 3 Support Vector Machine

Introduction

- *Constrained optimization*
- *Inequality optimization*
- *KKT condition*
- *Support Vector Machine*

Support Vector Machine (SVM) is also a binary classification model, often introduced as an alternative to logistic regression, since logistic regression has several notable limitations. SVM provides an elegant framework that allows us to transfer powerful results and intuition from optimization theory into machine learning.

3.1 Constrained Optimization

Unlike the unconstrained optimization problems we studied earlier, the support vector machine is originally formulated as a **constrained optimization problem**. This distinction is fundamental. In mathematics, unconstrained optimization is defined as:

$$\operatorname*{argmin}_{x} f(x) \tag{3.1}$$

When we deal with constrained optimization, we add an auxiliary constraint $h(x) = 0$ (where both $f(x)$ and $h(x)$ are differentiable). Considering the geometric meaning of the gradient, we observe:

1. For any point on the surface defined by $h(x)$, the gradient $\nabla h(x)$ must be orthogonal to the surface.
2. For any critical point x^* that achieves a local minimum, the gradient $\nabla f(x^*)$ must also be orthogonal to the surface.

Note

The second property can be understood by analogy with electrostatics: near a conductor, the electric field must be orthogonal to the surface. Otherwise, there would be a component of the field (gradient) along the constrained sub-hypersurface, violating equilibrium.

Extending this analogy: in the electrostatic case, such a tangential component would induce currents along the conductor's surface. In the optimization setting, a tangential component corresponds to a gradient lying along the constraint manifold, implying that the objective function can still be further optimized without violating the constraint.

Moreover, the charge distribution problem in electrostatics can itself be interpreted as an energy minimization process: the charges rearrange to minimize the system's electrostatic energy. In this sense, the equilibrium state of charges on a conductor can be seen as an optimization problem, which strengthens the analogy with constrained optimization in mathematics.

Combining the two conditions above, we obtain the **necessary** condition for a local minimum point, as shown in Figure 3.1:

$$\exists \lambda \quad \text{s.t.} \quad \nabla f(x^*) + \lambda \nabla h(x^*) = 0 \tag{3.2}$$

Remark

In some contour examples, the constraint condition does not cover the entire space. In such cases, the above equation may not *always be satisfied.*

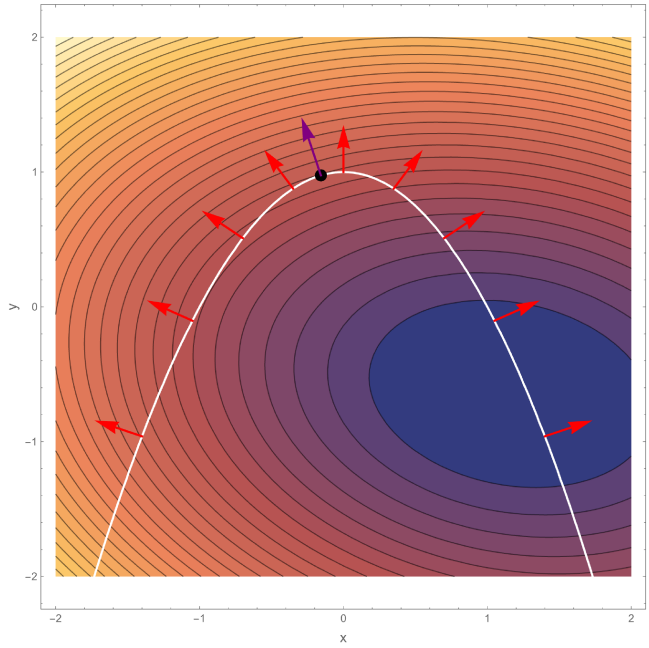


Figure 3.1: Illustration of the necessary condition for constrained local minima.

Now we introduce the *Lagrangian function*, which plays a central role in optimization theory:

$$L(x, \lambda) := f(x) + \underbrace{\lambda}_{\text{Lagrange Multiplier}} h(x) \tag{3.3}$$

Extending to the case of multiple constraints, the Lagrangian is defined as:

$$L(x, \lambda) := f(x) + \sum_{i=1}^{k} \lambda_i h_i(x) \tag{3.4}$$

Theorem 3.1

If x^* is a local minimum point, then there exists a set of multipliers $\lambda = (\lambda_1, \ldots, \lambda_k)$ such that

$$\begin{cases} \nabla_x L(x^*, \lambda) = 0, \\ \nabla_\lambda L(x^*, \lambda) = 0. \end{cases} \tag{3.5}$$

The first condition corresponds to the stationarity (minimum) condition, while the second condition enforces the constraints.

♡

Corollary 3.1

In multi-constrained problems, the minimum condition becomes

$$\nabla f(x^*) + \sum_{i=1}^{k} \lambda_i \nabla h_i(x^*) = 0. \tag{3.6}$$

♡

Remark

In higher dimensions, the constrained submanifold may possess certain free variables that can be chosen arbitrarily while still satisfying the minimum condition. Specifically, if $\nabla f(x^*)$ cannot be expressed as a linear combination of $\nabla h_1(x^*)$ and $\nabla h_2(x^*)$, it is possible to continue optimizing along the intersection submanifold defined by $h_1 = 0$ and $h_2 = 0$, as shown in Figure 3.2. Therefore, it must be emphasized that the above equation provides only a ***necessary condition****, not a sufficient one.*

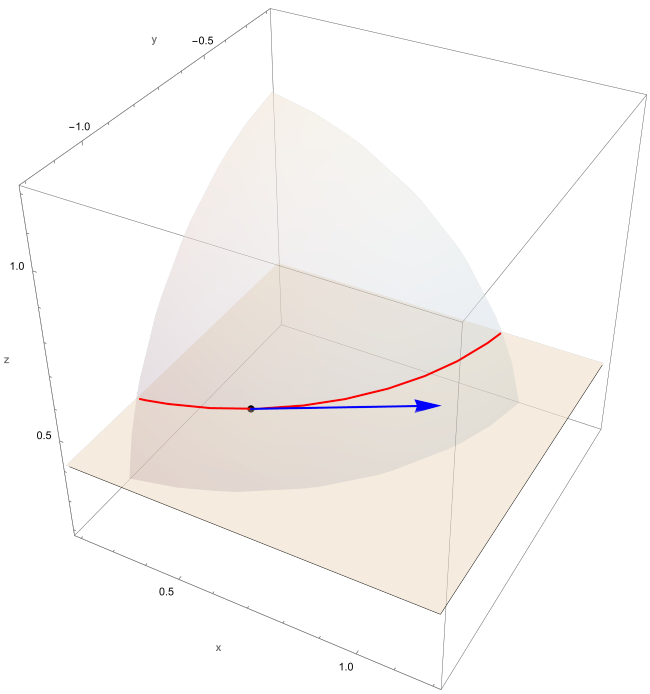


Figure 3.2: Illustration of optimization along the intersection of two constraint manifolds.

Example 3.1

Consider the following constrained optimization problem:

$$\underset{x_1, x_2}{\text{argmin}} \; x_1 + x_2 \tag{3.7}$$

subject to the constraint

$$(x_1 - 1)^2 + x_2^2 - 1 = 0 \tag{3.8}$$

Solution

$1 - \sqrt{2}$.

Note

The Lagrange multiplier method can provide critical points, but these do not always correspond to the minimum solution.

Note

In convex settings, any local minimum is also a global minimum.

A new problem arises: what if we have an inequality constraint instead of a strict equality, i.e., $g(x) \leq 0$?

3.2 Inequality optimization

We assume the feasible set of x is compact; otherwise, the existence of a minimum may be problematic.

By gradient analysis, we obtain:

1. For any point on the surface $g(x) = 0$, $\nabla g(x)$ is orthogonal to the surface and points outward from the feasible region $g(x) \leq 0$. This is straightforward, since $\nabla g(x)$ indicates the steepest direction of increase.
2. For a local minimum point x^*:
   (a). If x^* lies on the surface $g(x) = 0$ (in other words, the constraint is *active*), then $-\nabla f(x^*)$ must be aligned with $\nabla g(x^*)$. Equivalently, $\exists\, \mu > 0$ such that
   $$\nabla f(x^*) + \mu \nabla g(x^*) = 0$$
   (b). If x^* does not lie on the surface, i.e., $g(x) < 0$ (the constraint is *inactive*), we simply require $\nabla f(x^*) = 0$. In the unified form above, this corresponds to $\exists\, \mu = 0$ such that
   $$\nabla f(x^*) + \mu \nabla g(x^*) = 0$$

Note

How should we understand the meaning of an "active" constraint? When the inequality constraint is not tight (i.e., $g(x) < 0$), it does not affect gradient descent optimization. In this case, the constraint is effectively absent, which is why we call it inactive.

Again, we define the Lagrangian as

$$L(x, \mu) = f(x) + \mu g(x) \tag{3.9}$$

where $g(x)$ denotes the inequality constraint function. Following the same steps as in the equality-constrained case, we obtain:

Theorem 3.2

If x^* is a local minimum point, then

$$\begin{cases} \nabla_x L(x^*, \mu) = 0, \\ g(x^*) \leq 0, \\ \mu \geq 0, \\ \mu \cdot g(x^*) = 0. \end{cases} \tag{3.10}$$

The last condition (the *complementary slackness*) unifies the boundary and interior cases, since either $g(x^*) = 0$ (active constraint) or $\mu = 0$ (inactive constraint) must hold.

3.3 General Case

In the most general case, we consider the optimization problem

$$\operatorname*{argmin}_x f(x) \tag{3.11}$$

subject to equality constraints $h_i(x) = 0,\ i \in [k]$, and inequality constraints $g_j(x) \leq 0,\ j \in [l]$.

The corresponding Lagrangian is defined as

$$L(x, \lambda, \mu) := f(x) + \sum_{i \in [k]} \lambda_i h_i(x) + \sum_{j \in [l]} \mu_j g_j(x) \tag{3.12}$$

In this setting, a local minimum x^* must satisfy a series of conditions known as the **Karush–Kuhn–Tucker (KKT) conditions**:

Theorem 3.3 (KKT Conditions)

If x^* is a local minimum point, then there exist multipliers (λ, μ) such that

$$\begin{cases} \nabla_x L(x^*, \lambda, \mu) = 0, & \\ h_i(x^*) = 0, & \forall i \in [k], \\ g_j(x^*) \leq 0, & \forall j \in [l], \\ \mu_j \geq 0, & \forall j \in [l], \\ \mu_j \cdot g_j(x^*) = 0, & \forall j \in [l]. \end{cases} \tag{3.13}$$

♡

In most cases, the above conditions cannot be solved explicitly in closed form.

3.4 Support Vector Machine

3.4.1 Hard Margin

Consider a linearly separable dataset $\{(x_i, y_i)\}$ with $y_i \in \{-1, 1\}$, and a linear model

$$f(x) = w^\top x + b \tag{3.14}$$

Although introducing a regularization term can make the solution unique by adjusting the hyperparameter λ, this does not resolve the issue of determining which solution $f(x; \lambda)$ is preferable among all possible choices, since λ is itself arbitrary.

Our goal is for the model to generalize well to test data drawn from the same distribution as the training set. A direct and intuitive way to achieve this is to choose the separating hyperplane that maximizes the distance between the plane and the nearest datapoints.

Definition 3.1 (Support Vectors)

The datapoints that lie closest to the separating hyperplane, such that the vectors orthogonal to the hyperplane connect the hyperplane to these datapoints, are called support vectors.

♣

In the case of SVM, we aim to find the **Max-Margin Classifier**, i.e., the hyperplane that maximizes the minimum margin across all training points. Here, the *margin* refers to the distance from a point to the hyperplane, which is equivalently the norm of the vector associated with its support vector.

Remark

Why does $w^\top x + b = 0$ represent a hyperplane? Consider the hyperplane through the origin, given by $w^\top x = 0$. Translating this plane until it passes through some point $x_0 \notin \{x \mid w^\top x = 0\}$, we obtain

$$w^\top (x - x_0) = 0$$

Setting $-w^\top x_0 = b$, we recover the general hyperplane equation

$$w^\top x + b = 0$$

Remark

Note that the interpretation of b differs between the two cases, $y = w^\top x + b_f$ and $0 = w^\top x + b_e$. We denote b_f as the bias in the functional form and b_e as the bias in the implicit equation form to distinguish between the two. In the latter case, we observe that $|b_e| = \|w\|\|x_0\|$, which represents the offset determined by the reference point x_0.

The distance between a datapoint and the hyperplane is given by:

Theorem 3.4

$$d = \frac{|w^\top x + b|}{\|w\|}. \tag{3.15}$$

Geometrically, we obtain:

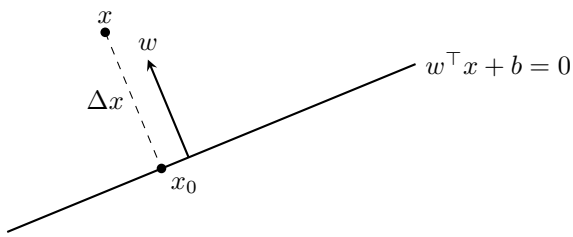


Figure 3.3: Distance from a point to a hyperplane.

Proof

1. *Consider*

$$\begin{cases} x_0 + \Delta x \dfrac{w}{\|w\|} = x, \\ w^\top x_0 + b = 0, \end{cases}$$

so that

$$\begin{aligned} & w^\top x_0 + w^\top \Delta x \frac{w}{\|w\|} = w^\top x \\ \Rightarrow \quad & -b + w^\top \Delta x \frac{w}{\|w\|} = w^\top x \\ \Rightarrow \quad & \Delta x \frac{w^\top w}{\|w\|} = w^\top x + b \\ \Rightarrow \quad & \Delta x \|w\| = w^\top x + b \end{aligned}$$

This matches exactly the expression we obtained in (3.15).

2. *From the Lagrangian perspective:*

$$\operatorname*{argmin}_{x_0} \frac{1}{2}\|x - x_0\|^2 \quad s.t.\ w^\top x_0 + b \quad \Rightarrow \quad L(x_0, \lambda) = \frac{1}{2}\|x - x_0\|^2 + \lambda(w^\top x_0 + b)$$

$$\Rightarrow \quad \begin{cases} \nabla_{x_0} L(x_0, \lambda) & = 0 \quad \Rightarrow \quad x_0 - x + \lambda w = 0 \\ \nabla_\lambda L(x_0, \lambda) & = 0 \quad \Rightarrow \quad w^\top x_0 + b = 0 \end{cases}$$

This leads to the same solution as above.

■

When $\Delta := (w^\top x + b)/\|w\| > 0$, the point x lies on the positive side of the hyperplane; when $\Delta = 0$, x lies exactly on the hyperplane; and when $\Delta < 0$, x lies on the negative side.

We define

$$\gamma_i = \frac{y_i(w^\top x_i + b)}{\|w\|} \tag{3.16}$$

to incorporate both the label information and the margin (distance to the hyperplane). If $\gamma_i > 0$, the point x_i is correctly classified.

Define the margin

$$\gamma = \operatorname*{argmin}_{i \in [n]} \gamma_i \tag{3.17}$$

and the SVM optimization problem can be formalized as

$$\underset{w,b}{\operatorname{argmax}}\ \gamma, \quad \text{s.t. } \forall i,\ \frac{y_i(w^\top x_i + b)}{\|w\|} \geq \gamma \tag{3.18}$$

This formulation is problematic, since γ is not an independent variable but is defined as the minimum of the γ_i's. To resolve this, suppose (x_0, y_0) is a point achieving the minimum margin γ. Then

$$\gamma = \frac{y_0(w^\top x_0 + b)}{\|w\|} \tag{3.19}$$

Thus, the SVM objective can be reformulated as

$$\underset{w,b}{\operatorname{argmax}}\ \frac{y_0(w^\top x_0 + b)}{\|w\|}, \quad \text{s.t. } \forall i,\ y_i(w^\top x_i + b) \geq y_0(w^\top x_0 + b). \tag{3.20}$$

We claim that we can make $y_0(w^\top x_0 + b)$ arbitrarily large without affecting the result.

Remark

This is immediate: scaling w and b by the same factor λ does not change the optimization, since the numerator $y_i(w^\top x_i + b)$ and the denominator $\|w\|$ both scale by λ, and the factor cancels during simplification.

Thus, we may set $y_0(w^\top x_0 + b) = 1$. Define the *functional margin* as $y_i(w^\top x_i + b)$ and the *geometric margin* as $y_i(w^\top x_i + b)/\|w\|$. Since the functional margin alone has no intrinsic meaning, it is natural to rescale it by setting the norm of the support vector to unity.

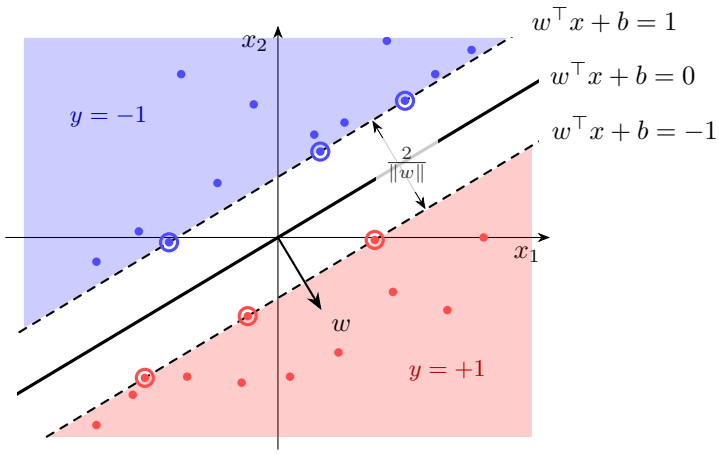


Figure 3.4: Support Vector Machine.

This yields the primal form of the SVM optimization problem:

$$\underset{w,b}{\operatorname{argmax}}\ \frac{1}{\|w\|}, \quad \text{s.t. } y_i(w^\top x_i + b) \geq 1 \quad \forall i \tag{3.21}$$

which is equivalent to

$$\underset{w,b}{\operatorname{argmin}}\ \frac{1}{2}\|w\|^2, \quad \text{s.t. } y_i(w^\top x_i + b) \geq 1 \quad \forall i \tag{3.22}$$

In other words, the goal is to separate the dataset correctly while ensuring a sufficient margin, and simultaneously minimizing the norm of w. This principle is known as **Structural Risk Minimization** (SRM).

Remark

Unlike logistic regression, datapoints far from separating hyperplane do not contribute loss. That is, only small subset of dataset is active as a constraint.

Proposition 3.1

SVM is a convex quadratic programming problem, which can be solved in polynomial time with standard packages. ♡

Problem 3.1

1. What if the dataset is not linearly separable?
2. What if γ too small because of outliers?

3.5 Dual Form Of SVM

3.5.1 Preliminaries from Convex Analysis*

⊛ *This subsection is included out of personal interest and is not part of the course material. Readers who wish to focus solely on the course content may safely skip to the next subsection.*

Given a constrained optimization problem

$$\begin{aligned} \underset{x\in\mathbb{R}^n}{\text{argmin}} \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \le 0, \quad i = 1, \dots, m, \\ & h_j(x) = 0, \quad j = 1, \dots, p, \end{aligned} \tag{3.23}$$

we define the *Lagrangian function* as

$$\mathcal{L}(x, \lambda, \nu) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x) + \sum_{j=1}^{p} \nu_j h_j(x), \tag{3.24}$$

where $\lambda_i \ge 0$ are the Lagrange multipliers associated with inequality constraints, and $\nu_j \in \mathbb{R}$ correspond to equality constraints.

Definition 3.2 (Lagrangian Dual Function)

The dual function *of a lagrangian function is defined by taking the infimum over the primal variable:*

$$\theta(\lambda, \nu) = \inf_x \mathcal{L}(x, \lambda, \nu). \tag{3.25}$$

♣

The function $\theta(\lambda, \nu)$ provides a lower bound on the optimal value of the primal problem, because for any feasible x and $\lambda \ge 0$,

$$\theta(\lambda, \nu) \le f(x). \tag{3.26}$$

Definition 3.3 (Lagrangian Dual Problem)

The dual problem *seeks the tightest possible lower bound on the primal optimum by maximizing $\theta(\lambda, \nu)$:*

$$\begin{aligned} \underset{\lambda,\nu}{\text{argmax}} \quad & \theta(\lambda, \nu) \\ s.t. \quad & \lambda_i \ge 0, \quad i = 1, \dots, m. \end{aligned} \tag{3.27}$$

Equivalently,

$$\underset{\lambda\ge 0,\nu}{\text{argmax}} \ \inf_x \ \mathcal{L}(x, \lambda, \nu). \tag{3.28}$$

♣

This formulation swaps the order of minimization and maximization, producing a problem in terms of the dual variables (λ, ν) instead of x. The resulting problem is always a *concave maximization*, even if the primal is nonconvex.

Theorem 3.5 (Weak Duality)

For any feasible x and any $\lambda \geq 0$, $\nu \in \mathbb{R}^p$, we have

$$\theta(\lambda, \nu) \leq f(x). \tag{3.29}$$

Consequently, if p^* and d^* denote the optimal values of the primal and dual problems respectively, then

$$d^* \leq p^*. \tag{3.30}$$

Proposition 3.2 (Slater's Condition)

Consider the convex optimization problem:

$$\begin{aligned} \underset{x \in \mathbb{R}^n}{\text{argmin}} \quad & f(x) \\ s.t. \quad & g_i(x) \leq 0, \quad i = 1, \ldots, m, \\ & h_j(x) = 0, \quad j = 1, \ldots, p, \end{aligned} \tag{3.31}$$

where f and all g_i are convex functions, and each h_j is affine.
If there exists a point $\bar{x}$ (called a strictly feasible point*) such that*

$$g_i(\bar{x}) < 0, \quad i = 1, \ldots, m, \qquad h_j(\bar{x}) = 0, \quad j = 1, \ldots, p, \tag{3.32}$$

*then the problem satisfies **Slater's condition**.*

Slater's condition ensures that the feasible region of a convex optimization problem is not degenerate. It requires that there exists at least one point that *strictly* satisfies all inequality constraints (that is, $x \in \text{Int}(C)$).

Geometrically, this means the feasible set has a *non-empty interior*. If such a point $\bar{x}$ exists, the constraint surfaces do not touch or overlap in a pathological way, and the optimizer has enough space to operate smoothly. Under this mild regularity assumption, the duality gap between the primal and dual problems vanishes, leading to **strong duality**.

Theorem 3.6 (Strong Duality)

If the primal problem is convex and satisfies the Slater condition *(i.e., there exists a strictly feasible point x such that $g_i(x) < 0$ and $h_j(x) = 0$), then*

$$d^* = p^*. \tag{3.33}$$

That is, the dual problem attains the same optimal value as the primal problem.

Proof

Consider the convex optimization problem:

$$\begin{aligned} \underset{x \in \mathbb{R}^n}{\text{argmin}} \quad & f(x) \\ s.t. \quad & Ax \leq b, \\ & Cx = d, \end{aligned} \tag{3.34}$$

where $f : \mathbb{R}^n \to \mathbb{R} \cup \{+\infty\}$ is a closed and convex function, and the constraints are linear.

The Lagrangian is defined as

$$\mathcal{L}(x, \lambda, \nu) = f(x) + \lambda^\top (Ax - b) + \nu^\top (Cx - d), \qquad \lambda \geq 0. \tag{3.35}$$

The dual function is given by

$$g(\lambda, \nu) = \inf_x \mathcal{L}(x, \lambda, \nu), \tag{3.36}$$

and the dual problem is

$$\underset{\lambda \geq 0, \nu}{\operatorname{argmax}}\ g(\lambda, \nu). \tag{3.37}$$

Weak duality always holds:

$$g(\lambda, \nu) \leq f(x), \qquad \forall x \text{ feasible},\ \lambda \geq 0, \nu. \tag{3.38}$$

Hence, $d^* \leq p^*$.

Now, suppose there exists a strictly feasible point $\bar{x}$ *such that*

$$A\bar{x} < b, \qquad C\bar{x} = d. \tag{3.39}$$

This satisfies the Slater's condition. By the strong duality theorem *in convex optimization (also derived from Fenchel–Rockafellar duality), there exists a saddle point* (x^*, λ^*, ν^*) *such that*

$$\mathcal{L}(x^*, \lambda, \nu) \leq \mathcal{L}(x^*, \lambda^*, \nu^*) \leq \mathcal{L}(x, \lambda^*, \nu^*), \qquad \forall x, \lambda \geq 0, \nu. \tag{3.40}$$

From this saddle-point condition, the minimax equality follows:

$$\underset{x}{\operatorname{argmin}}\ \underset{\lambda \geq 0, \nu}{\operatorname{argmax}}\ \mathcal{L}(x, \lambda, \nu) = \underset{\lambda \geq 0, \nu}{\operatorname{argmax}}\ \underset{x}{\operatorname{argmin}}\ \mathcal{L}(x, \lambda, \nu), \tag{3.41}$$

which implies

$$p^* = d^*. \tag{3.42}$$

Hence, strong duality holds. ■

Remark

Strong duality implies that solving the dual problem is equivalent to solving the primal one, often with additional computational and theoretical advantages, as in the case of Support Vector Machines (SVMs).

Lemma 3.1 (Gordan's Lemma)

Define the function

$$f(x) = \ln \sum_{i=1}^{m} e^{\langle a_i, x \rangle}, \tag{3.43}$$

where $\{a_i\}_{i=1}^m$ *are vectors in* $\mathbb{R}^n$*. The following statements are equivalent:*

(a) $f(x)$ *is bounded from below;*

(b) there exists $\lambda \in \mathbb{R}^m$, $\lambda \geq 0$, $\lambda \neq 0$, *such that*

$$\sum_{i=1}^{m} \lambda_i a_i = 0; \tag{3.44}$$

(c) there exists no x *satisfying*

$$\langle a_i, x \rangle < 0, \quad i = 1, \ldots, m. \tag{3.45}$$

♡

Moreover, the result can be generalized to a broader class of problems that also involve equality constraints.

Theorem 3.7 (Motzkin's transposition theorem)

Let $\{a_i\}_{i=1}^l$, $\{b_j\}_{j=1}^m$, *and* $\{c_k\}_{k=1}^p$ *be vectors in* $\mathbb{R}^n$, *and let* $\{\alpha_i\}_{i=1}^l$, $\{\beta_j\}_{j=1}^m$, *and* $\{\gamma_k\}_{k=1}^p$ *be scalars. Then the*

linear system

$$\begin{aligned} \langle a_i, x\rangle &< \alpha_i, \quad i = 1, \dots, l, \\ \langle b_j, x\rangle &\le \beta_j, \quad j = 1, \dots, m, \\ \langle c_k, x\rangle &= \gamma_k, \quad k = 1, \dots, p, \end{aligned} \tag{3.7}$$

is inconsistent if and only if there exist vectors (multipliers)

$$\lambda_0 \in \mathbb{R}, \quad \lambda \in \mathbb{R}^l, \quad \mu \in \mathbb{R}^m, \quad \delta \in \mathbb{R}^p, \tag{3.46}$$

satisfying

$$\begin{gathered} \sum_{i=1}^{l} \lambda_i a_i + \sum_{j=1}^{m} \mu_j b_j + \sum_{k=1}^{p} \delta_k c_k = 0, \\ \sum_{i=1}^{l} \lambda_i \alpha_i + \sum_{j=1}^{m} \mu_j \beta_j + \sum_{k=1}^{p} \delta_k \gamma_k + \lambda_0 = 0, \\ (\lambda_0, \lambda, \mu) \ge 0, \qquad (\lambda_0, \lambda) \ne 0. \end{gathered} \tag{3.8}$$

Definition 3.4 (Hilbert Space)

*A **Hilbert space** is a complete inner product space.*

3.5.2 Dual Theory in the SVM Problem

Under the intuition provided by the Lagrangian formulation, we can define the primal objective as

$$p^* = \operatorname*{argmin}_{w,b} \operatorname*{argmax}_{\alpha \ge 0} L(w, b, \alpha), \tag{3.47}$$

where the Lagrangian L is defined as

$$L(w, b, \alpha) = \frac{1}{2}\|w\|^2 + \sum_{i\in[n]} \alpha_i (1 - y_i(w^\top x_i + b)) = \frac{1}{2}\|w\|^2 + \sum_{i\in[n]} \alpha_i - \sum_{i\in[n]} \alpha_i y_i (w^\top x_i + b). \tag{3.48}$$

We claim that:

Proposition 3.3

p^* corresponds to the solution of the primal SVM problem.

To justify this, suppose the claim were false. Assume that some constraint is not satisfied, that is, there exists an index i violating the constraint such that

$$1 - y_i(w^\top x_i + b) > 0. \tag{3.49}$$

Then the inner maximization problem

$$\theta_p(w, b) := \operatorname*{argmax}_{\alpha \ge 0} L(w, b, \alpha) \tag{3.50}$$

can be driven to infinity, since the corresponding Lagrange multiplier $\alpha_i > 0$ can be increased arbitrarily to make $L(w, b, \alpha)$ unbounded above. Consequently,

$$\operatorname*{argmin}_{w,b} \theta_p(w, b) = \operatorname*{argmin}_{w,b} \infty \tag{3.51}$$

has no valid solution. Therefore, the initial assumption must be false, and the optimal dual variables α^* enforce all constraints to hold with equality or non-violation. Hence, the dual solution indeed corresponds to the optimal solution of the primal problem.

Consider a convex region defined by the constraint set C. We claim that for all $x_i \in \mathrm{Int}(C)$, that is, points located strictly inside the feasible region where all constraints are satisfied, the corresponding α_i is always set to 0. For all $x_i \in \partial C = \bar{C} - \mathrm{Int}(C)$, where the constraints become active, α_i may take positive values.

As a conclusion, regardless of which region the data points fall into, the following condition holds at the optimal solution due to the complementary slackness property in the KKT conditions:

$$\sum_{i \in [n]} \alpha_i \big(1 - y_i (w^\top x_i + b)\big) = 0. \tag{3.52}$$

This does not represent an algebraic identity but a structural condition satisfied at optimality. It implies that only the points lying exactly on the margin (the support vectors) contribute to the decision boundary, while all others have $\alpha_i = 0$. Therefore, at the optimum, the objective effectively reduces to minimizing $\|w\|^2$.

Define the dual problem of p^* as follows:

$$d^* = \operatorname*{argmax}_{\alpha \geq 0} \ \operatorname*{argmin}_{w,b} L(w, b, \alpha), \tag{3.53}$$

where p^* is a min–max problem, while the dual problem is formulated as a max–min problem.

Theorem 3.8 (Weak Duality)

$$d^* \leq p^*. \tag{3.54}$$

Proof

By the definition of the argmin *(or equivalently, the infimum), we have*

$$d^* = \operatorname*{argmax}_{\alpha \geq 0} \ \operatorname*{argmin}_{w,b} L(w, b, \alpha) \ \leq \ \operatorname*{argmax}_{\alpha \geq 0} \ L(w, b, \alpha) \tag{3.55}$$

for every (w, b).

We may then choose the best (w, b) *that minimizes the right-hand side, i.e.,*

$$d^* \leq \operatorname*{argmin}_{w,b} \ \operatorname*{argmax}_{\alpha \geq 0} L(w, b, \alpha) = p^*. \tag{3.56}$$

■

Weak duality always holds because no additional assumptions or constraints have been introduced in the argument above.

Here is another proof of the weak duality theorem:

Proof

Assume that (w_p, b_p) *is the optimal solution to the primal problem, and that* α_d *is the optimal solution to the dual problem. Then we have*

$$\begin{aligned} p^* &= \operatorname*{argmin}_{w,b} \ \operatorname*{argmax}_{\alpha \geq 0} L(w, b, \alpha) = \operatorname*{argmax}_{\alpha \geq 0} \ L(w_p, b_p, \alpha) \\ &\geq L(w_p, b_p, \alpha_d) \geq \operatorname*{argmin}_{w,b} \ L(w, b, \alpha_d) \\ &= \operatorname*{argmax}_{\alpha \geq 0} \ \operatorname*{argmin}_{w,b} \ L(w, b, \alpha) = d^*. \end{aligned}$$

■

Remark

We study the dual problem because, in many cases, it is analytically or computationally easier to solve than the original primal problem.

Weak duality only constrains the lower bound of the original problem. Is there a better situation for d^* if we include more conditions? The answer is yes. Click here for more information.

We claim that, for convex objectives, if all the constraints are linear functions, then the **Strong Duality** holds, where

$$d^* = p^*. \tag{3.57}$$

Remark

For completeness, I provide a brief proof in the previous subsection.

In the strong duality case, the inequalities in the second proof of the weak duality theorem become equalities. That is, the intermediate solutions (w, b) in the dual problem are also optimal for the primal problem. Equivalently, the solutions of the dual and primal problems are **consistent** under strong duality.

Now, lets explicitly solve the dual problem under strong duality.

Solution

$$\operatorname*{argmax}_{\alpha\geq 0} \operatorname*{argmin}_{w,b} L(w,b,\alpha) = \operatorname*{argmax}_{\alpha\geq 0} \operatorname*{argmin}_{w,b} \left(\frac{1}{2}\|w\|^2 + \sum_{i\in[n]} \alpha_i(1 - y_i(w^\top x_i + b))\right) \tag{3.58}$$

To find the inner minimization over w, b, we apply the KKT stationarity conditions:

$$\frac{\partial L}{\partial w} = w - \sum_{i\in[n]} \alpha_i y_i x_i = 0 \Rightarrow \quad w = \sum_{i\in[n]} \alpha_i y_i x_i, \tag{3.59}$$

$$\frac{\partial L}{\partial b} = -\sum_{i\in[n]} \alpha_i y_i = 0 \Rightarrow \quad \sum_{i\in[n]} \alpha_i y_i = 0. \tag{3.60}$$

Substitute back into $L(w, b, \alpha)$. Plugging $w = \sum_i \alpha_i y_i x_i$ into L, we obtain:

$$L(w,b,\alpha) = \frac{1}{2}\|w\|^2 + \sum_{i\in[n]} \alpha_i(1 - y_i(w^\top x_i + b)) \tag{3.61}$$

$$= \frac{1}{2}\left\|\sum_{i\in[n]} \alpha_i y_i x_i\right\|^2 + \sum_{i\in[n]} \alpha_i - \sum_{i\in[n]} \alpha_i y_i \left(\sum_{j\in[n]} \alpha_j y_j x_j^\top x_i + b\right) \tag{3.62}$$

$$= \frac{1}{2}\sum_{i\in[n]} \alpha_i y_i x_i^\top \sum_{j\in[n]} \alpha_j y_j x_j + \sum_{i\in[n]} \alpha_i - \sum_{i\in[n]} \alpha_i y_i \left(\sum_{j\in[n]} \alpha_j y_j x_j^\top x_i + b\right) \tag{3.63}$$

which follows the constraints:

$$\alpha \geq 0, \quad \sum_i \alpha_i y_i = 0 \tag{3.64}$$

Simplify L using $\sum_i \alpha_i y_i = 0$, we have

$$L(w,b,\alpha) = \sum_{i\in[n]} \alpha_i - \frac{1}{2}\sum_{i\in[n]}\sum_{j\in[n]} \alpha_i \alpha_j y_i y_j x_i^\top x_j. \tag{3.65}$$

Thus, the SVM dual problem becomes:

$$\boxed{\begin{aligned} \underset{\alpha \geq 0}{\operatorname{argmax}} \quad & W(\alpha) = \sum_{i \in [n]} \alpha_i - \frac{1}{2} \sum_{i,j \in [n]} \alpha_i \alpha_j y_i y_j x_i^\top x_j, \\ s.t. \quad & \sum_{i \in [n]} \alpha_i y_i = 0. \end{aligned}} \tag{3.66}$$

Let α^* be the optimization solution of the uppon problem. Under strong duality, the optimal α^* gives rise to the optimal w^* and b^* through:

$$w^* = \sum_{i \in [n]} \alpha_i^* y_i x_i, \qquad b^* = y_i - w^{*\top} x_i \quad \textit{for any support vector } \alpha_i^* > 0. \tag{3.67}$$

We can confirm whether (x_i, y_i) is support vector by checking whether α_i^* is positive (o.w. is zero). Thus:

$$w^* = \sum_{i \in \text{S.V.s}} \alpha_i^* y_i x_i, \qquad b^* = y_i - w^{*\top} x_i \quad \textit{for any support vector } \alpha_i^* > 0. \tag{3.68}$$

Note

At this point, the reasoning effectively reproduces Gordan's Lemma within the derivation process. I provided a brief discussion in the previous subsection.

Thus, an SVM model can be fully reconstructed from its support vectors and their corresponding coefficients α_i^*. For any test point x, the prediction depends solely on the support vectors and their weights:

$$f(x) = w^{*\top} x + b^* = \sum_{i \in \text{S.V.}} \alpha_i^* y_i \, (x_i^\top x) + b^* > 0. \tag{3.69}$$

The red term in the equation above measures the *similarity* between the test point x and each support vector x_i.

To capture non-linear relationships, we can replace this inner product with a more general **kernel function** $K(x_i, x)$, thereby mapping the data into a higher-dimensional feature space while preserving computational efficiency. We will discuss kernel tricks in the next section.

We now introduce the practical algorithm commonly used in training SVMs: the **Sequential Minimal Optimization** (SMO) method. The main idea is to iteratively update a small subset of dual variables α_i while keeping all others fixed.

However, because of the equality constraint

$$\sum_{i=1}^{n} \alpha_i y_i = 0, \tag{3.70}$$

if we fix $(n-1)$ of the α_j, the remaining one α_i is automatically determined, leaving no degree of freedom. To overcome this, SMO randomly selects a pair (α_i, α_j) and optimizes them jointly while keeping all other α_k (for $k \neq i, j$) fixed. Under this setup, the equality constraint becomes

$$\alpha_i y_i + \alpha_j y_j = -\sum_{k \neq i,j} \alpha_k y_k = \text{constant}. \tag{3.71}$$

Hence, α_j can be expressed as a linear function of α_i, and the original optimization problem reduces to a one-dimensional quadratic programming problem in α_i, which is convex (positive semidefinite).

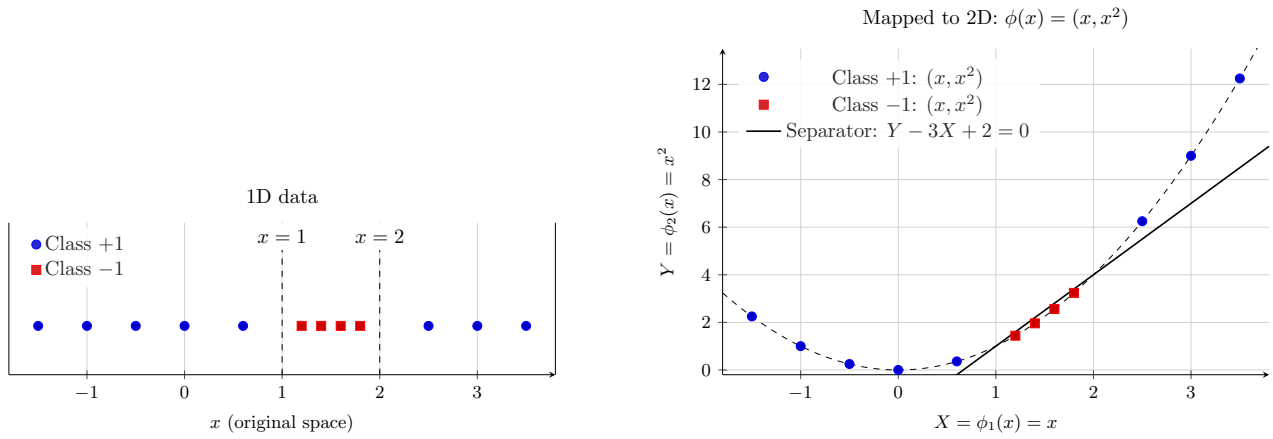
This reduction admits a **closed-form solution**, allowing efficient updates of the chosen pair (α_i, α_j).

By repeating this process iteratively over different pairs, the algorithm converges to the optimal solution of the dual problem.

3.6 Kernel

In the previous section, we only considered linearly separable datasets, which is not always satisfied in general cases.

For example, consider a one-dimensional separation problem that cannot be separated by a single point (a one-dimensional plane); we can introduce an extra dimension to transform the dataset into a linearly separable one.



That is, we want to find a function φ that maps $\mathbb{R}^d \mapsto \mathbb{R}^{d'}$ to help the separation machine complete its work. For instance, for a two-dimensional input (x_1, x_2), we can define the mapping

$$\varphi(x) := (1, x_1, x_2, x_1^2, x_2^2, x_1 x_2) \in \mathbb{R}^6, \tag{3.72}$$

which makes several implicit features in $\mathbb{R}^2$ explicit.

We can then calculate the similarity between x_1 and x_2 through their transformed features $\varphi(x)$ in (3.69):

$$f(x) = w^{*\top} x + b^* = \sum_{i \in \text{S.V.}} \alpha_i^* y_i (\varphi(x_i)^\top \varphi(x)) + b^* > 0. \tag{3.73}$$

Correspondingly, for (3.65), the dual objective becomes

$$L = \sum_{i \in [n]} \alpha_i - \frac{1}{2} \sum_{i,j \in [n]} \alpha_i \alpha_j y_i y_j \langle \varphi(x_i), \varphi(x_j) \rangle. \tag{3.74}$$

Calculating the inner product in the high-dimensional feature space may be computationally expensive, as it often requires complex and high-order operations. Therefore, we define a **kernel function** that allows us to compute the inner product in the high-dimensional space indirectly, using only the original input vectors:

$$K(x_i, x_j) := \langle \varphi(x_i), \varphi(x_j) \rangle. \tag{3.75}$$

Definition 3.5 (Kernel Function)

*A function $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is called a **kernel function** if there exists a Hilbert space $\mathcal{H}$ and a mapping*

$$\phi : \mathcal{X} \to \mathcal{H} \tag{3.76}$$

such that for all $x, x' \in \mathcal{X}$,

$$K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}. \tag{3.77}$$

♣

In the previous example, the kernel corresponding to $\varphi(x)$ can be written as

$$K(x, z) = (x^\top z + 1)^2. \tag{3.78}$$

We can easily verify this claim:

$$\begin{aligned} K(x,z) &= (x_1z_1 + x_2z_2 + 1)^2 \\ &= \langle (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2),\ (1, \sqrt{2}z_1, \sqrt{2}z_2, z_1^2, z_2^2, \sqrt{2}z_1z_2)\rangle. \end{aligned}$$

Thus, we have effectively reduced the computational complexity from $O(d')$ to $O(d)$, thereby saving computational resources.

The dual formulation in (3.74) can then be simplified using the kernel function:

$$L = \sum_{i\in[n]} \alpha_i - \frac{1}{2} \sum_{i,j\in[n]} \alpha_i\alpha_j y_i y_j K(x_i, x_j) \tag{3.79}$$

We now consider a new question: what kind of function K can be regarded as a valid kernel? Mercer's Theorem provides the precise answer to this problem.

Theorem 3.9 (Mercer's Theorem (Finite case))

A symmetric function $K(x,x')$ is a valid kernel if and only if, for any finite set $\{x_1,\dots,x_n\} \subset \mathcal{X}$, the Gram matrix

$$K_{ij} = K(x_i, x_j) \tag{3.80}$$

is positive semidefinite, i.e.

$$\forall c \in \mathbb{R}^n, \quad c^\top K c \geq 0. \tag{3.81}$$

Proof

In the finite-dimensional case.Let $\{x_1,\dots,x_n\} \subset \mathcal{X}$ be any finite subset, and define the Gram matrix $K \in \mathbb{R}^{n\times n}$ by

$$K_{ij} = K(x_i, x_j), \qquad i,j = 1,\dots,n. \tag{3.82}$$

Since K is symmetric, we have $K = K^\top$.

By the spectral theorem for real symmetric matrices, there exists an orthonormal basis of eigenvectors $\{v_1,\dots,v_n\}$ and corresponding real eigenvalues $\lambda_1,\dots,\lambda_n$ such that

$$K = V\Lambda V^\top, \tag{3.83}$$

where $V = [v_1,\dots,v_n]$ is orthogonal and $\Lambda = \mathrm{diag}(\lambda_1,\dots,\lambda_n)$.

The kernel K being positive semidefinite implies that for all $c \in \mathbb{R}^n$,

$$c^\top K c = \sum_{i,j} c_i K(x_i, x_j) c_j \geq 0, \tag{3.84}$$

which in turn implies $\lambda_i \geq 0$ for all i.

Now, define feature functions

$$\phi_i(x_j) = \sqrt{\lambda_i}\, v_i(j), \tag{3.85}$$

i.e. the j-th component of v_i scaled by $\sqrt{\lambda_i}$. Then for any $x_p, x_q \in \{x_1,\dots,x_n\}$,

$$K(x_p, x_q) = K_{pq} = \sum_{i=1}^{n} \lambda_i v_i(p) v_i(q) = \sum_{i=1}^{n} \phi_i(x_p)\,\phi_i(x_q). \tag{3.86}$$

Hence, in the finite-dimensional case, K can be written as an inner product

$$K(x_p, x_q) = \langle \phi(x_p), \phi(x_q)\rangle, \qquad \phi(x) = (\phi_1(x),\dots,\phi_n(x))^\top. \tag{3.87}$$

This directly establishes the Mercer expansion in the finite case, and the general (infinite-dimensional) theorem follows by taking the limit as $n \to \infty$. ■

Theorem 3.10 (Mercer's Theorem)

Let $\mathcal{X} \subset \mathbb{R}^n$ be a compact set, and let $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a continuous, symmetric function.
*Then K is a **kernel** if and only if there exists an orthonormal basis $\{\phi_i\}_{i=1}^{\infty}$ of $L^2(\mathcal{X})$ and a sequence of non-negative eigenvalues $\{\lambda_i\}_{i=1}^{\infty}$, $\lambda_i \geq 0$, such that for all $x, x' \in \mathcal{X}$,*

$$K(x, x') = \sum_{i=1}^{\infty} \lambda_i \, \phi_i(x) \, \phi_i(x'), \tag{3.88}$$

where the series converges absolutely and uniformly on $\mathcal{X} \times \mathcal{X}$. That is, K is positive semi-definite.
Equivalently, for any square-integrable function $f \in L^2(\mathcal{X})$,

$$\iint_{\mathcal{X}\times\mathcal{X}} f(x) K(x, x') f(x') \, dx \, dx' = \sum_{i=1}^{\infty} \lambda_i \Big(\int_{\mathcal{X}} f(x) \phi_i(x) \, dx \Big)^2 \geq 0. \tag{3.89}$$

♡

Here are some commonly used kernels:

1. **Linear kernel**: simply applies the inner product in the original space,

$$K(x, z) = x^\top z; \tag{3.90}$$

2. **Polynomial kernel**: can be understood as mapping $\mathbb{R}^n$ into a feature space of dimension $\binom{n+p}{p}$,

$$K(x, z) = (x^\top z + 1)^p; \tag{3.91}$$

3. **Gaussian / RBF kernel (Radial Basis Function)**:

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2} \right). \tag{3.92}$$

Remark

The RBF kernel can be intuitively understood as measuring the similarity between two vectors based on their Euclidean distance.

The RBF kernel makes any dataset linearly separable, as it effectively maps x and z into an **infinite-dimensional** Hilbert space before taking their inner product, thus providing a powerful tool for nonlinear classification.

This property follows directly from the Taylor expansion of the exponential function, noting that the polynomial basis is complete.

Remark

$$\textit{Taylor Expansion:} \quad f(x) = f(0) + f'(0)\, x + \frac{1}{2!} f''(0)\, x^2 + \frac{1}{3!} f^{(3)}(0)\, x^3 + \cdots \tag{3.93}$$

$$k(x, z) = \exp\left(-\frac{\|x\|^2}{2\sigma^2} \right) \exp\left(-\frac{\|z\|^2}{2\sigma^2} \right) \exp\left(\frac{x^\top z}{\sigma^2} \right) \tag{3.94}$$

$$= \exp\left(-\frac{\|x\|^2}{2\sigma^2} \right) \exp\left(-\frac{\|z\|^2}{2\sigma^2} \right) \left[1 + \frac{x^\top z}{\sigma^2} + \frac{1}{2!} \left(\frac{x^\top z}{\sigma^2} \right)^2 + \frac{1}{3!} \left(\frac{x^\top z}{\sigma^2} \right)^3 + \cdots \right] \tag{3.95}$$

The only hyperparameter introduced here is σ^2, which determines the rate at which higher-order polynomial terms decay. A larger σ^2 implies a faster decay of these terms. Thus, σ^2 effectively controls the dimensionality of the feature space involved in the inner product.

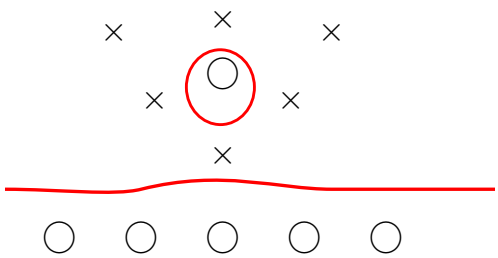
Proposition 3.4

The RBF kernel enables nonlinear classification.

♡

The above proposition follows from the behavior of the RBF kernel as $\sigma^2 \to 0^+$. In this limit, only the data point x_i closest to x contributes significantly to the kernel value, since its exponential decay is the slowest. In other words, the RBF kernel behaves almost like a **1-NN (nearest neighbor)** classifier in this regime.

To demonstrate the extraordinary capability of the RBF kernel, we note that it can represent **multiply connected**, or even **disconnected**, decision regions:



However, this expressive power also comes with a significant risk of **overfitting**.

We have now solved the non-linearly separable problem mentioned at the end of the last section, and we arrive at the next challenge: the presence of outliers.

To address this issue, we introduce the concept of **slack variables**. The idea is intuitive yet powerful: instead of forcing every data point to lie strictly on the correct side of the margin, we allow some samples to *violate* the margin constraint slightly, while penalizing these violations in the objective function.

Formally, we define non-negative variables

$$\xi_i \geq 0, \quad i = 1, \ldots, n, \tag{3.96}$$

called *slack variables*, representing the amount by which each sample fails to satisfy the margin condition. The optimization problem then becomes

$$\begin{aligned} \underset{w,b,\xi}{\text{argmin}} \quad & \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i, \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad i = 1, \ldots, n, \\ & \xi_i \geq 0, \quad i = 1, \ldots, n. \end{aligned} \tag{3.97}$$

where $C > 0$ is a hyperparameter controlling the trade-off between maximizing the margin and minimizing classification errors.

In this way, the model, known as the **Soft-Margin SVM**, allows small violations of the margin to achieve a better balance between robustness and generalization. This is the most commonly used form of SVM in practice.

We claim that the constraints on the slack variables are equivalent to the following formulation:

$$\xi_i \geq \max(0,\ 1 - y_i(w^\top x_i + b)). \tag{3.98}$$

In the minimization problem

$$\underset{w,b,\xi}{\text{argmin}} \ \frac{1}{2}\|w\|^2 + C\sum_{i}\xi_i, \tag{3.99}$$

it is crucial to observe that each ξ_i will always take the minimal feasible value, either 0 or $1 - y_i(w^\top x_i + b)$. Otherwise, if ξ_i were larger than necessary, the objective $\sum_i \xi_i$ could be further reduced by decreasing ξ_i.

Thus, the optimization problem can be reformulated as an unconstrained objective:

$$\underset{w,b}{\text{argmin}} \ \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\max(0,\ 1 - y_i(w^\top x_i + b)). \tag{3.100}$$

This formulation introduces the **Hinge loss**.

Definition 3.6 (Hinge loss)

The hinge loss function $L(z)$ is defined as:

$$L(z) = \max(0,\ 1 - z). \tag{3.101}$$

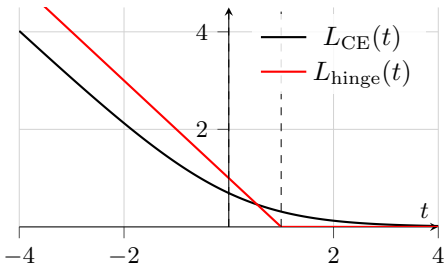


Figure 3.5: Hinge loss and CE loss.

Intuitively, the hinge loss $L_{\mathrm{hinge}}(t)$ remains zero when $t \geq 1$. This property directly enforces a margin: samples that already lie beyond the decision boundary (i.e., sufficiently far from the separating hyperplane) incur no loss, while those within or on the wrong side of the margin contribute a positive penalty. In this sense, the hinge loss explicitly encourages a wider margin between classes.

For comparison, consider the cross-entropy loss:

$$-\log(\sigma(z)) = \log(1 + e^{-z}), \tag{3.102}$$

where $\sigma(z)$ denotes the sigmoid function.

Remark

In class, we used $\log_2$ instead of the natural logarithm, leading to an approximation when $x \to -\infty$:

$$\log(1 + e^{-z}) \sim \frac{-z}{\log_2 e}, \tag{3.103}$$

which results in a slightly different scaling in the plotted curve.

Both the cross-entropy loss and the hinge loss serve as *substitute losses* for the zero-one loss:

$$L_{\text{zero-one}}(z) = \begin{cases} 1, & z < 0, \\ 0, & z \geq 0. \end{cases} \tag{3.104}$$

The zero-one loss is non-differentiable and provides no gradient information, so gradient-based optimization methods cannot be directly applied in this case.

Note

A training point is a support vector if and only if $\alpha_i > 0$. Indeed, by the KKT conditions, $\alpha_i > 0$ implies the constraint is active, so the point lies on or inside the margin. Even when $\xi_i > 0$ (margin violation or misclassification), it still contributes to

$$w = \sum_i \alpha_i y_i \phi(x_i),$$

and thus affects the decision boundary. The existence of a slack variable does not exclude a point from being a support vector.

Chapter 4 Representer Theorem

Introduction

- *Representer Theorem*
- *Dual form of Linear Regression*

4.1 Revisit the Preliminaries

We first provide a brief overview of the previous chapters on Support Vector Machines (SVM). The introduction of slack variables $\xi_i \geq 0,\ \forall i \in [n]$ allows the model to tolerate outliers or samples that do not strictly follow the shared distribution. However, it is important to properly scale the penalty on the slack variables, since inappropriate choices of this scaling factor C may severely affect the model's generalization ability.

The primal optimization problem of the soft-margin SVM can be written as:

$$\begin{aligned} &\operatorname*{argmin}_{w,b,\xi} \ \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i \in [n]. \end{aligned}$$

We can further express the decision function of the SVM as:

$$f(x) = w^\top \varphi(x), \tag{4.1}$$

where $\varphi(x)$ denotes a feature mapping that transforms the input x into a (possibly higher-dimensional) feature space.

The objective function can then be rewritten as:

$$\operatorname*{argmin}_{w} \sum_{i\in[n]} \mathrm{Hinge}\big(f(x_i)\big) + \frac{\lambda}{2}\|w\|^2. \tag{4.2}$$

Remark

There is a minor difference between the above expression and the original SVM formulation: the bias term b can be absorbed into w by augmenting the input vector with an additional dimension (e.g., appending $x_0 = 1$).

By rewriting the $\mathrm{Hinge}\big(f(x_i)\big)$ term in terms of the slack variables ξ_i, we can derive the dual formulation of the soft-margin SVM:

$$\operatorname*{argmax}_{\alpha\geq 0,\beta\geq 0} \ \operatorname*{argmin}_{w,b} \left[\sum_{i\in[n]}\xi_i + \frac{\lambda}{2}w^\top w + \sum_{i\in[n]}\alpha_i\big(1 - y_i w^\top\varphi(x_i) - \xi_i\big) - \sum_{i\in[n]}\beta_i\xi_i\right]. \tag{4.3}$$

Here, α_i, β_i are both Lagrange multiplier. The gradient of the uppon targer brings:

$$\left.\begin{aligned} \partial_w L = 0 \\ \partial_{\xi_i} L = 0 \end{aligned}\right\} \quad \Rightarrow \quad \begin{cases} w = \dfrac{1}{\lambda}\displaystyle\sum_{i\in[n]}\alpha_i y_i \varphi(x_i) \\ \alpha_i + \beta_i = 1 \end{cases} \tag{4.4}$$

Bring back to (4.3):

$$\text{Dual}: \frac{1}{2}\lambda\left(\frac{1}{\lambda}\right)^2 \sum_{i,j\in[n]}\alpha_i\alpha_j y_i y_j \varphi^\top(x_i)\varphi(x_j) - \sum_{i\in[n]}\alpha_i y_i \varphi^\top(x_i)\frac{1}{\lambda}\sum_{j\in[n]}\alpha_j y_j \varphi(x_j) + \sum_{i\in[n]}\alpha_i \tag{4.5}$$

By simplification and kernel tricks:

$$\text{Dual}: \underset{\alpha}{\operatorname{argmax}} -\frac{1}{2\lambda} \sum_{i,j\in[n]} \alpha_i \alpha_j y_i y_j k(x_i, x_j) + \sum_{i\in[n]} \alpha_i \quad s.t.\ 0 \le \alpha_i \le 1 \tag{4.6}$$

It can be observed that the only difference between the dual problems of the soft-margin and hard-margin SVMs lies in the constraint on α_i. In the soft-margin formulation, each α_i is required to satisfy $\alpha_i \le 1$ (which is equivalent to introducing $\beta_i \ge 0$ and enforcing $\alpha_i + \beta_i = 1$). This constraint limits the influence of any single support vector, preventing the model from overemphasizing outliers or noisy samples.

Ultimately, the prediction function $f(x)$ can be reformulated as a linear combination of kernel evaluations between the input point x and the training data points x_i:

$$f(x) = \sum_{i=1}^{n} \alpha_i\, k(x_i, x), \tag{4.7}$$

where $k(x_i, x) = \varphi(x_i)^\top \varphi(x)$ denotes the kernel function.

In the previous part, we introduced the notion of the *feature space*. We now provide a new example, the **Ridge Regression**, to demonstrate that this concept is widely used in the field of machine learning.

The objective function of Ridge Regression is:

$$\underset{w}{\operatorname{argmin}} \frac{1}{2} \sum_{i\in[n]} \big(y_i - w^\top \varphi(x_i)\big)^2 + \frac{1}{2}\lambda\, w^\top w =: L(w) \tag{4.8}$$

Taking the derivative with respect to w gives:

$$\frac{\partial L}{\partial w} = -\sum_{i\in[n]} \big(y_i - w^\top \varphi(x_i)\big)\varphi(x_i) + \lambda w = 0 \tag{4.9}$$

Rearranging terms yields:

$$w = \frac{1}{\lambda} \sum_{i\in[n]} \big(y_i - w^\top \varphi(x_i)\big)\varphi(x_i) \tag{4.10}$$

We can rewrite this as:

$$w = \sum_{i\in[n]} \alpha_i\, \varphi(x_i) \tag{4.11}$$

where $\alpha_i = \frac{1}{\lambda}\big(y_i - w^\top \varphi(x_i)\big)$.

Substituting this representation into the prediction function gives:

$$f(x) = w^\top \varphi(x) = \sum_{i\in[n]} \alpha_i\, k(x_i, x) \tag{4.12}$$

where $k(x_i, x) = \varphi(x_i)^\top \varphi(x)$ is the kernel function.

The shared form also occurs in the logistic regression problem.

The objective function of regularized logistic regression is:

$$\min_{w} \sum_{i\in[n]} \log\big(1 + e^{-y_i w^\top \varphi(x_i)}\big) + \frac{1}{2}\lambda\, w^\top w \tag{4.13}$$

Taking the derivative with respect to w gives:

$$\frac{\partial L}{\partial w} = \sum_{i\in[n]} \frac{e^{-y_i w^\top \varphi(x_i)}}{1 + e^{-y_i w^\top \varphi(x_i)}} (-y_i)\varphi(x_i) + \lambda w = 0 \tag{4.14}$$

Rearranging the terms, we can express w as:

$$w = \sum_{i\in[n]} \alpha_i\, \varphi(x_i) \tag{4.15}$$

where

$$\alpha_i = \frac{1}{\lambda} \frac{y_i e^{-y_i w^\top \varphi(x_i)}}{1 + e^{-y_i w^\top \varphi(x_i)}}. \tag{4.16}$$

Substituting this representation into the prediction function gives:

$$f(x) = w^\top \varphi(x) = \sum_{i \in [n]} \alpha_i \, k(x_i, x), \tag{4.17}$$

The above observation suggests a deeper connection between *representation* and the *kernel trick*. It shows that all the regression models we have discussed so far can be interpreted as special cases of the same underlying idea.

4.2 Representer Theorem

We have already defined the concept of a *Hilbert space* in Definition 3.4. Building upon that foundation, we now introduce the notion of a **Reproducing Kernel Hilbert Space (RKHS)**, which extends the idea of inner product spaces to functional spaces, allowing the kernel trick to be formally grounded in Hilbert space theory.

Definition 4.1 (Reproducing Kernel Hilbert Space (RKHS))

Let $\mathcal{H}$ be a Hilbert space of functions $f : \mathcal{X} \to \mathbb{R}$ defined on a nonempty set $\mathcal{X}$. If there exists a function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ such that:

1. *For every $x \in \mathcal{X}$, the function $k(\cdot, x) \in \mathcal{H}$.*
2. *For every $f \in \mathcal{H}$ and every $x \in \mathcal{X}$, the* reproducing property *holds:*

$$f(x) = \langle f, \, k(\cdot, x) \rangle_{\mathcal{H}}, \tag{4.18}$$

*then $\mathcal{H}$ is called a **Reproducing Kernel Hilbert Space**, and k is called its **reproducing kernel**.* ♣

Remark

*In fact, the above definition is quite intuitive once we recall that a Hilbert space differs from a general vector space primarily in its **completeness** under an **inner product**. In other words, a Hilbert space is a complete inner product space, which extends the familiar geometric structure of finite-dimensional Euclidean spaces to infinite dimensions.*

Note

The completeness of a Reproducing Kernel Hilbert Space (RKHS) refers to the property that every Cauchy sequence in the space converges to a limit that still lies within the space. In geometric terms, this means the RKHS is "closed under limits" with respect to its inner-product-induced norm.

Moreover, by a fundamental result in Hilbert space theory, every separable Hilbert space admits a complete orthonormal basis. *Hence, an RKHS possesses (possibly infinite-dimensional) orthonormal basis functions $\{e_i\}_{i=1}^{\infty}$ such that any function $f \in \mathcal{H}$ can be expressed as*

$$f = \sum_{i=1}^{\infty} \langle f, e_i \rangle_{\mathcal{H}} \, e_i, \tag{4.19}$$

where the series converges in the Hilbert norm. In the context of kernel methods, Mercer's theorem ensures that the kernel function itself can be decomposed as

$$k(x, x') = \sum_{i=1}^{\infty} \lambda_i \, \phi_i(x) \phi_i(x'), \tag{4.20}$$

where $\{\sqrt{\lambda_i} \, \phi_i\}$ forms a complete orthonormal basis in the RKHS.

Theorem 4.1 (Mercer's Theorem)

Let $k(x, x')$ be a continuous, symmetric, and positive semi-definite kernel function defined on a compact domain $\mathcal{X} \subset \mathbb{R}^d$. Then there exists an orthonormal basis of eigenfunctions $\{\phi_i\}_{i=1}^{\infty}$ in $L^2(\mathcal{X})$ and a sequence of non-negative eigenvalues $\{\lambda_i\}_{i=1}^{\infty}$ such that

$$k(x, x') = \sum_{i=1}^{\infty} \lambda_i \, \phi_i(x) \, \phi_i(x'), \tag{4.21}$$

where the series converges absolutely and uniformly.

♡

This result shows that any Mercer kernel defines a compact, self-adjoint, and positive operator $\mathcal{T}_k$ on $L^2(\mathcal{X})$:

$$(\mathcal{T}_k f)(x) = \int_{\mathcal{X}} k(x, x') \, f(x') \, dx'. \tag{4.22}$$

The eigenfunctions $\{\phi_i\}$ and eigenvalues $\{\lambda_i\}$ then form the spectral decomposition of $\mathcal{T}_k$.

Proposition 4.1

The RKHS $\mathcal{H}_k$ associated with k can be realized as the completion of the span of the kernel sections

$$\mathcal{H}_0 = \operatorname{span}\langle \, k(x, \cdot) \mid x \in \mathcal{X} \, \rangle. \tag{4.23}$$

Each $k(x, \cdot)$ can itself be expanded in the eigenbasis $\{\phi_i\}$ as

$$k(x, \cdot) = \sum_{i=1}^{\infty} \lambda_i \, \phi_i(x) \, \phi_i(\cdot). \tag{4.24}$$

♡

Consequently, every function in the RKHS can be approximated arbitrarily well by finite linear combinations of kernel sections $k(x_i, \cdot)$, since these functions span a dense subspace of $\mathcal{H}_k$. That is,

$$f = \sum_{i=1}^{\infty} \alpha_i \, k(x_i, \cdot), \tag{4.25}$$

where the series converges in the Hilbert norm.

Note

I have not listed the properties of the inner product here, as they should be familiar to anyone with basic mathematical knowledge, although Prof. Zhang presented them in detail during class for the sake of completeness.

Briefly, an RKHS is a space in which the value of a function at any point can be expressed as an inner product between the function itself and the corresponding kernel function.

Note

This property of the kernel function bears a striking resemblance to the idea of tensor contraction. *In tensor analysis, a multivariate tensor can be transformed into a new object by contracting over one or more indices, effectively "fixing" certain variables while leaving others free. Likewise, in the context of RKHS, a kernel $k(x, x')$ can be viewed as a bivariate function that, once one argument (say x_i) is fixed, induces a new function*

$$k(x_i, \cdot) : x' \mapsto k(x_i, x') \tag{4.26}$$

in the RKHS. These kernel sections $k(x_i, \cdot)$ thus play a role analogous to partially contracted tensors: each retains one free variable and collectively they span the functional space.

Example 4.1

Consider $f \in \mathcal{H}$, where $\mathcal{H}$ is a Hilbert sapce, $f(x) = w^\top x$, $x, w \in \mathbb{R}^d$, then $k(z,x) = z^\top x$ is the reproducing kernel of $\mathcal{H} = \text{span}\langle k(x,\cdot), x \in \mathbb{R}^d\rangle$ under inner product $\langle k(w,\cdot), k(\cdot,x)\rangle_{\mathcal{H}} := k(w,x)$.

Proof

$$\langle k(w,\cdot), k(\cdot,x)\rangle_{\mathcal{H}} = k(w,x) = w^\top x = f(x) \tag{4.27}$$

■

Example 4.2

General form. $f(x) = w^\top \varphi(x)$ and $w \in \text{span}\langle \varphi(x) : x \in \mathbb{R}^d\rangle$, $\varphi(x)$ can be expand under polynomial basis. Prove $k(z,x) := \varphi^\top(z)\varphi(x)$ is a reproduce kernel under inner product $\langle k(w,\cdot), k(\cdot,x)\rangle_{\mathcal{H}} := k(w,x)$.

Proof

Assume φ^{-1} exist:

$$\langle k(\varphi^{-1}(w),\cdot), k(\cdot,x)\rangle_{\mathcal{H}} = w^\top \varphi(x) = f(x) \tag{4.28}$$

Otherwise, expand f with kernel basis:

$$\sum_{i=1}^{\infty} \langle \alpha_i k(x_i,\cdot), k(\cdot,x)\rangle_{\mathcal{H}} = \underbrace{\sum_{i=1}^{\infty} \alpha_i \varphi^\top(x_i)}_{:= w^\top} \varphi(x) = w^\top \varphi(x) = f(x) \tag{4.29}$$

■

We can demonstrated that:

Proposition 4.2

The inner product of RKHS must have the term of:

$$\langle k(z,\cdot), k(\cdot,x)\rangle = k(z,x) \tag{4.30}$$

♡

Proof

$$\sum_{i=1}^{\infty} \langle \alpha_i k(x_i,\cdot), k(\cdot,x)\rangle_{\mathcal{H}} \underbrace{=}_{\text{by RKHS's properties}} f(x) = \underbrace{\sum_{i=1}^{\infty} \alpha_i k(x_i,x)}_{\text{by } f(x)\text{'s def}} \tag{4.31}$$

Seperate the uppon formula by α_i:

$$\langle k(z,\cdot), k(\cdot,x)\rangle = k(z,x) \tag{4.32}$$

■

In a Reproducing Kernel Hilbert Space (RKHS) $\mathcal{H}$, the norm of a function $f \in \mathcal{H}$ is induced by the inner product:

$$\|f\|_{\mathcal{H}}^2 = \langle f, f\rangle_{\mathcal{H}}. \tag{4.33}$$

For a linear function of the form

$$f(x) = w^\top x, \tag{4.34}$$

we have

$$\langle f, f\rangle_{\mathcal{H}} = \langle k(w,\cdot),\, k(w,\cdot)\rangle_{\mathcal{H}} = k(w,w) = w^\top w. \tag{4.35}$$

Thus, the RKHS norm of f coincides with the Euclidean norm of its weight vector w.

For a feature-mapped function

$$f(x) = w^\top \varphi(x), \tag{4.36}$$

its inner product in the RKHS is given by

$$\langle f, f\rangle_{\mathcal{H}} = \langle k(\varphi^{-1}(w),\cdot),\, k(\varphi^{-1}(w),\cdot)\rangle_{\mathcal{H}} = k(\varphi^{-1}(w), \varphi^{-1}(w)). \tag{4.37}$$

Using the reproducing property of the kernel, we have

$$k(\varphi^{-1}(w), \varphi^{-1}(w)) = \langle \varphi(\varphi^{-1}(w)),\, \varphi(\varphi^{-1}(w))\rangle_{\mathcal{H}} = \varphi(\varphi^{-1}(w))^\top \varphi(\varphi^{-1}(w)) = w^\top w. \tag{4.38}$$

Now we came to the core of this chapter.

Theorem 4.2 (Representer Theorem)

Let $\mathcal{H}$ be a Reproducing Kernel Hilbert Space (RKHS) over a set $\mathcal{X}$ with reproducing kernel $k(\cdot,\cdot)$. Consider the regularized empirical risk minimization problem of the form:

$$\inf_{f\in\mathcal{H}} J(f) := \sum_{i\in[n]} L(f(x_i, y_i)) + R(\|f\|_{\mathcal{H}}), \tag{4.39}$$

where $L : \mathbb{R}^2 \to \mathbb{R}$ is an arbitrary loss function depending only on the function values at the sample points $\{x_i\}_{i=1}^n$, and R is a regularization function that $[0,\infty) \to \mathbb{R}$.
Then the minimizer $f^\star \in \mathcal{H}$ exists and admits the finite representation

$$f^\star = \sum_{i=1}^n \alpha_i\, k(x_i,\cdot), \tag{4.40}$$

for some coefficients $\alpha_1, \ldots, \alpha_n \in \mathbb{R}$.

♡

Proof

Any $f \in \mathcal{H}$ can be decomposed into two orthogonal components:

$$f = f_\parallel + f_\perp, \tag{4.41}$$

where $f_\parallel$ lies in the subspace $\mathcal{H}_0 := \operatorname{span}\{k(x_i,\cdot)\}_{i=1}^n$ and $f_\perp$ is orthogonal to $\mathcal{H}_0$.

We have

$$f_\perp(x_i) = \langle f_\perp,\, k(\cdot, x_i)\rangle_{\mathcal{H}} = 0, \quad \forall i. \tag{4.42}$$

Hence the loss term $L(f(x_1), \ldots, f(x_n))$ depends only on $f_\parallel$. Moreover, by the Pythagorean theorem,

$$R(\|f\|_{\mathcal{H}}) = R\left(\sqrt{\|\sum_{i\in[n]} \alpha_i k(x_i,\cdot)\|_{\mathcal{H}} + \|f_\perp\|_{\mathcal{H}}}\right) \geq R\left(|\sum_{i\in[n]} \alpha_i k(x_i,\cdot)|\right) \tag{4.43}$$

Replacing f with $f_\parallel$ thus never increases the objective in (4.39), implying that an optimal solution must lie in $\mathcal{H}_0$.

Therefore, the minimizer $f^\star$ can be expressed as a finite kernel expansion:

$$f^\star = \sum_{i=1}^n \alpha_i\, k(x_i,\cdot). \tag{4.44}$$

■

The Representer Theorem carries a profound geometric interpretation. Although the feature mapping

$$\varphi : \mathcal{X} \to \mathcal{H} \tag{4.45}$$

may embed the data into a potentially infinite-dimensional manifold within the RKHS, the optimization of f does not

require direct manipulation in this high-dimensional space.

Specifically, the theorem states that the optimal function takes the form

$$f^\star = \sum_{i=1}^{n} \alpha_i \, k(x_i, \cdot), \tag{4.46}$$

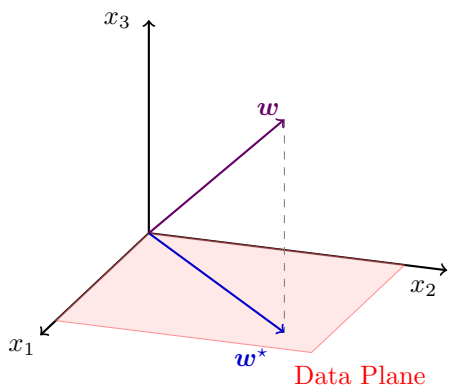
which lies entirely within the finite-dimensional submanifold

$$\mathcal{M}_n = \operatorname{span}\langle k(x_1, \cdot), k(x_2, \cdot), \ldots, k(x_n, \cdot)\rangle \subseteq \mathcal{H}. \tag{4.47}$$

Thus, even though $\mathcal{H}$ may be infinite-dimensional, the optimization problem effectively reduces to solving for the coefficients

$$\alpha_1, \, \alpha_2, \, \ldots, \, \alpha_n, \tag{4.48}$$

within this n-dimensional subspace.



Geometrically, the kernel trick enables us to perform optimization on a low-dimensional submanifold embedded in a high-dimensional feature manifold. Therefore, instead of optimizing a weight vector w in the full RKHS, we only need to search for the optimal combination of kernel coefficients $\{\alpha_i\}$ within the subspace spanned by the training data.

The above discussion reveals that a wide range of learning algorithms admit solutions expressed as weighted sums of kernel functions evaluated on finite training samples. This formulation transforms the prediction task into a problem of computing similarities between data points, rather than directly optimizing over high-dimensional parameters.

In this sense, kernel methods serve as a bridge between *parametric* and *non-parametric* models (such as k-Nearest Neighbors), combining the expressive flexibility of similarity-based approaches with the theoretical rigor of parameterized optimization frameworks.

4.3 Dual Form of Linear Regression

Consider the regularized least squares problem:

$$\underset{w}{\operatorname{argmin}} \; J(w) = \frac{1}{2} \sum_{i \in [n]} \big(y_i - w^\top \varphi(x_i)\big)^2 + \frac{1}{2} \lambda \, w^\top w, \tag{4.49}$$

where $\varphi(x_i)$ is the feature mapping and $\lambda > 0$ is a regularization coefficient. Taking the derivative of $J(w)$ with respect to w gives

$$\frac{\partial J}{\partial w} = \sum_{i \in [n]} \big(w^\top \varphi(x_i) - y_i\big)\varphi(x_i) + \lambda w = 0. \tag{4.50}$$

Rearranging terms, we can express w as a linear combination of feature vectors:

$$w = \sum_{i \in [n]} \alpha_i \, \varphi(x_i), \tag{4.51}$$

where $\alpha_i \in \mathbb{R}$ are the dual coefficients.

Let the feature matrix be

$$\Phi = \begin{pmatrix} \varphi(x_1)^\top \\ \varphi(x_2)^\top \\ \vdots \\ \varphi(x_n)^\top \end{pmatrix} \in \mathbb{R}^{n \times d}. \tag{4.52}$$

Then w can be compactly written as

$$w = \Phi^\top \alpha. \tag{4.53}$$

Substituting into the objective function yields:

$$J(w) = \frac{1}{2}(\Phi w - y)^\top (\Phi w - y) + \frac{1}{2}\lambda \, w^\top w. \tag{4.54}$$

By substituting $w = \Phi^\top \alpha$, we obtain:

$$J(\alpha) = \frac{1}{2}(\Phi\Phi^\top \alpha - y)^\top (\Phi\Phi^\top \alpha - y) + \frac{1}{2}\lambda \, \alpha^\top \Phi\Phi^\top \alpha. \tag{4.55}$$

Defining the kernel matrix (**Gram Matrix**):

$$K = \Phi\Phi^\top, \quad K_{ij} = \langle \varphi(x_i), \varphi(x_j) \rangle = k(x_i, x_j), \tag{4.56}$$

the dual objective becomes:

$$J(\alpha) = \frac{1}{2}(K\alpha - y)^\top (K\alpha - y) + \frac{1}{2}\lambda \, \alpha^\top K\alpha. \tag{4.57}$$

By derivation:

$$J(\alpha) = \frac{1}{2}\alpha^\top K K \alpha + \frac{1}{2} y^\top y - \alpha^\top K y + \frac{\lambda}{2}\alpha^\top K \alpha \tag{4.58}$$

$$\frac{\partial J(\alpha)}{\partial \alpha} = K K \alpha - K y + \lambda K \alpha = 0 \tag{4.59}$$

If $K \succ 0$:

$$(K + \lambda I)\alpha = y \quad \Rightarrow \quad \alpha = (K + \lambda I)^{-1} y \tag{4.60}$$

By simplification, notice that $w = \Phi^\top \alpha$, the prediction function can be expressed as:

$$f(x) = w^\top \varphi(x) = \begin{pmatrix} k(x_1, x) \\ \vdots \\ k(x_n, x) \end{pmatrix} (K + \lambda I)^{-1} y. \tag{4.61}$$

Chapter 5 Learning Theory

Introduction

- *In-Sample Error*
- *Out-of-Sample Error*
- *Generalization Error*
- *PAC*
- *Shelah's Lemma*
- *VC-Dim*

5.1 Learning Theory

Learning theory forms the foundation of machine learning. It explains why a model can generalize to data that did not appear during training. We may draw an analogy here:

- Training a model $\longrightarrow$ Taking a course;
- Training process $\longrightarrow$ Doing exercises or homework;
- Testing phase $\longrightarrow$ Taking an exam.

From our own learning experience, we know that students who perform well during practice do not always achieve high scores in exams. Likewise, a model that fits the training data well may still perform poorly on unseen data. This motivates us to introduce theoretical concepts that can quantitatively describe and explain such phenomena.

Definition 5.1 (In-sample Error)

The in-sample error E_{in} *is defined as the empirical error rate of a hypothesis evaluated on the training dataset.* ♣

Let $h \in \mathcal{H}$ denote a hypothesis or model, for example a linear classifier $h(x) := \text{sign}(w^\top x + b)$. Then the in-sample error can be formally written as

$$E_{\text{in}} = \frac{1}{n} \sum_{i \in [n]} \mathbb{1}\big(h(x_i) \neq y_i\big), \qquad D := \{(x_i, y_i) \mid i \in [n]\}. \tag{5.1}$$

Here D denotes the training dataset, and each $y_i \in \{-1, 1\}$ is the ground-truth label prepared for the classification task.

Oppositely, we also wish to evaluate the model's ability to perform well on unseen or more general tasks. This motivates the definition of a new concept:

Definition 5.2 (Out-of-sample Error)

The out-of-sample error E_{out} *measures how well a model generalizes beyond the training data.* ♣

In the previous example, E_{out} can be expressed as

$$E_{\text{out}} = \mathbb{P}\big(h(x) \neq y\big) = \mathbb{E}_{(x,y)\sim p_{xy}}\left[\mathbb{1}\big(h(x) \neq y\big)\right], \tag{5.2}$$

where $(x, y) \sim p_{xy}$ are i.i.d. samples drawn from the true underlying data distribution.

To quantitatively capture the generalization ability of a model, we define the following notion:

Definition 5.3 (Generalization Error)

The generalization error *is defined as the difference between the out-of-sample and in-sample errors:*

$$E_{\text{out}}(h) - E_{\text{in}}(h). \tag{5.3}$$

♣

Remark

In practice, we often introduce regularization terms to prevent E_{in} from becoming excessively small, which can help improve generalization performance.

Our ultimate goal is to bound the quantity $E_{\text{out}}(h) - E_{\text{in}}(h)$. However, it remains an open question whether such a bound can be feasibly obtained for arbitrary hypotheses.

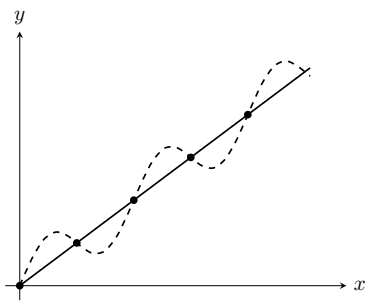


Figure 5.1: An example where $E_{\text{in}} = 0$ but E_{out} remains large.

It is crucial to acknowledge that, in essence, no finite dataset can guarantee a definitive prediction of future outcomes. The optimization of a model solely based on minimizing $E_{\text{in}}(h)$ does not ensure a small generalization gap $E_{\text{out}}(h) - E_{\text{in}}(h)$, which may in fact be arbitrarily large. **However**, according to probabilistic learning theory, we can establish that with high probability $(1 - \delta)$, where $\delta \ll 1$, the generalization error is bounded by a small quantity $\varepsilon(\mathcal{H}, n, \delta)$ depending on the hypothesis space $\mathcal{H}$, the number of samples n, and the confidence parameter δ:

$$\mathbb{P}(|E_{\text{out}}(h) - E_{\text{in}}(h)| \leq \varepsilon(\mathcal{H}, n, \delta)) \geq 1 - \delta. \tag{5.4}$$

This forms the cornerstone of statistical learning theory: while perfect certainty is unattainable, we can quantify uncertainty and guarantee performance with high probability.

The theorem above leads to the framework known as the **Probably Approximately Correct (PAC) Learning Theory**. Before introducing the PAC formulation in detail, we first review several fundamental mathematical tools.

Theorem 5.1 (Hoeffding's Inequality)

Let $X_1, \ldots, X_n$ be independent random variables such that each X_i is almost surely bounded, $X_i \in [a_i, b_i]$. Define the empirical mean $\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$. Then, for any $\varepsilon > 0$, we have

$$\mathbb{P}\left(|\bar{X} - \mathbb{E}\bar{X}| \geq \varepsilon\right) \leq 2\exp\left(-\frac{2n^2\varepsilon^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right). \tag{5.5}$$

♡

Proof

Let $S = \sum_{i=1}^{n} X_i$. We first bound the upper tail $\mathbb{P}(S - \mathbb{E}S \geq t)$; the lower tail is analogous and the two-sided bound follows by union.

By Markov's inequality, for any $\lambda > 0$,

$$\mathbb{P}(S - \mathbb{E}S \geq t) = \mathbb{P}\left(e^{\lambda(S-\mathbb{E}S)} \geq e^{\lambda t}\right) \leq e^{-\lambda t}\,\mathbb{E}\,e^{\lambda(S-\mathbb{E}S)}. \tag{5.6}$$

Because $X_1, \ldots, X_n$ are independent,

$$\mathbb{E}\,e^{\lambda(S-\mathbb{E}S)} = \prod_{i=1}^{n}\mathbb{E}\,e^{\lambda(X_i-\mathbb{E}X_i)}. \tag{5.7}$$

For each bounded random variable $Y \in [a, b]$ with $\mathbb{E}Y = \mu$, Hoeffding's lemma gives

$$\mathbb{E}\, e^{\lambda(Y-\mu)} \leq \exp\left(\frac{\lambda^2(b-a)^2}{8}\right). \tag{5.8}$$

Applying this to each X_i yields

$$\mathbb{E}\, e^{\lambda(S-\mathbb{E}S)} \leq \exp\left(\frac{\lambda^2}{8}\sum_{i=1}^{n}(b_i - a_i)^2\right). \tag{5.9}$$

Hence,

$$\mathbb{P}(S - \mathbb{E}S \geq t) \leq \exp\left(-\lambda t + \frac{\lambda^2}{8}\sum_{i=1}^{n}(b_i - a_i)^2\right). \tag{5.10}$$

Optimizing over $\lambda > 0$, we find

$$\lambda^\star = \frac{4t}{\sum_{i=1}^{n}(b_i - a_i)^2}, \tag{5.11}$$

which gives

$$\mathbb{P}(S - \mathbb{E}S \geq t) \leq \exp\left(-\frac{2t^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right). \tag{5.12}$$

By symmetry,

$$\mathbb{P}(S - \mathbb{E}S \leq -t) \leq \exp\left(-\frac{2t^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right). \tag{5.13}$$

Combining both tails gives

$$\mathbb{P}(|S - \mathbb{E}S| \geq t) \leq 2\exp\left(-\frac{2t^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right). \tag{5.14}$$

Finally, setting $t = n\varepsilon$ and noting $\bar{X} = S/n$, we obtain

$$\mathbb{P}\big(|\bar{X} - \mathbb{E}\bar{X}| \geq \varepsilon\big) \leq 2\exp\left(-\frac{2n^2\varepsilon^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right). \tag{5.15}$$

■

Now, for a **given and fixed** hypothesis h, we obtain the following result:

Theorem 5.2 (First Bound of PAC Learning)

For any $\varepsilon > 0$,

$$\mathbb{P}(E_{\text{out}}(h) - E_{\text{in}}(h) \geq \varepsilon) \leq \exp(-2n\varepsilon^2). \tag{5.16}$$

♡

Here, $E_{\text{in}}(h)$ serves as the empirical mean of the random variables $\mathbb{1}\big(h(x_i) \neq y_i\big)$, as defined previously. In other words, each indicator $\mathbb{1}\big(h(x_i) \neq y_i\big)$ represents a bounded random variable in $[0, 1]$, and Hoeffding's inequality ensures that the deviation between the empirical error $E_{\text{in}}(h)$ and the expected error $E_{\text{out}}(h)$ decreases exponentially with the number of samples n.

Proof

By taking expectation over the definition of E_{in}, we have

$$\begin{aligned}\mathbb{E}[\bar{X}] &= \frac{1}{n}\sum_{i=1}^{n}\mathbb{E}[X_i] \\ &= \frac{1}{n}\sum_{i=1}^{n}\mathbb{E}_{(x_i,y_i)\sim p_{xy}}\left[\mathbb{1}(h(x_i)\neq y_i)\right] \\ &= \frac{1}{n}\sum_{i=1}^{n}E_{\text{out}}(h) = E_{\text{out}}(h).\end{aligned}$$

The third equality follows directly from the definition of $E_{\text{out}}(h)$. Applying Hoeffding's inequality, note that each random variable $\mathbb{1}(h(x_i)\neq y_i)$ is bounded in $\{0,1\}$, so $(b_i-a_i)^2 = 1$ for all i, and the denominator $\sum_{i=1}^{n}(b_i-a_i)^2 = n$ cancels accordingly. This completes the proof. ■

However, there is a fundamental limitation in the condition of this first PAC bound: the hypothesis h must be fixed *before* observing the training data. In practice, the learning algorithm selects h based on the dataset D, which introduces statistical dependence between h and D. (Parameters of model h is determined by the chosen training dataset D) This dependence violates the independence assumption required by Hoeffding's inequality, and therefore the bound no longer holds directly once h is data-dependent.

To overcome this limitation, we turn to analyzing the **union bound** over the hypothesis space $\mathcal{H}$, which removes the dependency on a single fixed hypothesis chosen after observing the training data. We first assume that $\mathcal{H}$ is a finite set, which actually is not always satisfied in practice (e.g., the space of linear models has infinite cardinality, $|\text{GL}_n| = \infty$). That is, let $\mathcal{H} = \{h_1,\ldots,h_M\}$.

In the worst case, we have

$$\begin{aligned}\mathbb{P}(\exists\, h\in\mathcal{H} : E_{\text{out}}(h) - E_{\text{in}}(h) \geq \varepsilon) &= \mathbb{P}\left(\bigcup_{i=1}^{M}\{E_{\text{out}}(h_i) - E_{\text{in}}(h_i)\geq\varepsilon\}\right) \\ &\underbrace{\leq}_{\text{Union Bound}} \sum_{i=1}^{M}\mathbb{P}(E_{\text{out}}(h_i) - E_{\text{in}}(h_i)\geq\varepsilon) \\ &\leq M\cdot\exp(-2n\varepsilon^2).\end{aligned}$$

Hence, we obtain the following result:

Theorem 5.3 (First Practical Bound of PAC Learning)

With probability at least $1-\delta$,

$$E_{\text{out}}(h_i) - E_{\text{in}}(h_i) < \sqrt{\frac{1}{2n}\log\frac{M}{\delta}}. \tag{5.17}$$

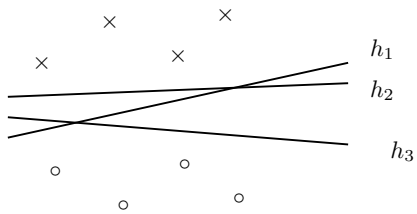
Remark

$$\delta := M\cdot\exp(-2n\varepsilon^2) \quad\Leftrightarrow\quad \varepsilon = \sqrt{\frac{1}{2n}\log\frac{M}{\delta}} \tag{5.18}$$

This result reveals a fundamental trade-off between model complexity and generalization. As the hypothesis space $\mathcal{H}$ becomes richer, the total number of possible hypotheses M increases. A larger M expands the logarithmic term $\log\frac{M}{\delta}$, thereby loosening the bound on $E_{\text{out}} - E_{\text{in}}$. In other words, even though a highly expressive model may achieve $E_{\text{in}} = 0$ by perfectly fitting the training data, its generalization gap can still remain large. To maintain reliable generalization for

such expressive models, one must increase the sample size n accordingly. This highlights the essential balance between data volume and model capacity in learning theory.

Now consider an infinite hypothesis space $\mathcal{H}$, for example the reproducing kernel Hilbert space (RKHS) of linear classifiers $\{h : h(x) = w^\top x + b\}$. We claim that the bound provided in the theorem above becomes an extreme overestimation in such cases. Intuitively, the union bound treats all hypotheses as completely distinct, even if many of them make identical predictions on the dataset. For example, the following three classifiers h_1, h_2, and h_3 may produce exactly the same classification results, yet the union bound still counts them as three separate events, thereby inflating the estimated probability by a factor of three.



Note

A natural idea is to quotient out an equivalence relation on $\mathcal{H}$ so that its effective measure decreases. This leads us to the concept of dichotomy*, where we seek an isomorphism that maps hypotheses producing identical classification results to the same image, thereby identifying them within a common kernel space. By quotienting out the equivalence classes induced by* ker*, which acts as a normal subgroup, we obtain a new structure—still a group (or more appropriately here, a reduced hypothesis space)— a space of smaller measure, serving as a kind of* renormalized *hypothesis space.*

Assume a binary classification setting

$$y \in \{-1, +1\}, \qquad x \in \mathcal{X}, \tag{5.19}$$

with hypothesis space

$$\mathcal{H} = \{h : \mathcal{X} \to \{-1, +1\}\}. \tag{5.20}$$

Given n samples

$$x_1, x_2, \ldots, x_n \in \mathcal{X}, \tag{5.21}$$

for any $h \in \mathcal{H}$ we obtain an n–tuple of labels

$$\big(h(x_1), h(x_2), \ldots, h(x_n)\big) \in \{-1, +1\}^n, \tag{5.22}$$

which is called a *dichotomy* of the set $\{x_1, \ldots, x_n\}$ induced by $\mathcal{H}$.

The growth function $m_{\mathcal{H}}(n)$ is defined as:

Definition 5.4 (Growth Function)

$$m_{\mathcal{H}}(n) = \max_{x_1,\ldots,x_n \in \mathcal{X}} \Big|\big\{(h(x_1), \ldots, h(x_n)) : h \in \mathcal{H}\big\}\Big|, \tag{5.23}$$

♣

i.e., the maximum number of distinct dichotomies that $\mathcal{H}$ can implement on any set of n points. It measures the effective size of the hypothesis space $\mathcal{H}$.

Note

The growth function can be interpreted as a quotient construction. Define the evaluation map

$$\Phi_n : \mathcal{H} \to \{-1,+1\}^n, \qquad \Phi_n(h) = (h(x_1), h(x_2), \ldots, h(x_n)). \tag{5.24}$$

This map induces an equivalence relation on $\mathcal{H}$:

$$h_1 \sim h_2 \quad \Leftrightarrow \quad h_1(x_i) = h_2(x_i),\ \forall i = 1, \ldots, n. \tag{5.25}$$

Each equivalence class corresponds to one unique dichotomy, so two hypotheses are considered identical if they generate the same labeling on the given sample set. Therefore, the growth function can be written as

$$m_{\mathcal{H}}(n) = \left|\mathcal{H}/\ker(\Phi_n)\right|, \tag{5.26}$$

representing the number of equivalence classes of $\mathcal{H}$ under this relation. Intuitively, this quotient construction mods out by the kernel of the evaluation map, focusing only on the empirically distinguishable behaviors of hypotheses on n samples.

Since there are at most 2^n different ± 1 labelings on n points, we always have

$$m_{\mathcal{H}}(n) \le 2^n. \tag{5.27}$$

Now let $S = \{x_1, \ldots, x_n\} \subset \mathcal{X}$ be a finite subset. We say that $\mathcal{H}$ *shatters* S if it can realize all possible ± 1 assignments on S, that is,

$$\mathcal{H}(x_1, \ldots, x_n) = \{-1,+1\}^n. \tag{5.28}$$

Equivalently, for every labeling $(y_1, \ldots, y_n) \in \{-1,+1\}^n$ there exists some $h \in \mathcal{H}$ such that $h(x_i) = y_i$ for all $i = 1, \ldots, n$.

Example 5.1

Consider three sample points x_1, x_2, x_3 and a hypothesis set $\mathcal{H}$ that induces the following dichotomies:

$$\mathcal{H}(x_1, x_2, x_3) = \{(+1,-1,-1),\ (-1,+1,-1),\ (-1,+1,+1)\}. \tag{5.29}$$

Determine which subsets of $\{x_1, x_2, x_3\}$ can be *shattered* by $\mathcal{H}$.

Solution

- *For any single point $\{x_i\}$, both $+1$ and -1 appear in the projections of $\mathcal{H}(x_1, x_2, x_3)$, so every singleton subset is shattered.*
- *For pairs of points:*

  $\{x_1, x_2\} \Rightarrow \{(+1,-1),\ (-1,+1)\}$ *(missing $(+1,+1)$ and $(-1,-1)$) $\Rightarrow$ not shattered;*

  $\{x_1, x_3\} \Rightarrow \{(+1,-1),\ (-1,+1),\ (-1,-1)\}$ *(missing $(+1,+1)$) $\Rightarrow$ not shattered;*

  $\{x_2, x_3\} \Rightarrow \{(-1,-1),\ (+1,-1),\ (+1,+1)\}$ *(all $2^2 = 4$ combinations not present) $\Rightarrow$ not shattered.*
- *For the full set $\{x_1, x_2, x_3\}$, since only $3 < 2^3 = 8$ dichotomies are realized, it is clearly not shattered.*

Hence, $\mathcal{H}$ can only shatter the singleton subsets $\{x_1\}$, $\{x_2\}$, *and* $\{x_3\}$, *but no pair or triple of points.*

Example 5.2

Let $\mathcal{X} = \mathbb{R}^2$, and consider the hypothesis set of all linear classifiers

$$\mathcal{H} = \{\, h_{w,b}(x) = \operatorname{sign}(w^\top x + b) \mid w \in \mathbb{R}^2,\ b \in \mathbb{R} \,\}. \tag{5.30}$$

Determine the value of $m_{\mathcal{H}}(3)$, the maximum number of dichotomies that $\mathcal{H}$ can produce on any 3 points in $\mathbb{R}^2$.

Solution

A separating hyperplane in $\mathbb{R}^2$ is a line of the form $w^\top x + b = 0$. The sign of $w^\top x + b$ assigns each point to one of two half-spaces.
For any given labeling $(y_1, y_2, y_3) \in \{-1, +1\}^3$, we can always find some line that places the $+1$ points on one side and the -1 points on the other, as long as the three points are not collinear. This is because two points determine a line, and the third point's label can always be satisfied by shifting or rotating that line slightly.
Hence every possible assignment of ± 1 labels to the three points is realizable:

$$\mathcal{H}(x_1, x_2, x_3) = \{-1, +1\}^3. \tag{5.31}$$

Therefore, the growth function value is

$$m_{\mathcal{H}}(3) = 2^3 = 8. \tag{5.32}$$

Remark

More examples have been presented in class, but I'm not goint to note these in our lecture.

$m_{\mathcal{H}}(n)$ quantified the ability of a model.

5.2 VC-Dim

Definition 5.5 (Vapnik–Chervonenkis Dimension)

The VC dimension $\mathrm{dvc}(\mathcal{H})$ *is the largest integer n such that*

$$m_{\mathcal{H}}(n) = 2^n. \tag{5.33}$$

♣

Remark

Vapnik is the inventor of the Support Vector Machine (SVM) and one of the founders of statistical learning theory.

The VC dimension measures the *capacity* or *expressive power* of a hypothesis class. Formally, it is the largest integer n such that $\mathcal{H}$ can realize all 2^n possible dichotomies on some set of n points. Equivalently, it quantifies how complex the decision boundaries in $\mathcal{H}$ can be: a larger VC dimension means $\mathcal{H}$ can represent more distinct labelings of data, i.e. greater capacity.

It turns out that if $\mathrm{dvc}(\mathcal{H})$ is finite, then $m_{\mathcal{H}}(n)$ can be bounded in terms of $\mathrm{dvc}(\mathcal{H})$.

Theorem 5.4 ((Sauer–)Shelah's Lemma)

Let $\mathcal{H}$ be a class of binary-valued functions on a domain $\mathcal{X}$. Define its growth function as

$$m_{\mathcal{H}}(n) = \max_{\substack{S \subseteq \mathcal{X} \\ |S| = n}} \left| \{ (h(x))_{x \in S} : h \in \mathcal{H} \} \right|. \tag{5.34}$$

If $\mathrm{dvc}(\mathcal{H}) = d < \infty$, then for all $n \geq 0$,

$$m_{\mathcal{H}}(n) \leq \sum_{i=0}^{d} \binom{n}{i} \sim O(n^d) \tag{5.35}$$

♡

Proof

We proceed by induction on n. The base case is trivial and we proved that in the next proof for completeness.

Step 1: Divide the hypothesis class. *Fix $x_1, \ldots, x_n \in \mathcal{X}$, and define*

$$\mathcal{H}_1(x_1, \ldots, x_{n-1}) = \{(h(x_1), \ldots, h(x_{n-1})) \mid h \in \mathcal{H},\ h(x_n) = +1\}, \tag{5.36}$$

$$\mathcal{H}_2(x_1, \ldots, x_{n-1}) = \{(h(x_1), \ldots, h(x_{n-1})) \mid h \in \mathcal{H},\ h(x_n) = -1\}. \tag{5.37}$$

Then

$$\mathcal{H}(x_1, \ldots, x_n) = \{(\mathbf{v}, +1) \mid \mathbf{v} \in \mathcal{H}_1(x_1, \ldots, x_{n-1})\} \cup \{(\mathbf{v}, -1) \mid \mathbf{v} \in \mathcal{H}_2(x_1, \ldots, x_{n-1})\}. \tag{5.38}$$

Step 2: Counting distinct dichotomies. *If $\mathbf{v}$ belongs to both $\mathcal{H}_1$ and $\mathcal{H}_2$, then $\mathcal{H}$ realizes both labelings $(\mathbf{v}, +1)$ and $(\mathbf{v}, -1)$. Hence,*

$$|\mathcal{H}(x_1, \ldots, x_n)| \leq |\mathcal{H}_1(x_1, \ldots, x_{n-1})| + |\mathcal{H}_2(x_1, \ldots, x_{n-1})|. \tag{5.39}$$

Step 3: Bounding the VC dimension. *If $\mathrm{dvc}(\mathcal{H}) = d$, then $\mathrm{dvc}(\mathcal{H}_1) \leq d$ and $\mathrm{dvc}(\mathcal{H}_2) \leq d - 1$. Otherwise, if $\mathcal{H}_2$ could shatter a subset of size d, then $\mathcal{H}$ could shatter that subset together with x_n, contradicting $\mathrm{dvc}(\mathcal{H}) = d$.*

Step 4: Apply the induction hypothesis. *By the inductive assumption,*

$$|\mathcal{H}_1(x_1, \ldots, x_{n-1})| \leq \sum_{i=0}^{d} \binom{n-1}{i}, \qquad |\mathcal{H}_2(x_1, \ldots, x_{n-1})| \leq \sum_{i=0}^{d-1} \binom{n-1}{i}. \tag{5.40}$$

Therefore,

$$|\mathcal{H}(x_1, \ldots, x_n)| \leq \sum_{i=0}^{d} \binom{n-1}{i} + \sum_{i=0}^{d-1} \binom{n-1}{i} \tag{5.41}$$

$$= \sum_{i=0}^{d} \left(\binom{n-1}{i} + \binom{n-1}{i-1} \right) = \sum_{i=0}^{d} \binom{n}{i}, \tag{5.42}$$

where the last equality uses Pascal's identity. Hence the claim holds for all n. ■

⊛ *The following proof is included out of personal interest and is not part of the course material. Readers who wish to focus solely on the course content may safely skip to the next part.*

Proof

Here we provide another proof.

Fix n and choose $S \subseteq \mathcal{X}$ with $|S| = n$ such that $m_{\mathcal{H}}(n) = |\mathcal{H}|_S|$, where

$$\mathcal{H}|_S := \{\, (h(x))_{x \in S} : h \in \mathcal{H} \} \tag{5.43}$$

is the set of labelings realized on S. We prove by induction on n that

$$|\mathcal{H}|_S| \leq \sum_{i=0}^{d} \binom{n}{i}. \tag{5.44}$$

Base cases. *If $n = 0$, then $|\mathcal{H}|_S| = 1$ and*

$$\sum_{i=0}^{d} \binom{0}{i} = 1, \tag{5.45}$$

so the claim holds. If $d = 0$, then no single point can be labeled in two different ways, hence $|\mathcal{H}|_S| = 1$ for all n, while

$$\sum_{i=0}^{0} \binom{n}{i} = 1, \tag{5.46}$$

so the bound again holds.

Inductive step. *Assume the statement true for all pairs (n', d') with $n' < n$ and the same d. Pick some $x \in S$ and set*

$$S' := S \setminus \{x\}, \qquad |S'| = n - 1. \tag{5.47}$$

For each $h \in \mathcal{H}$, let $h|_{S'}$ be its restriction to S'. Define

$$\mathcal{A} := \{\, h|_{S'} : h \in \mathcal{H} \}, \tag{5.48}$$

$$\mathcal{B} := \left\{\, h|_{S'} : h \in \mathcal{H},\ \exists\, h' \in \mathcal{H} \text{ with } h'(x) \neq h(x),\ h'|_{S'} = h|_{S'} \right\}. \tag{5.49}$$

Thus, $\mathcal{A}$ is the set of all restrictions on S', and $\mathcal{B}$ collects those restrictions for which $\mathcal{H}$ contains at least two hypotheses that agree on S' but disagree on x.

Every labeling of S comes from choosing a restriction in $\mathcal{A}$, and then choosing the value at x. Labelings for which the value at x is forced *(no alternative hypothesis with the same restriction) contribute at most $|\mathcal{A}|$ patterns; labelings for which the value at x can be flipped (i.e. restrictions in $\mathcal{B}$) contribute at most $|\mathcal{B}|$ additional patterns. Hence*

$$|\mathcal{H}|_S| \leq |\mathcal{A}| + |\mathcal{B}|. \tag{5.50}$$

Note that $|\mathcal{A}| \leq m_{\mathcal{H}}(n-1)$, because $\mathcal{A}$ is a set of labelings on S' of size $n-1$. To bound $|\mathcal{B}|$, define a new hypothesis class

$$\mathcal{H}_x := \left\{\, h|_{S'} : h \in \mathcal{H},\ \exists\, h' \in \mathcal{H} \text{ with } h'(x) \neq h(x),\ h'|_{S'} = h|_{S'} \right\}. \tag{5.51}$$

Then $\mathcal{B}$ is precisely $\mathcal{H}_x$ viewed as a set of labelings on S', so $|\mathcal{B}| \leq m_{\mathcal{H}_x}(n-1)$.

We claim that

$$\operatorname{dvc}(\mathcal{H}_x) \leq d - 1. \tag{5.52}$$

Otherwise, suppose $\mathcal{H}_x$ shatters some $T \subseteq S'$ with $|T| = d$. By the definition of $\mathcal{H}_x$, for every labeling y_T on T and either choice of label $y_x \in \{-1, +1\}$ at x, there exists $h \in \mathcal{H}$ such that

$$h|_T = y_T, \qquad h(x) = y_x. \tag{5.53}$$

Hence $\mathcal{H}$ can realize all labelings on $T \cup \{x\}$, so $\mathcal{H}$ shatters a set of size $d+1$, contradicting $\operatorname{dvc}(\mathcal{H}) = d$. *Thus the claim holds.*

By the induction hypothesis applied to $\mathcal{H}$ on $n-1$ points and to $\mathcal{H}_x$ whose VC dimension is at most $d-1$, we obtain

$$|\mathcal{A}| \leq \sum_{i=0}^{d} \binom{n-1}{i}, \qquad |\mathcal{B}| \leq \sum_{i=0}^{d-1} \binom{n-1}{i}. \tag{5.54}$$

Therefore,

$$|\mathcal{H}|_S| \leq \sum_{i=0}^{d} \binom{n-1}{i} + \sum_{i=0}^{d-1} \binom{n-1}{i} \tag{5.55}$$

$$= \sum_{i=0}^{d} \left(\binom{n-1}{i} + \binom{n-1}{i-1} \right) \tag{5.56}$$

$$= \sum_{i=0}^{d} \binom{n}{i}. \tag{5.57}$$

■

Note

The inductive structure of the proof for the Sauer–Shelah Lemma is deeply reminiscent of strategies used in mathematical logic, *particularly in proving* completeness theorems *that connect syntactic derivability and semantic truth. In both settings, we start by imposing an ordering (or enumeration) on a collection of objects—whether they are*

formulas, sentences, or finite subsets of $\mathcal{X}$—so that induction can be applied coherently.

Just as in logic we construct models step by step according to a fixed sequence of formulas (often by extending consistent sets in a well-ordered fashion), here we inductively analyze the growth of $\mathcal{H}$ by removing or reintroducing one element of the sample set at a time. The key similarity is structural: both arguments rely on well-founded recursion over a totally ordered domain to control how local extensions (adding one more formula or one more point) affect global consistency or combinatorial growth.

Remark

Plugging this bound into the standard uniform convergence inequality for a finite hypothesis class, we obtain a generalization bound of the form

$$\varepsilon = \sqrt{\frac{1}{2n}\log\frac{M}{\delta}} \sim O\left(\frac{\mathrm{dvc}\log n}{n}\right)^{1/2} \tag{5.58}$$

Thus, when $\mathrm{dvc}(\mathcal{H})$ *is finite, the estimation error ε decreases as n grows, with the complexity of the class entering only through the polynomial factor controlled by d.*

It can be rigorously shown that:

Theorem 5.5 (VC Generalization Bound)

With probability at least $1-\delta$, the following inequality holds:

$$E_{\mathrm{out}}(h_i) - E_{\mathrm{in}}(h_i) < \sqrt{\frac{8}{n}\log\frac{4m_{\mathcal{H}}(2n)}{\delta}}. \tag{5.59}$$

Moreover,

$$E_{\mathrm{out}}(h_i) - E_{\mathrm{in}}(h_i) = O\left(\sqrt{\frac{\mathrm{d_{VC}}\log n}{n}}\right) \xrightarrow[n\to\infty]{} 0. \tag{5.60}$$

Although the VC generalization bound is relatively loose in practice, its importance lies in converting an inherently infinite hypothesis space into a finitely bounded one. This theorem establishes a concrete analytical foundation for the entire field of machine learning, demonstrating that with sufficient data, generalization is not merely empirical, but theoretically guaranteed.

Chapter 6 Gaussian Process

Introduction

- *Gaussian Process Definition*
- *Kernel and Covariance Structure*
- *Joint–Conditional Gaussian Identities*
- *Gaussian Process Regression*
- *RBF Kernel and Limit Behaviors*
- *Bayesian Linear Regression as GP*

6.1 Stochastic Process

Definition 6.1 (Stochastic Process)

A stochastic process *is a collection of random variables (possibly infinitely many) indexed by a set* $\mathcal{I}$.

The choice of index set $\mathcal{I}$ is flexible; common examples include the natural numbers $\mathbb{N}$ or the real line $\mathbb{R}$. We may freely draw sample pairs (x_i, y_i) with $y_i \sim p(x_i)$, and in the limit of infinitely many samples the joint distribution of these pairs becomes determined. Conversely, we may also characterize a stochastic process directly by specifying the joint distributions of the random variables $\{y_i\}_{i\in\mathcal{I}}$ for all finite index subsets.

Definition 6.2 (Markov Random Process)

A Markov random process $\{y_i\}_{i\in\mathbb{N}}$ *is a stochastic process satisfying*

$$p(y_{t+1} \mid y_1, \dots, y_t) = p(y_{t+1} \mid y_t), \qquad \forall t \in \mathbb{N}. \tag{6.1}$$

Definition 6.3 (Bernoulli Random Process)

A Bernoulli random process $\{y_i\}_{i\in\mathbb{N}}$ *is a stochastic process satisfying*

$$p(y_1, \dots, y_n) = \prod_{i=1}^{n} p(y_i), \tag{6.2}$$

meaning that the random variables are mutually independent, and each $y_i \sim \mathcal{B}(1, p_i)$ *follows a Bernoulli distribution.*

Definition 6.4 (Multivariate Gaussian)

For $x \in \mathbb{R}^d$*, we say that* x *follows a* multivariate Gaussian distribution *if*

$$p(x) = \frac{1}{(2\pi)^{d/2}} \frac{1}{\det(\Sigma)^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\right), \tag{6.3}$$

where $\mu \in \mathbb{R}^d$ *is the mean vector and* $\Sigma \in \mathbb{R}^{d\times d}$ *is a positive-definite symmetric covariance matrix. We denote this distribution by* $x \sim \mathcal{N}(\mu, \Sigma)$.

It is easy to verify that the multivariate Gaussian reduces to the standard one-dimensional Gaussian distribution when $d = 1$. Moreover, when the covariance matrix Σ is diagonal, the multivariate Gaussian factorizes into the product of d independent one-dimensional Gaussian distributions.

Remark

More detailed mathematical treatment can be found in Pattern Recognition and Machine Learning *(Bishop, Sec. 2.3).*

Theorem 6.1

Any affine (or linear) transformation of a Gaussian random variable remains Gaussian.

The above property can in fact be taken as an equivalent way to *define* the Gaussian distribution, due to the equivalence between the standard density-based definition and this transformation property.

Note

This property also extends to the case where the covariance matrix Σ is degenerate.

Definition 6.5 (Gaussian Process)

A Gaussian Process *(GP) is a collection of random variables $\{f(x)\}_{x\in\mathcal{X}}$ indexed by an input space $\mathcal{X}$, such that every finite subcollection*

$$\big(f(x_1), f(x_2), \ldots, f(x_n)\big) \tag{6.4}$$

is jointly multivariate Gaussian for all choices of $x_1, \ldots, x_n \in \mathcal{X}$.

A natural question arises: to specify a Gaussian process, do we need to provide a different set of parameters for every collection of n random variables? If that were the case, the notion of a Gaussian process would lose its meaning, since incorporating more sample points would require specifying exponentially many parameters, which is an infeasible task when n is large.

Fortunately, Gaussian processes enjoy the desirable closure property under linear transformations. This guarantees that adding new random variables does not affect the parameters of existing ones, because the projection

$$(y_1, \ldots, y_{n+1}) \mapsto (y_1, \ldots, y_n)$$

is itself a linear (though degenerate) transformation.

Note

$$\begin{pmatrix} y_1 \\ \vdots \\ y_n \\ 0 \end{pmatrix} = \operatorname{diag}(1, \ldots, 1, 0) \begin{pmatrix} y_1 \\ \vdots \\ y_n \\ y_{n+1} \end{pmatrix}. \tag{6.5}$$

More precisely, a Gaussian process is completely specified once we assign a mean value μ_i to every random variable y_i, and a covariance value Σ_{ij} to every pair (y_i, y_j). These quantities must satisfy the usual symmetry and positive semi-definiteness conditions; under these consistency requirements, they uniquely determine a Gaussian process.

We write a Gaussian process as

$$f \sim \mathcal{GP}(m(\cdot),\, k(\cdot,\cdot))\,, \tag{6.6}$$

where the mean function $m : \mathcal{X} \to \mathbb{R}$ is defined by

$$m(x) = \mathbb{E}[f(x)] := \int_{\mathcal{X}} f(x)\, \mathrm{d}\mu(x), \tag{6.7}$$

and the covariance kernel $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is given by

$$k(x, x') = \mathrm{Cov}(f(x), f(x')) = \mathbb{E}[(f(x) - m(x))(f(x') - m(x'))] , \tag{6.8}$$

which must be positive semi-definite. In short, once the mean function and covariance kernel are fixed, the entire Gaussian process is already written; nothing further needs to be said.

Remark

In practice, we may assume that the kernel k is strictly positive definite. If the corresponding covariance matrix Σ is only positive semi-definite, we can add a small regularization term on the diagonal, for example

$$\Sigma_\varepsilon = \Sigma + \varepsilon I, \qquad \varepsilon > 0, \tag{6.9}$$

which ensures $\Sigma_\varepsilon \succ 0$. This is equivalent to adding a small independent Gaussian noise term to the process and is commonly used to avoid numerical degeneracy.

Example 6.1

In geostatistical terrain modeling, assume we observe discrete elevation samples

$$\{(x_i,\ y_i)\}_{i=1}^n, \qquad x_i \in \mathcal{X} \subset \mathbb{R}^2, \tag{6.10}$$

where each y_i denotes the measured height at location x_i.
To obtain a smooth terrain surface from these samples, we place a Gaussian process prior on the unknown height function $f : \mathcal{X} \to \mathbb{R}$:

$$f \sim \mathcal{GP}(0,\ k_{\mathrm{RBF}}(\cdot, \cdot)) . \tag{6.11}$$

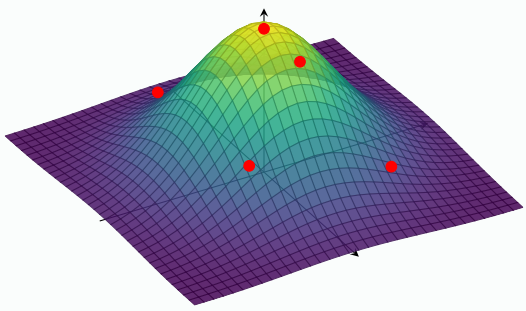
The radial basis function (RBF) kernel is given by

$$k_{\mathrm{RBF}}(x, x') = \sigma^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right), \tag{6.12}$$

which controls smoothness and spatial correlation of the terrain.
Conditioning on the observed elevations $\{y_i\}$, the posterior mean

$$m_f(x) = \mathbb{E}\left[f(x) \mid \{y_i\}_{i=1}^n\right] \tag{6.13}$$

yields a smooth interpolating surface, where the height at each location is a kernel-weighted combination of nearby observations.



Thus, the RBF kernel spreads the influence of each sample point smoothly across the region, producing a continuous terrain map consistent with the measured elevations.

In many applications, the mean function is set to be identically zero, $m(x) \equiv 0$, which corresponds to placing an uninformative prior on f. Under the assumption that the mean function is identically zero, $m(x) \equiv 0$, and given a kernel $k(x_i, x_j)$, the vector of training labels satisfies

$$y \sim \mathcal{N}(0, K), \tag{6.14}$$

where K is the Gram matrix defined previously, and the mean 0 denotes the zero vector in $\mathbb{R}^n$ with the same dimension as

y.

Let $\{x_i\}_{i=1}^n$ be the training inputs and $\{y_i\}$ their corresponding observations. The above Gaussian model describes the joint distribution induced by the Gaussian process prior on the training set. For a new test point $(x_\star, y_\star)$, we can augment the training vector to obtain the joint prior

$$\begin{pmatrix} y_\star \\ y \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} k(x_\star, x_\star) & k(x_\star, \cdot)^\top \\ k(x_\star, \cdot) & K \end{pmatrix}\right), \tag{6.15}$$

where

$$k(x_\star, \cdot) = \begin{pmatrix} k(x_\star, x_1) \\ \vdots \\ k(x_\star, x_n) \end{pmatrix}. \tag{6.16}$$

To make predictions, we require the conditional distribution $p(y_\star \mid y)$. Since the joint distribution is Gaussian and the marginal $p(y)$ is known, the conditional distribution can be obtained directly using the standard formula for conditioning a multivariate normal distribution.

Jumping directly to the conclusion, let

$$x = \begin{pmatrix} x_a \\ x_b \end{pmatrix}, \qquad x_a \in \mathbb{R}^a,\ x_b \in \mathbb{R}^b, \tag{6.17}$$

and assume

$$x \sim \mathcal{N}\left(\begin{pmatrix} \mu_a \\ \mu_b \end{pmatrix}, \begin{pmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{pmatrix}\right). \tag{6.18}$$

Then the following standard results hold:

- **Marginalization:**

$$x_a \sim \mathcal{N}(\mu_a, \Sigma_{aa}), \qquad x_b \sim \mathcal{N}(\mu_b, \Sigma_{bb}). \tag{6.19}$$

- **Conditioning:**

$$p(x_a \mid x_b) = \mathcal{N}\big(\mu_{a|b},\ \Sigma_{a|b}\big), \tag{6.20}$$

  where the conditional mean and covariance are

$$\begin{aligned} \mu_{a|b} &= \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(x_b - \mu_b), \\ \Sigma_{a|b} &= \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}. \end{aligned} \tag{6.21}$$

Remark

Intuitively, the matrix $\Sigma_{ab}\Sigma_{bb}^{-1}$ acts as a linear projection that describes how variations in x_b influence our estimate of x_a. It adjusts the mean of x_a in proportion to how far x_b lies from its own mean.

Returning to the main track, the predictive distribution for a test label $y_\star$ conditioned on the observed vector y is

$$p(y_\star \mid y) = \mathcal{N}(y_\star \mid \mu_\star,\ \Sigma_\star), \tag{6.22}$$

where the posterior mean is given by

$$\mu_\star = k(x_\star, \cdot)^\top K^{-1} y, \tag{6.23}$$

and the posterior variance is

$$\Sigma_\star = k(x_\star, x_\star) - k(x_\star, \cdot)^\top K^{-1} k(x_\star, \cdot). \tag{6.24}$$

Assume that the underlying function values y are noise-free, and that in practice we only observe

$$\hat{y} = y + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2 I). \tag{6.25}$$

Under this observation model, one can show that the joint distribution of the latent value $y_\star$ at a test point $x_\star$ and the

observed vector $\hat{y}$ at the training inputs is given by

Theorem 6.2

$$\begin{pmatrix} y_\star \\ \hat{y} \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} k(x_\star, x_\star) & k(x_\star, \cdot)^\top \\ k(x_\star, \cdot) & K + \sigma^2 I \end{pmatrix}\right), \tag{6.26}$$

where K denotes the Gram matrix with entries $K_{ij} = k(x_i, x_j)$.

Proof

Notice that ε is independent of $(y_\star, y)$ and $\mathbb{E}[\varepsilon] = \mathbb{E}[y_\star] = \mathbb{E}[y] = 0$:

$$\begin{aligned}
\mathrm{Cov}(y_\star, y_\star) &= \mathrm{Var}(y_\star) = k(x_\star, x_\star), \\
\mathrm{Cov}(y_\star, \hat{y}) &= \mathrm{Cov}\big(y_\star,\ y + \varepsilon\big) \\
&= \mathrm{Cov}(y_\star, y) + \mathrm{Cov}(y_\star, \varepsilon) \\
&= k(x_\star, \cdot) + 0 = k(x_\star, \cdot), \\
\mathrm{Cov}(\hat{y}, \hat{y}) &= \mathrm{Cov}\big(y + \varepsilon,\ y + \varepsilon\big) \\
&= \mathrm{Cov}(y, y) + \mathrm{Cov}(y, \varepsilon) + \mathrm{Cov}(\varepsilon, y) + \mathrm{Cov}(\varepsilon, \varepsilon) \\
&= K + 0 + 0 + \sigma^2 I_n \\
&= K + \sigma^2 I_n.
\end{aligned}$$

■

Corollary 6.1

$$y_\star \mid \hat{y} \sim \mathcal{N}(\mu_\star, \Sigma_\star) \tag{6.27}$$

where $\mu_\star = k(x_\star)^\top (K + \sigma^2 I)^{-1} \hat{y}$.

Comparing the above predictive formulas with (4.61), we arrive at a striking conclusion:

Proposition 6.1

Linear regression, when viewed from the Bayesian perspective, induces a Gaussian process.

Proof

Take any finite collection of inputs $\{x_i\}_{i=1}^n \subset \mathcal{X}$ and define

$$f_i := f(x_i) = w^\top \phi(x_i), \qquad i = 1, \ldots, n. \tag{6.28}$$

Stack these values into

$$f := \begin{pmatrix} f_1 \\ \vdots \\ f_n \end{pmatrix} \in \mathbb{R}^n. \tag{6.29}$$

Let $\Phi \in \mathbb{R}^{n \times p}$ be the feature matrix

$$\Phi := \begin{pmatrix} \phi(x_1)^\top \\ \vdots \\ \phi(x_n)^\top \end{pmatrix}, \tag{6.30}$$

so that

$$f = \Phi w. \tag{6.31}$$

With the prior $w \sim \mathcal{N}(0, \sigma_w^2 I_p)$, f *is a linear transformation of a Gaussian vector, hence*

$$f \sim \mathcal{N}\big(0,\ \sigma_w^2 \Phi\Phi^\top\big). \tag{6.32}$$

Therefore, for all i, j,

$$\mathrm{Cov}\big(f(x_i), f(x_j)\big) = \big(\sigma_w^2 \Phi\Phi^\top\big)_{ij} = \sigma_w^2\, \phi(x_i)^\top \phi(x_j). \tag{6.33}$$

Define the kernel

$$k(x, x') := \sigma_w^2\, \phi(x)^\top \phi(x'). \tag{6.34}$$

Then for any finite set $\{x_i\}_{i=1}^n$, *the vector* $(f(x_1), \ldots, f(x_n))$ *is multivariate Gaussian with mean zero and covariance matrix* $[k(x_i, x_j)]_{i,j}$. *By the definition of a Gaussian process, this implies*

$$f \sim \mathcal{GP}\big(0,\ k(\cdot,\cdot)\big). \tag{6.35}$$

Now add Gaussian observation noise:

$$y_i = f(x_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \tag{6.36}$$

with $\{\varepsilon_i\}$ *i.i.d. and independent of* w. *Let* $y = f + \varepsilon$ *where* $\varepsilon \sim \mathcal{N}(0, \sigma^2 I_n)$. *Then* y *is also Gaussian with*

$$\mathrm{Cov}(y_i, y_j) = \mathrm{Cov}\big(f(x_i), f(x_j)\big) + \mathrm{Cov}(\varepsilon_i, \varepsilon_j) \tag{6.37}$$

$$= \sigma_w^2\, \phi(x_i)^\top \phi(x_j) + \sigma^2 \delta_{ij}. \tag{6.38}$$

Hence the effective covariance function for the observed process is

$$k_{\mathrm{obs}}(x, x') = \sigma_w^2\, \phi(x)^\top \phi(x') + \sigma^2 \delta_{x,x'}, \tag{6.39}$$

which is exactly the GP regression model with kernel as stated. ■

Remark

The GP framework is powerful because it allows us to work directly with the covariance kernel $k(x, x')$ *rather than introducing a weight vector* w. *This enables the kernel trick and makes infinite-dimensional feature spaces accessible without ever defining* $\phi(x)$ *explicitly.*

Remark

In practice, Gaussian process regression requires inverting the $n \times n$ *covariance matrix* $K + \sigma^2 I$, *which has a cubic time complexity* $O(n^3)$ *and becomes prohibitive when the dataset is large. To address this issue, one typically resorts to approximation techniques such as selecting a small set of inducing points or using low-rank and sparse approximations of the kernel matrix. These methods preserve most of the predictive power of the GP while significantly reducing computational cost.*

6.2 Gaussian Process Regression (GPR)

Review the previous conclusion. Given training inputs $X = \{x_i\}_{i=1}^n$ with observations

$$y = f(X) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2 I), \tag{6.40}$$

and a test input $x_\star$, the joint distribution between y and the latent value $f_\star = f(x_\star)$ is

$$\begin{pmatrix} f_\star \\ y \end{pmatrix} \sim \mathcal{N}\left(0,\ \begin{pmatrix} k(x_\star, x_\star) & k(x_\star, X)^\top \\ k(x_\star, X) & K + \sigma^2 I \end{pmatrix}\right), \tag{6.41}$$

where

$$K_{ij} = k(x_i, x_j), \qquad k(x_\star, X) = \begin{pmatrix} k(x_\star, x_1) \\ \vdots \\ k(x_\star, x_n) \end{pmatrix}. \tag{6.42}$$

Conditioning on the observed vector y, the predictive distribution of $f_\star$ is Gaussian:

$$f_\star \mid y \sim \mathcal{N}(\mu_\star,\ \Sigma_\star), \tag{6.43}$$

with posterior mean

$$\mu_\star = k(x_\star, X)^\top (K + \sigma^2 I)^{-1} y, \tag{6.44}$$

and predictive variance

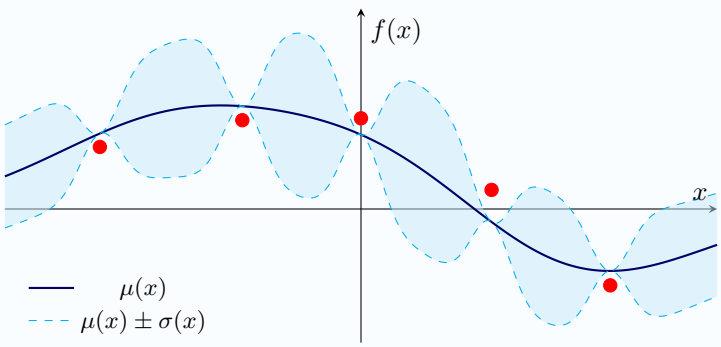
$$\Sigma_\star = k(x_\star, x_\star) - k(x_\star, X)^\top (K + \sigma^2 I)^{-1} k(x_\star, X). \tag{6.45}$$

Remark

Consider the RBF kernel

$$k_\ell(x, x') = \sigma_f^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right), \tag{6.46}$$

where $\ell > 0$ is the length-scale parameter.



For any fixed x, x', we have

$$\lim_{\ell \to 0} k_\ell(x, x') = \begin{cases} \sigma_f^2, & x = x', \\ 0, & x \neq x'. \end{cases} \tag{6.47}$$

In particular, for distinct training inputs $\{x_i\}_{i=1}^n$, the Gram matrix K_ℓ converges to a diagonal matrix:

$$\lim_{\ell \to 0} K_\ell = \sigma_f^2 I. \tag{6.48}$$

In the noise-free case (i.e. $\sigma^2 = 0$ in the observation model), the predictive mean at a training point x_j is

$$\mu_\ell(x_j) = k_\ell(x_j, X)^\top K_\ell^{-1} y. \tag{6.49}$$

In the limit $\ell \to 0$, we have

$$k_\ell(x_j, X) \longrightarrow \sigma_f^2 e_j, \qquad K_\ell^{-1} \longrightarrow \frac{1}{\sigma_f^2} I, \tag{6.50}$$

where e_j is the j-th standard basis vector in $\mathbb{R}^n$. Hence

$$\lim_{\ell \to 0} \mu_\ell(x_j) = \sigma_f^2 e_j^\top \left(\tfrac{1}{\sigma_f^2} I_n\right) y = e_j^\top y = y_j. \tag{6.51}$$

That is, in the limit $\ell \to 0$, the GP interpolates the data exactly at every training input: each training point is matched perfectly by the predictive mean.

From the discussion above, we know that the *posterior mean* of a Gaussian Process provides the predictive value for a test input, while the associated *uncertainty* can be quantified through the *predictive variance*. The relative standard

deviation is simply the square root of this predictive variance.

Gaussian Process Regression (GPR) serves as the mathematical foundation of **Bayesian Optimization (BO)**, a framework for performing optimization when the objective function is expensive, noisy, or lacks analytic structure.

Definition 6.6 (Black-box Optimization)

An optimization problem is called black-box optimization *if and only if the analytical form of the objective function is unknown and no gradient information is available.* ♣

Since the gradient of the objective function is unavailable, gradient-based methods such as gradient descent cannot be applied in black-box settings. The only feasible operation is *point-wise evaluation*: we may query the black box at a finite number of input locations and observe the corresponding outputs.

Bayesian Optimization (BO) aims to construct a probabilistic surrogate model of the black-box function and use it to locate the maximizer of y with as few evaluations as possible. This is particularly important when each evaluation is expensive, for instance in hyperparameter tuning.

The typical BO procedure proceeds as follows:

1. Randomly or uniformly select a small set of initial points $x_1, \ldots, x_n$ and obtain their evaluations $y_1, \ldots, y_n$.
2. Fit a Gaussian Process Regression (GPR) model using the collected data.
3. Use an acquisition function $a(x)$ to select one or a batch of new query points; evaluate them, augment the dataset, and refit the GPR model.
4. Repeat the process until the evaluation budget is exhausted, and return the point achieving the maximum observed value of y.

Here are two commonly used acquisition functions in Bayesian Optimization:

1. **Expected Improvement (EI)**:
$$a(x) = \mathbb{E}\big[(y(x) - y_{\max})^+\big], \tag{6.52}$$
where
$$[z]^+ := \max(0, z). \tag{6.53}$$
Under the GPR model, the predictive distribution is
$$y(x) \sim \mathcal{N}\big(\mu(x),\, \Sigma(x)\big), \tag{6.54}$$
so EI admits a closed-form expression obtained by integrating over the tail above $y_{\max}$:
$$a(x) = \int_{y_{\max}}^{+\infty} \big(y(x) - y_{\max}\big)\, \mathcal{N}(y(x) \,|\, \mu(x),\, \Sigma(x))\, \mathrm{d}y(x). \tag{6.55}$$
2. **Upper Confidence Bound (UCB)**:
$$a(x) = \mu(x) + \kappa\, \sigma(x), \tag{6.56}$$
where $\sigma(x)$ denotes the predictive standard deviation and $\kappa > 0$ controls the exploration–exploitation balance. In essence, UCB prefers points where the model is either promising (large μ) or highly uncertain (large σ).

Remark

Optuna *is a widely used automatic hyperparameter tuning library, recommended in class for practical Bayesian Optimization.*

Chapter 7 Trees and Ensemble Learning

Introduction

- *Decision Trees and Axis-Aligned Splits*
- *Information Gain and Mutual-Information*
- *Feature Selection and Purity Measures*
- *Bagging and Variance Reduction*
- *Random Forests and Out-of-Bag Evaluation*
- *Boosting and Functional Gradient View*

Review: Gaussian Processes and Bayesian Optimization

In the previous chapter, we introduced Gaussian Processes (GPs) and showed how they can be used for regression. Given training inputs

$$X = (x_1, \dots, x_n), \qquad y = (y_1, \dots, y_n)^\top, \tag{7.1}$$

and a test point $x_\star$, a GP prior with kernel $k(\cdot,\cdot)$ and i.i.d. Gaussian observation noise of variance λ yields the joint Gaussian distribution

$$\begin{pmatrix} y \\ f_\star \end{pmatrix} \sim \mathcal{N}\left(0, \begin{bmatrix} K(X,X) + \lambda I_n & k(X,x_\star) \\ k(x_\star, X) & k(x_\star, x_\star) \end{bmatrix}\right), \tag{7.2}$$

where $K(X,X)$ is the $n \times n$ Gram matrix and $k(X,x_\star) = (k(x_1,x_\star), \dots, k(x_n,x_\star))^\top$. Conditioning on the observed y gives the posterior predictive distribution for $f_\star = f(x_\star)$:

$$\mu_\star = \mathbb{E}[f_\star \mid X, y, x_\star] = k(x_\star, X)^\top \big(K(X,X) + \lambda I_n\big)^{-1} y, \tag{7.3}$$

$$\sigma_\star^2 = \mathrm{Var}[f_\star \mid X, y, x_\star] = k(x_\star, x_\star) - k(x_\star, X)^\top \big(K(X,X) + \lambda I_n\big)^{-1} k(x_\star, X). \tag{7.4}$$

By evaluating $\mu_\star$ at many test points and optionally plotting $\mu_\star \pm \sigma_\star$, we obtain a smooth predictive curve together with credible intervals that quantify uncertainty.

Black-Box Optimization with Gaussian Processes

An important application of GP regression is *Bayesian Optimization* (BO), which addresses *black-box optimization* problems:

$$\operatorname*{argmax}_{x \in \mathcal{X}} f(x), \tag{7.5}$$

where:

- $f(x)$ has no known analytic form (no closed-form expression);
- gradients $\nabla f(x)$ are unavailable;
- each evaluation of $f(x)$ is *expensive*.

We can, however, query the black box at chosen points:

$$y = f(x) + \varepsilon, \tag{7.6}$$

and use these point evaluations to build a surrogate model.

A canonical example is *hyperparameter tuning* for neural networks or other machine learning models. Here:

- x encodes a vector of hyperparameters (learning rate, width, depth, regularization strength, …);
- $f(x)$ is the validation performance (e.g. accuracy) obtained by training the model with hyperparameters x and evaluating on a validation set.

Each function evaluation requires a full train–validate cycle and is therefore very costly, so we must find good x using as *few* evaluations as possible.

Remark

Bayesian Optimization with a GP surrogate proceeds iteratively:

1. ***Initialization.*** *Select an initial design* $\{x_i\}_{i=1}^n$ *(e.g. random or space-filling) and evaluate* $y_i = f(x_i)$.
2. ***Fit GP surrogate.*** *Use* $\{(x_i, y_i)\}_{i=1}^n$ *to fit a GP regression model, yielding posterior mean* $\mu(x)$ *and variance* $\sigma^2(x)$ *for all* $x \in \mathcal{X}$.
3. ***Acquisition maximization.*** *Define an* acquisition function $a(x)$ *that uses* $\mu(x)$ *and* $\sigma(x)$ *to score candidate points, and choose the next evaluation point(s) by*
$$x_{next} \in \operatorname*{argmax}_{x \in \mathcal{X}} a(x). \tag{7.7}$$
4. ***Evaluate and update.*** *Query the black box at* x_{next} *to obtain* $y_{next} = f(x_{next})$*, augment the dataset, and refit (or update) the GP surrogate.*
5. ***Repeat*** *Steps 2–4 until a pre-specified evaluation budget is exhausted, then return the best observed* y *(or its maximizer).*

The acquisition function is responsible for balancing exploration *(sampling uncertain regions where* $\sigma(x)$ *is large) and* exploitation *(sampling near currently promising regions where* $\mu(x)$ *is large).*

Acquisition Functions

Let $y_{\max}$ denote the best function value observed so far, and suppose the GP posterior at x is Gaussian:
$$f(x) \mid \mathcal{D} \sim \mathcal{N}\big(\mu(x), \sigma^2(x)\big). \tag{7.8}$$

Two widely used acquisition functions are:

- **Expected Improvement (EI).** Define the *improvement* at x as
$$I(x) = \big(f(x) - y_{\max}\big)_+ = \max\{f(x) - y_{\max}, 0\}. \tag{7.9}$$
The Expected Improvement is
$$a_{\mathrm{EI}}(x) = \mathbb{E}[I(x) \mid \mathcal{D}]. \tag{7.10}$$
Under the Gaussian posterior, this has a closed form:
$$a_{\mathrm{EI}}(x) = (\mu(x) - y_{\max})\Phi(z) + \sigma(x)\phi(z), \qquad z = \frac{\mu(x) - y_{\max}}{\sigma(x)}, \tag{7.11}$$
where Φ and ϕ are the CDF and PDF of the standard normal distribution, respectively. EI focuses on regions that, on average, are likely to yield improvements over the current best value.
- **Upper Confidence Bound (UCB).** For a parameter $\kappa > 0$, define
$$a_{\mathrm{UCB}}(x) = \mu(x) + \kappa\,\sigma(x). \tag{7.12}$$
When $\kappa = 1$, $a_{\mathrm{UCB}}(x)$ follows the upper envelope $\mu(x) + \sigma(x)$ of the GP posterior. Larger κ encourages more exploration (sampling high-uncertainty regions), while smaller κ favors exploitation.

In practice, BO alternates between refining the surrogate in regions where data are scarce and zooming in on promising areas, progressively improving our estimate of both the location and value of the optimum.

Remark

Many modern hyperparameter optimization libraries implement Bayesian Optimization with GP or related surrogates. For example, `Optuna` *provides a flexible interface where the user specifies:*

- *the search space of hyperparameters;*

- *an evaluation budget (maximum number of trials);*
- *an objective function that trains and evaluates the model.*

The library then automatically manages the BO loop, returning (approximately) optimal hyperparameters within the given budget.

7.1 Decision Trees and Feature Selection

7.1.1 Decision Trees and Axis-Aligned Splits

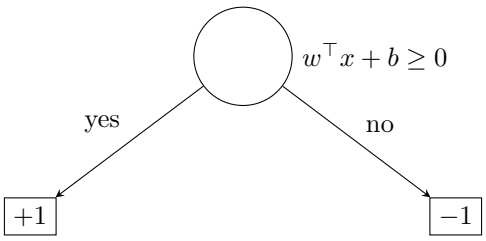
Previously, we used linear models (or linear models in a high-dimensional feature space) to perform binary classification. The classifier can be written as

$$f(x) = \begin{cases} +1, & \text{if } w^\top x + b \geq 0, \\ -1, & \text{if } w^\top x + b < 0. \end{cases} \tag{7.13}$$

The decision boundary is the hyperplane

$$w^\top x + b = 0. \tag{7.14}$$

Another way to obtain more flexible, nonlinear decision boundaries is to use *tree-based models*. A simple decision tree implementing the rule above can be drawn as follows:



Definition 7.1 (Decision Tree)

A decision tree *is a tree consisting of a root node, internal nodes, leaf nodes, and directed edges. Each internal node partitions the data according to some feature. For any input example, its final prediction is given by the label of the leaf node it reaches.* ♣

Example 7.1

We consider a binary classification setting where the label set is

$$Y \in \{-1, +1\}. \tag{7.15}$$

Here, the interpretation is:

$$+1: \text{ a good researcher}, \qquad -1: \text{ a bad researcher}. \tag{7.16}$$

Assume we have three binary features:

- Feature A:

$$+1: \text{ hardworking}, \qquad -1: \text{ not hardworking}. \tag{7.17}$$

- Feature B:

$$+1: \text{ has good vision}, \qquad -1: \text{ not}. \tag{7.18}$$

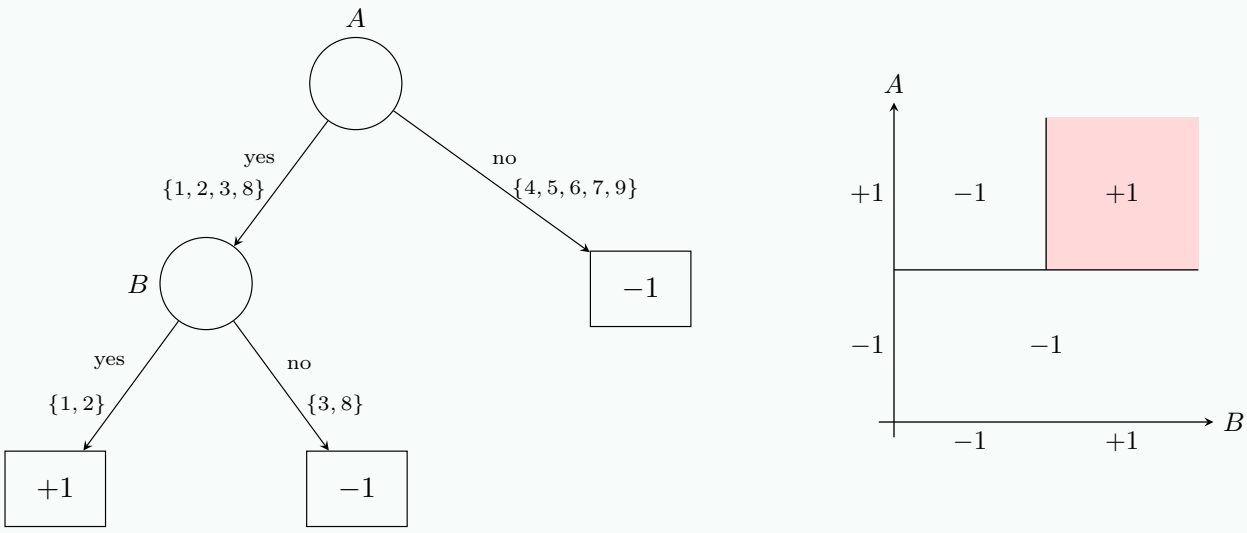
ID	A	B	C	y
1	✓	✓	✓	✓
2	✓	✗	✗	✓
3	✓	✗	✓	✗
4	✗	✓	✓	✗
5	✗	✗	✗	✗
6	✗	✗	✗	✗
7	✗	✓	✗	✗
8	✓	✗	✓	✗
9	✗	✓	✓	✗

- Feature C:

$$+1: \text{ likes bananas}, \qquad -1: \text{ not}. \tag{7.19}$$

A decision tree may use these features at its internal nodes to determine whether an example will ultimately be predicted as $+1$ or -1.

Solution



If we choose feature C as the root node, the samples are not separated as cleanly as when we use feature A. This leads to a natural question: *given many candidate features, how can we choose the one that produces the "purest" split at the root?*

From the example above, a good feature should split the training set into subsets whose labels are as pure (i.e., as close to all positive or all negative) as possible. To quantify this notion of purity—or, conversely, of randomness—we introduce the notion of entropy.

7.1.2 Entropy, Information Gain, and Mutual Information

Definition 7.2 (Entropy)

For a discrete random variable X with distribution P, the (Shannon) entropy of P is defined as

$$H(P) := -\sum_x P(x) \log P(x). \tag{7.20}$$

Entropy measures the intrinsic uncertainty of the distribution. Equivalently,

$$H(P) = \mathbb{E}_{X\sim P}\big[-\log P(X)\big], \tag{7.21}$$

so $-\log P(X)$ can be viewed as the information content *of the outcome X, and $H(P)$ is its average over many draws.* ♣

Entropy has several essential properties.

1. **Non-negativity.** Since $\log P(x) \le 0$ for all x, we have

$$H(P) = -\sum_x P(x) \log P(x) \ge 0, \tag{7.22}$$

with equality if and only if the distribution is degenerate (i.e., $P(x) = 1$ for some x).

2. **Upper bound.** Using Jensen's inequality:

$$H(P) = -\mathbb{E}_P[\log P(X)] \leq -\log \mathbb{E}_P[P(X)] = -\log \sum_x P(x)^2. \tag{7.23}$$

In particular, for a distribution supported on n distinct outcomes,

$$H(P) \leq \log n, \tag{7.24}$$

and equality holds if and only if P is the uniform distribution:

$$P(x) = \frac{1}{n}. \tag{7.25}$$

Remark

Beyond measuring uncertainty, entropy also characterizes the fundamental limit of lossless data compression. For a discrete source with distribution P, any prefix-free coding scheme assigns a codeword of length $\ell(x)$ to each symbol x. Shannon's source coding theorem states that the expected code length satisfies

$$\mathbb{E}[\ell(X)] \;\geq\; H(P) := -\sum_x P(x) \log P(x), \tag{7.26}$$

with equality achievable asymptotically.
Thus, the entropy $H(P)$ represents the minimal achievable average number of bits required to encode samples drawn from P. In this sense, entropy quantifies both the intrinsic uncertainty of a distribution and the optimal compression rate for a lossless encoding.

Definition 7.3 (Cross-Entropy)

Let P and Q be two distributions over the same support. The cross-entropy between P and Q is defined as

$$H(P, Q) := -\sum_x P(x) \log Q(x) = \mathbb{E}_P[-\log Q(X)]. \tag{7.27}$$

♣

Observe:

$$H(P, Q) - H(P) = -\sum_x P(x) \log Q(x) + \sum_x P(x) \log P(x) = \sum_x P(x) \log \frac{P(x)}{Q(x)}. \tag{7.28}$$

Definition 7.4 (Kullback–Leibler (KL) Divergence)

Let P and Q be two probability distributions over the same support. The Kullback–Leibler divergence from P to Q is defined as

$$D_{\mathrm{KL}}(P \,\|\, Q) := \sum_x P(x) \log \frac{P(x)}{Q(x)} \tag{7.29}$$

in the discrete case, or

$$D_{\mathrm{KL}}(P \,\|\, Q) := \int p(x) \log \frac{p(x)}{q(x)} \, \mathrm{d}x \tag{7.30}$$

in the continuous case.
KL divergence measures how different the distribution P is from Q. It satisfies

$$D_{\mathrm{KL}}(P \| Q) \geq 0, \tag{7.31}$$

with equality if and only if $P = Q$ almost everywhere.

♣

Thus,

$$H(P, Q) - H(P) = D_{\mathrm{KL}}(P \| Q) \geq 0, \tag{7.32}$$

with equality if and only if $P = Q$.

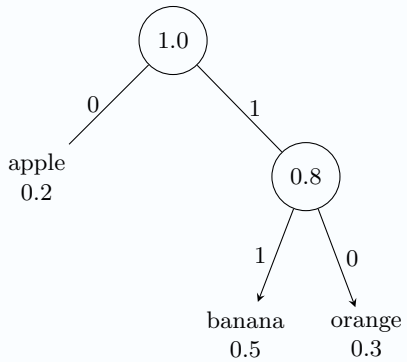
Cross-entropy decomposes into:

$$H(P, Q) = H(P) + D_{\mathrm{KL}}(P \| Q), \tag{7.33}$$

meaning that cross-entropy is always at least as large as entropy and grows with the divergence between P and Q.

Remark

Huffman coding is a greedy algorithm for constructing an optimal prefix-free code when the symbol probabilities are known. More frequent symbols receive shorter codewords and rare symbols receive longer ones. Shannon's source coding theorem guarantees that the expected code length of a Huffman code is no larger than $H(P) + 1$ bits per symbol, so Huffman coding is essentially optimal among all lossless prefix codes.



Huffman coding is tightly connected to entropy and cross-entropy.

1. Entropy as the optimal code length. *For a source with distribution P, the entropy*

$$H(P) = -\sum_x P(x) \log P(x) \tag{7.34}$$

is the theoretical lower bound on the average number of bits needed to encode symbols drawn from P. Huffman coding produces the optimal prefix-free code, and its expected code length L_{Huff} satisfies

$$H(P) \leq L_{\mathrm{Huff}} < H(P) + 1. \tag{7.35}$$

2. Cross-entropy as the cost of using a wrong code. *Suppose we design a Huffman code based on a* wrong *distribution Q but the true data come from P. Then the expected code length becomes*

$$\mathbb{E}_P[-\log Q(X)] = H(P, Q), \tag{7.36}$$

which is the cross-entropy between P and Q.

3. Extra penalty equals KL divergence. *The additional number of bits required due to the mismatch between P and Q is*

$$H(P, Q) - H(P) = D_{\mathrm{KL}}(P \| Q). \tag{7.37}$$

Thus, the KL divergence quantifies the inefficiency of using a code optimized for Q when the true distribution is P.

Summary. *Entropy gives the minimal achievable rate, Huffman coding attains it (up to one bit), and cross-entropy / KL divergence measure how much extra cost arises when the coding distribution does not match the true distribution.*

Theorem 7.1 (Shannon's Source Coding Theorem*)

Let X be a discrete memoryless source with distribution P. For any prefix-free code with codeword lengths $\{\ell(x)\}$, the expected code length satisfies

$$\mathbb{E}[\ell(X)] \;\geq\; H(P). \tag{7.38}$$

Moreover, for every $\varepsilon > 0$, there exists a prefix-free code such that

$$\mathbb{E}[\ell(X)] \;\leq\; H(P) + \varepsilon, \tag{7.39}$$

for sufficiently long block coding. Thus $H(P)$ is the optimal achievable rate for lossless compression. ♡

Proof

The theorem consists of two parts: a converse (no code beats entropy) and an achievability (entropy can be approached arbitrarily closely).

1. Converse: No prefix-free code can beat $H(P)$. *For any prefix-free code, Kraft's inequality gives*

$$\sum_x 2^{-\ell(x)} \leq 1. \tag{7.40}$$

Multiplying both sides by $P(x)$ and applying Jensen's inequality to the convex function $-\log(\cdot)$ yields

$$\mathbb{E}[\ell(X)] \;=\; \sum_x P(x)\ell(x) \;\geq\; -\sum_x P(x)\log P(x) \;=\; H(P). \tag{7.41}$$

Thus entropy is a lower bound on any valid coding scheme.

2. Achievability: Constructing codes approaching $H(P)$. *Consider n i.i.d. samples X^n from the source. The Asymptotic Equipartition Property (AEP) states that for any $\varepsilon > 0$, with probability approaching 1, sequences in the typical set $\mathcal{T}_\varepsilon^{(n)}$ satisfy*

$$-\frac{1}{n}\log P(X^n) \in [H(P) - \varepsilon,\; H(P) + \varepsilon]. \tag{7.42}$$

The typical set contains approximately $2^{nH(P)}$ sequences. We can therefore assign to each typical sequence a codeword of length

$$nH(P) + o(n),$$

while atypical sequences receive longer codewords but contribute negligibly to the expected length. This yields a block code with

$$\mathbb{E}[\ell(X^n)] \leq n(H(P) + \varepsilon). \tag{7.43}$$

3. Conclusion. *Combining the converse and achievability gives the optimal rate:*

$$\lim_{n\to\infty} \frac{1}{n}\mathbb{E}[\ell(X^n)] = H(P).$$

Thus $H(P)$ is both a fundamental lower bound and an achievable rate for lossless source coding. ■

To measure how "pure" the labels in a dataset are, we use the notion of *information gain*.

Let D be a training set. Labels lie in a finite set $Y \in \{1, \ldots, K\}$. For each label k, define the subset

$$C_k := \{x \in D : y(x) = k\}. \tag{7.44}$$

The entropy of D with respect to its empirical label distribution is

$$H(D) = -\sum_{k=1}^{K} \frac{|C_k|}{|D|} \log \frac{|C_k|}{|D|} \approx -\sum_{y=1}^{K} P(y)\log P(y) = H(Y), \tag{7.45}$$

where $P(y)$ denotes the empirical frequency of label y.

Let A be a feature taking values in $\{a_1, \ldots, a_m\}$, and let

$$D_{a_i} := \{x \in D : A(x) = a_i\} \tag{7.46}$$

be the subset of samples with feature value a_i.

The conditional entropy of labels given the feature is

$$H(D \mid A) = \sum_{i=1}^{m} \frac{|D_{a_i}|}{|D|} H(D_{a_i}) = \sum_{i=1}^{m} \frac{|D_{a_i}|}{|D|} \left(-\sum_{k=1}^{K} \frac{|D_{a_i} \cap C_k|}{|D_{a_i}|} \log \frac{|D_{a_i} \cap C_k|}{|D_{a_i}|} \right), \tag{7.47}$$

Note

$$H(D \mid A) = \mathbb{E}_{x|A}[H(D_x)] \tag{7.48}$$

The *information gain* of feature A is defined as

Definition 7.5 (Information gain (Mutual Information))

$$g(D, A) = H(D) - H(D \mid A). \tag{7.49}$$

A large value of $g(D, A)$ means that feature A significantly reduces label uncertainty, producing purer label subsets and therefore acting as a good splitting feature in decision tree construction.

More generally, for two random variables X and Y, their *mutual information* is

$$I(X;Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) = H(X) + H(Y) - H(X,Y). \tag{7.50}$$

In our setting, information gain is exactly the mutual information between the feature and the label:

$$g(D, A) \approx I(Y; A). \tag{7.51}$$

We therefore choose the best feature A by

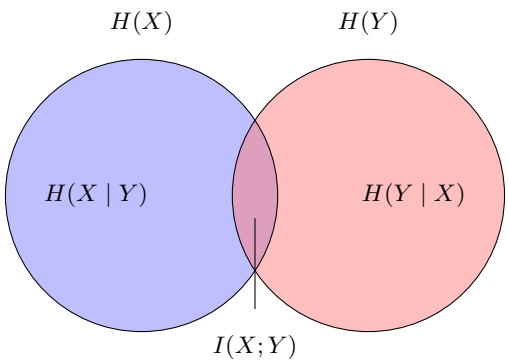
$$A^\star = \operatorname*{argmax}_A g(D, A). \tag{7.52}$$

If two random variables satisfy $I(X;Y) = 0$, then they are independent: knowing one of them does not reduce the uncertainty about the other.

Mutual information has several important properties:

1. **Symmetry.** $I(X;Y) = I(Y;X)$.
2. **Non-negativity.** $I(X;Y) \geq 0$, with equality if and only if X and Y are independent.
3. **Alternative forms.** $I(X;Y) = H(X) + H(Y) - H(X,Y)$.

The larger the mutual information, the more correlated X and Y are. When X and Y are highly correlated, knowing X significantly reduces the uncertainty about Y, and vice versa.



7.1.3 Information Gain Ratio

Information gain has a significant drawback: it tends to favor features with many possible values.

Example 7.2

Consider a dataset with two features: A (taking values a_1, a_2) and B (taking values $b_1, \ldots, b_{10}$). Suppose that:

- Feature A splits the data into two subsets, each containing 5 classes uniformly.
- Feature B splits the data into 10 pure subsets, each containing only one class.

Then $g(D, A) = \log 10 - \log 5 = \log 2$, while $g(D, B) = \log 10 - 0 = \log 10$. Information gain would prefer feature B, even though it may have poor generalization ability.

More extremely, if we use the index (row number) as a feature, each example would fall into its own pure subset, yielding maximum information gain $g(D, \text{index}) = H(D)$, but such a feature has no generalization ability for new test examples.

To address this issue, we introduce the *information gain ratio*, which penalizes features with many values.

Definition 7.6 (Information Gain Ratio)

The information gain ratio of feature A is defined as

$$\mathrm{GR}(D, A) = \frac{g(D, A)}{H(D, A)}, \tag{7.53}$$

where $H(D, A)$ is the entropy of feature A itself:

$$H(D, A) = -\sum_{i=1}^{m} \frac{|D_{a_i}|}{|D|} \log \frac{|D_{a_i}|}{|D|}. \tag{7.54}$$

This measures the intrinsic uncertainty of the feature A's distribution. ♣

The denominator $H(D, A)$ penalizes features with many values. When A has many uniformly distributed values, $H(D, A)$ approaches $\log m$, which reduces the gain ratio. This helps prevent overfitting to features that achieve high purity simply by having many possible values.

7.1.4 Gini Index

Another common purity measure is the *Gini index*, which serves as an alternative to entropy.

Definition 7.7 (Gini Index)

For a dataset D with labels in $\{1, \ldots, K\}$, the Gini index is defined as

$$\mathrm{Gini}(D) = 1 - \sum_{k=1}^{K} \left(\frac{|C_k|}{|D|}\right)^2 = \sum_{k=1}^{K} \frac{|C_k|}{|D|} \left(1 - \frac{|C_k|}{|D|}\right), \tag{7.55}$$

where $C_k = \{x \in D : y(x) = k\}$. ♣

The Gini index measures the expected error rate if we randomly label examples according to the class distribution. It satisfies:

- $\mathrm{Gini}(D) = 0$ when D is pure (all examples have the same label).
- $\mathrm{Gini}(D)$ is maximized when the class distribution is uniform, taking the value $1 - 1/K$ for K classes.
- For binary classification ($K = 2$), $\mathrm{Gini}(D) = 2p(1 - p)$, where p is the proportion of the positive class.

The conditional Gini index given feature A is:

$$\mathrm{Gini}(D \mid A) = \sum_{i=1}^{m} \frac{|D_{a_i}|}{|D|} \mathrm{Gini}(D_{a_i}). \tag{7.56}$$

For feature selection, we choose the feature that minimizes the conditional Gini index:

$$A^\star = \operatorname*{argmin}_{A} \mathrm{Gini}(D \mid A). \tag{7.57}$$

Remark

For binary classification, the Gini index and entropy (scaled by $1/2$) are very similar functions of the class probability p. Both achieve their minimum at $p \in \{0, 1\}$ and maximum at $p = 1/2$.

For regression problems, where labels are continuous, we use a different criterion based on squared loss.

Definition 7.8 (L2 Loss Criterion)

For a regression problem, after splitting by feature A into subsets $D_1, \ldots, D_m$, define the mean label in each subset:

$$\bar{y}_{D_i} = \frac{1}{|D_i|} \sum_{j \in D_i} y_j. \tag{7.58}$$

The L2 loss criterion is:

$$L(D, A) = \sum_{i=1}^{m} \sum_{j \in D_i} (y_j - \bar{y}_{D_i})^2. \tag{7.59}$$

We choose the feature that minimizes this criterion:

$$A^\star = \operatorname*{argmin}_A L(D, A). \tag{7.60}$$

♣

This criterion measures the variance within each subset after splitting. A smaller L2 loss indicates purer subsets, where examples in each subset have labels close to their subset mean. The mean $\bar{y}_{D_i}$ serves as the prediction for examples in subset D_i.

7.1.5 Building Decision Trees: Greedy Algorithm

Given multiple feature selection criteria (information gain, information gain ratio, Gini index, or L2 loss), we can build a decision tree. The naive approach of enumerating all possible trees is computationally infeasible (super-exponential complexity). Instead, we use a *greedy algorithm*.

The greedy algorithm builds the tree recursively:

Input: Training set D, feature set $\mathcal{F}$.

Procedure:

1. **Choose a feature** $A \in \mathcal{F}$ according to a purity metric (e.g., information gain, information gain ratio, Gini index, or L2 loss).
2. **Partition** D into subsets $D_1, \ldots, D_m$ based on feature A's values $a_1, \ldots, a_m$.
3. **Recursively build subtrees** for each subset D_i using the remaining features $\mathcal{F} \setminus \{A\}$.
   Why remove A? Since all examples in D_i share the same value for feature A, further splitting by A would be redundant.
4. **Termination conditions:**
   - The feature set $\mathcal{F}$ is empty (no more features available).
   - The subset D_i is pure (contains only one class for classification, or has very low variance for regression).
   - Maximum depth is reached.
   - Purity improvement is below a threshold.
   - Number of samples in a node is below a minimum threshold.
5. When termination occurs, the node becomes a **leaf node** with label equal to the majority class (for classification) or the mean value (for regression) in that subset.

Prediction: For a test example, traverse the tree according to its feature values until reaching a leaf node, then use the leaf's label as the prediction.

Remark

The naive approach of enumerating all possible trees would require exploring all permutations of feature selection orders, leading to super-exponential complexity. The greedy algorithm reduces this to polynomial time by making locally optimal choices at each step, though it does not guarantee global optimality.

Remark

To prevent overfitting, common regularization techniques include:

- *Limiting the maximum depth of the tree.*
- *Setting a minimum number of samples required to split a node.*
- *Setting a minimum purity improvement threshold.*
- *Using a validation set to monitor performance and stop early.*

Remark

The greedy algorithm is heuristic: it makes locally optimal choices at each step but does not guarantee a globally optimal tree. Despite this, decision trees and their ensemble variants (Random Forests, Gradient Boosting) are among the most powerful methods in practice, especially for tabular data.

7.1.6 Continuous Features in Decision Trees

In the preceding discussion, we implicitly assumed that features are discrete, taking values in a finite set. For such a feature A with possible values $\{a_1, \ldots, a_m\}$, a split on A partitions the training set into m subsets $D_{a_1}, \ldots, D_{a_m}$.

In practice, many features are continuous. To handle a continuous feature A, we can discretize it by introducing *thresholds*. Let

$$\{a^{(1)}, \ldots, a^{(n)}\} \tag{7.61}$$

be the distinct values of A observed in the training set (for some feature dimension). Sort them so that

$$a^{(1)} < a^{(2)} < \cdots < a^{(n)}. \tag{7.62}$$

Candidate thresholds can then be placed between successive values:

$$\tau_j = \frac{a^{(j)} + a^{(j+1)}}{2}, \qquad j = 1, \ldots, n-1. \tag{7.63}$$

Each threshold τ_j defines a binary split

$$A \leq \tau_j \quad \text{vs.} \quad A > \tau_j, \tag{7.64}$$

which we can evaluate using any of our usual criteria (information gain, Gini index, squared loss for regression, etc.). By scanning over all candidate thresholds and choosing the best one according to the purity measure, the tree handles continuous features while remaining axis-aligned.

At prediction time, a test example with continuous feature value $A(x)$ is routed through the tree by comparing $A(x)$ with the learned thresholds at each internal node, just as if the feature had been discrete.

7.2 From Single Trees to Ensembles

So far, we have seen how to build and select a *single* decision tree, by choosing features that maximize information gain at each split. In practice, however, a single tree often suffers from two issues:

- **High variance.** Small changes in the training data may lead to very different trees, especially when the tree is grown

deep and fits the training set closely.

- **Limited accuracy.** A single tree is easy to interpret, but its predictive performance may lag behind more powerful models such as kernel methods or neural networks.

Ensemble learning addresses these issues by combining the predictions of many base learners (often decision trees) to form a stronger model.

Definition 7.9 (Ensemble Learning)

Ensemble learning constructs a predictor by aggregating a collection of base learners (also called weak learners*). Given base predictors $f_1, \dots, f_M$, the ensemble predictor takes the form*

$$F(x) = \begin{cases} \text{majority vote}\big(f_1(x), \dots, f_M(x)\big), & \textit{(classification)}, \\ \dfrac{1}{M} \sum_{m=1}^{M} f_m(x), & \textit{(regression)}. \end{cases} \tag{7.65}$$

The key idea is that even if each f_m is only moderately accurate, their combination can be significantly more accurate and more robust. ♣

7.2.1 Bias–Variance Decomposition

Consider a supervised learning setting with training dataset D, drawn from some unknown data-generating distribution. Let x denote a test input with true label y, and let

$$f(x; D) \tag{7.66}$$

denote the prediction of a learning algorithm trained on D when evaluated at x. Under squared loss, the prediction error at (x, y) for a given training set D is

$$\big(f(x; D) - y\big)^2. \tag{7.67}$$

Since D is random, it is natural to measure performance by averaging over the random draw of D. We therefore consider

$$\mathbb{E}_D\Big[\big(f(x; D) - y\big)^2\Big], \tag{7.68}$$

which quantifies the expected squared error at x, averaged over the randomness in the training data.

Define the *average prediction* at x as

$$\bar{f}(x) := \mathbb{E}_D\big[f(x; D)\big], \tag{7.69}$$

that is, the expected prediction we would obtain if we could repeatedly sample training sets D from the underlying distribution and retrain the model. Insert and subtract $\bar{f}(x)$ inside the square:

$$\mathbb{E}_D\Big[\big(f(x; D) - y\big)^2\Big] = \mathbb{E}_D\Big[\big(f(x; D) - \bar{f}(x) + \bar{f}(x) - y\big)^2\Big] \tag{7.70}$$

$$= \mathbb{E}_D\Big[\big(f(x; D) - \bar{f}(x)\big)^2\Big] + \big(\bar{f}(x) - y\big)^2 \tag{7.71}$$

$$+ 2\,\mathbb{E}_D\Big[\big(f(x; D) - \bar{f}(x)\big)\big(\bar{f}(x) - y\big)\Big]. \tag{7.72}$$

The cross term vanishes because

$$\mathbb{E}_D[f(x; D) - \bar{f}(x)] = \mathbb{E}_D[f(x; D)] - \bar{f}(x) = 0, \tag{7.73}$$

and $\bar{f}(x) - y$ does not depend on D. Hence

$$\mathbb{E}_D\Big[\big(f(x; D) - y\big)^2\Big] = \underbrace{\mathbb{E}_D\Big[\big(f(x; D) - \bar{f}(x)\big)^2\Big]}_{\text{variance at } x} + \underbrace{\big(\bar{f}(x) - y\big)^2}_{\text{bias}^2 \text{ at } x}. \tag{7.74}$$

Definition 7.10 (Bias and variance at a test point)

For a fixed test input x with true label y, the variance *and* (squared) bias *of a learning algorithm are defined as*

$$\mathrm{Var}_D\left[f(x;D)\right] = \mathbb{E}_D\left[\left(f(x;D) - \bar{f}(x)\right)^2\right], \tag{7.75}$$

$$\mathrm{Bias}^2(x) = \left(\bar{f}(x) - y\right)^2. \tag{7.76}$$

♣

Intuitively:

- The variance term measures how sensitive the predictor $f(x;D)$ is to fluctuations in the training data: for different draws of D, predictions at x may vary significantly around their mean $\bar{f}(x)$.
- The bias term measures how far, on average, the prediction is from the true label y. Even if we could train on infinitely many different datasets, the average predictor $\bar{f}(x)$ might still systematically miss the true target due to limited model capacity or misspecification.

High variance typically arises from *overfitting* particular training sets, while high bias indicates that the model class itself is too simple to capture the underlying relationship, even when averaged over many datasets.

Example 7.3 – Linear model on nonlinear data

Suppose the data arise from a quadratic relationship, but we insist on fitting a linear model. Even if we repeatedly resample training sets and average the resulting fitted lines, the averaged predictor $\bar{f}(x)$ remains linear and cannot match the true quadratic curve everywhere. This manifests as a persistent bias term.

Bias and variance suggest complementary strategies:

- To reduce **variance**, we can increase the training set size or average multiple diverse models (as in bagging and random forests).
- To reduce **bias**, we can increase model capacity, e.g. by using more flexible hypothesis classes or boosting weak learners.

Two classical ensemble strategies are:

- *Bagging* (Bootstrap Aggregating): build many base learners independently on randomized versions of the data, and average or vote.
- *Boosting*: build base learners sequentially, each one focusing on correcting the mistakes of the previous ones.

In modern machine learning practice, decision-tree ensembles based on bagging and boosting are among the most powerful off-the-shelf methods, with famous examples including Random Forests, Gradient Boosting Trees, and XGBoost.

7.2.2 Bagging: Bootstrap Aggregating

Bagging is designed to reduce the variance of an unstable base learner (such as a deep decision tree) by averaging many independently trained copies.

Definition 7.11 (Bagging)

Let D be a training dataset of size n, and let $\mathcal{A}$ be a base learning algorithm that outputs a predictor $f = \mathcal{A}(D)$. Bagging constructs M bootstrap datasets $D^{(1)}, \ldots, D^{(M)}$, where each $D^{(m)}$ is obtained by sampling n points with replacement *from D.*
On each bootstrap dataset $D^{(m)}$, we train a base learner

$$f_m = \mathcal{A}\left(D^{(m)}\right), \qquad m = 1, \ldots, M. \tag{7.77}$$

The bagged ensemble predictor is

$$F_{\text{bag}}(x) = \begin{cases} \operatorname{sign}\Big(\frac{1}{M}\sum_{m=1}^{M} f_m(x)\Big), & \textit{classification}, \\ \frac{1}{M}\sum_{m=1}^{M} f_m(x), & \textit{regression}. \end{cases} \tag{7.78}$$

♣

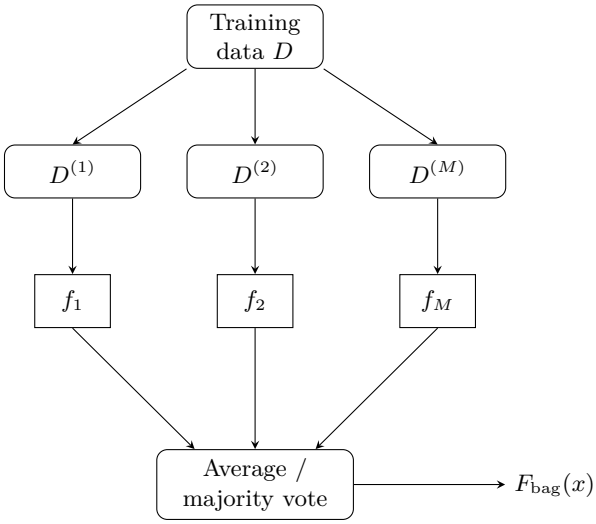
Intuitively, each bootstrap sample $D^{(m)}$ can be viewed as a noisy version of the original dataset. The base learners f_m will differ from one another, and averaging their predictions cancels out part of the randomness.

Remark

Consider a simplified setting where we average M identically distributed base predictors $f_1, \ldots, f_M$ for regression, each with variance σ^2 and pairwise correlation ρ. Then the variance of the averaged predictor is

$$\operatorname{Var}\big(F_{\text{bag}}(x)\big) = \frac{1}{M^2}\sum_{m=1}^{M}\sum_{m'=1}^{M} \operatorname{Cov}\big(f_m(x), f_{m'}(x)\big) \approx \sigma^2\left(\rho + \frac{1-\rho}{M}\right). \tag{7.79}$$

As M grows, the second term $(1-\rho)/M$ vanishes, so the variance is dominated by $\rho\sigma^2$. Thus, bagging is most effective when base learners are accurate but have high variance and are not too strongly correlated.



7.2.3 Random Forests

Random Forests specialize bagging to decision trees, and further inject randomness at the *feature* level to decorrelate the trees.

Definition 7.12 (Random Forest)

A Random Forest is an ensemble of decision trees trained with two sources of randomness:

1. ***Bootstrap sampling of data.*** *For each tree m, sample a bootstrap dataset $D^{(m)}$ from D.*
2. ***Random feature selection at each split.*** *When splitting a node, instead of considering all features, randomly select a subset $\mathcal{F}$ of features (of size d_{sub}), and choose the best split only among features in $\mathcal{F}$.*

The final prediction aggregates all trees by majority vote or averaging, as in bagging.

♣

The additional randomness in feature selection has two important effects:

- It reduces the correlation between different trees, which strengthens the variance reduction effect of averaging.

- It forces each tree to explore different feature combinations, sometimes discovering useful patterns that a single greedy tree might miss.

In practice, Random Forests are strong general-purpose models with relatively few hyperparameters. Common choices include:

- the number of trees M (often in the hundreds),
- the maximum depth of each tree,
- the number of features d_{sub} considered at each split (e.g., $\sqrt{d}$ for classification, where d is the total number of features).

Remark

In each bootstrap sample $D^{(m)}$, roughly a fraction $1 - 1/e \approx 0.63$ of the original training points are included, and the remaining points are left out. For any training example, we can average the predictions of all trees that did not *see this example during training; this is called the* out-of-bag *prediction. Aggregating these predictions over the training set provides an internal estimate of the generalization error, without using a separate validation set.*

7.2.4 Boosting

While bagging focuses on variance reduction by averaging many independent learners, boosting builds an ensemble *sequentially*. Each new learner tries to correct the mistakes of the current ensemble, effectively turning a collection of weak learners into a strong one.

Definition 7.13 (Boosting)

Given a training set $D = \{(x_i, y_i)\}_{i=1}^n$, boosting maintains an ensemble predictor

$$F_0(x) \equiv 0, \qquad F_M(x) = \sum_{m=1}^{M} \alpha_m f_m(x), \tag{7.80}$$

where each f_m is a base learner (often a shallow tree) and the weights α_m control their influence.
At iteration m, boosting chooses f_m to focus on the current residual errors or misclassified examples of F_{m-1}, and then updates the ensemble to F_m.

♣

One classical example is AdaBoost for binary classification. It maintains a distribution of weights over training samples, gives higher weights to misclassified points, and fits a new weak learner to this reweighted dataset at each iteration.

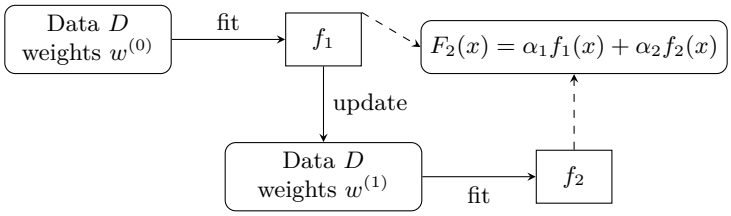
Remark

- *Bagging trains base learners* in parallel *on resampled datasets and primarily reduces variance.*
- *Boosting trains base learners* sequentially, *focusing on difficult samples, and can reduce both bias and variance, but is also more prone to overfitting if not regularized (e.g., via tree depth, learning rate, or early stopping).*

Modern implementations such as Gradient Boosting Trees and XGBoost can be viewed as performing a form of functional gradient descent in function space, where each tree fits the negative gradient of the loss with respect to the current ensemble prediction.

AdaBoost can be derived as stagewise minimization of the exponential loss

$$\mathcal{L}(F) = \sum_{i=1}^{n} \exp\big(-y_i F(x_i)\big), \qquad y_i \in \{-1, +1\}.$$



Suppose at round m we update

$$F_m(x) = F_{m-1}(x) + \alpha h(x), \qquad h(x) \in \{-1, +1\}.$$

Then

$$\begin{aligned}\mathcal{L}(F_m) &= \sum_{i=1}^{n} \exp\big(-y_i(F_{m-1}(x_i) + \alpha h(x_i))\big) \\ &= \sum_{i=1}^{n} \exp\big(-y_i F_{m-1}(x_i)\big) \exp\big(-\alpha y_i h(x_i)\big). \end{aligned} \tag{7.81}$$

Define the normalized weights (sigmoid)

$$w_i^{(m)} = \frac{\exp\big(-y_i F_{m-1}(x_i)\big)}{\sum_{j=1}^{n} \exp\big(-y_j F_{m-1}(x_j)\big)}$$

Minimizing $\mathcal{L}(F_m)$ w.r.t. α is equivalent to minimizing

$$Z(\alpha) := \sum_{i=1}^{n} w_i^{(m)} \exp\big(-\alpha y_i h(x_i)\big).$$

Let the weighted error of h under $w^{(m)}$ be

$$\varepsilon = \sum_{i=1}^{n} w_i^{(m)} \mathbb{1}\big(h(x_i) \neq y_i\big).$$

Note that $y_i h(x_i) = +1$ if the point is correctly classified and $y_i h(x_i) = -1$ otherwise. Splitting the sum into correct vs. incorrect sets,

$$Z(\alpha) = (1 - \varepsilon)e^{-\alpha} + \varepsilon e^{\alpha}.$$

Differentiate and set to zero:

$$\frac{\mathrm{d}Z}{\mathrm{d}\alpha} = -(1 - \varepsilon)e^{-\alpha} + \varepsilon e^{\alpha} = 0 \quad \Longrightarrow \quad e^{2\alpha} = \frac{1 - \varepsilon}{\varepsilon}.$$

Hence the minimizer is

$$\alpha^{\star} = \frac{1}{2} \log \frac{1 - \varepsilon}{\varepsilon}.$$

Substituting $\alpha^{\star}$ back gives the minimum

$$Z(\alpha^{\star}) = 2\sqrt{\varepsilon(1 - \varepsilon)},$$

which is < 1 whenever $\varepsilon < \frac{1}{2}$. This explains why AdaBoost requires each weak learner to perform slightly better than random guessing.

Example 7.4 – AdaBoost

Let's use AdaBoost to solve a one-dimensional binary classification problem.
Consider five training samples with labels $y_i \in \{-1, +1\}$:

i	1	2	3	4	5
x_i	1	2	3	4	5
y_i	+1	+1	−1	−1	+1

Solution

AdaBoost starts with uniform sample weights

$$w_i^{(1)} = \frac{1}{5}, \qquad i = 1, \ldots, 5.$$

A decision stump is parameterized by a threshold t and a polarity $s \in \{+1, -1\}$:

$$h_{t,s}(x) = \begin{cases} s, & x < t, \\ -s, & x \geq t. \end{cases}$$

Since the data are one-dimensional and sorted, candidate thresholds are the midpoints

$$t \in \{1.5,\ 2.5,\ 3.5,\ 4.5\}.$$

We evaluate the weighted classification error

$$\varepsilon_1(t, s) = \sum_{i=1}^{5} w_i^{(1)} \mathbb{1}\big(h_{t,s}(x_i) \neq y_i\big)$$

for all (t, s) and select the minimizer. The optimal choice is $(t_1, s_1) = (2.5, +1)$, i.e.

$$h_1(x) = \begin{cases} +1, & x < 2.5, \\ -1, & x \geq 2.5, \end{cases}$$

which misclassifies only sample $i = 5$. Hence

$$\varepsilon_1 = \frac{1}{5}.$$

AdaBoost assigns the weak learner a coefficient

$$\alpha_1 = \frac{1}{2} \log \frac{1 - \varepsilon_1}{\varepsilon_1} = \frac{1}{2} \log 4 = \log 2.$$

Weights are updated according to

$$w_i^{(2)} = \frac{w_i^{(1)} \exp\big(-\alpha_1 y_i h_1(x_i)\big)}{Z_1},$$

where Z_1 normalizes the weights to sum to 1. Explicitly,

$$w^{(2)} = (0.125,\ 0.125,\ 0.125,\ 0.125,\ 0.5).$$

The misclassified point x_5 now carries half of the total weight.
Using weights $w^{(2)}$, we again evaluate all candidate stumps. The minimum weighted error is attained at $(t_2, s_2) = (4.5, -1)$:

$$h_2(x) = \begin{cases} -1, & x < 4.5, \\ +1, & x \geq 4.5, \end{cases}$$

which misclassifies only samples $i = 1, 2$. Therefore,

$$\varepsilon_2 = w_1^{(2)} + w_2^{(2)} = 0.25.$$

$$\alpha_2 = \frac{1}{2} \log \frac{1 - \varepsilon_2}{\varepsilon_2} = \frac{1}{2} \log 3.$$

After two iterations, the AdaBoost classifier is

$$F_2(x) = \alpha_1 h_1(x) + \alpha_2 h_2(x), \qquad \hat{y} = \operatorname{sign}\big(F_2(x)\big).$$

We now compute the final predictions of F_2 on the five training samples. First evaluate the two stumps:

i	1	2	3	4	5
x_i	1	2	3	4	5
$h_1(x_i)$	+1	+1	−1	−1	−1
$h_2(x_i)$	−1	−1	−1	−1	+1

Hence,

$$F_2(x_1) = \alpha_1 - \alpha_2 = \log 2 - \frac{1}{2}\log 3 = \frac{1}{2}\log\frac{4}{3} > 0, \tag{7.82}$$

$$F_2(x_2) = \alpha_1 - \alpha_2 = \frac{1}{2}\log\frac{4}{3} > 0, \tag{7.83}$$

$$F_2(x_3) = -\alpha_1 - \alpha_2 < 0, \tag{7.84}$$

$$F_2(x_4) = -\alpha_1 - \alpha_2 < 0, \tag{7.85}$$

$$F_2(x_5) = -\alpha_1 + \alpha_2 = \frac{1}{2}\log 3 - \log 2 = \frac{1}{2}\log\frac{3}{4} < 0. \tag{7.86}$$

Therefore,

$$\hat{y}(x_1) = +1, \quad \hat{y}(x_2) = +1, \quad \hat{y}(x_3) = -1, \quad \hat{y}(x_4) = -1, \quad \hat{y}(x_5) = -1,$$

The last sample is still misclassified, but with a smaller margin*, since* $F_2(x_5) = -\alpha_1 + \alpha_2$ *is closer to zero than* $F_1(x_5) = -\alpha_1$.

Note

From a geometric perspective, both Random Forests and boosting-based tree ensembles can be seen as constructing a complex, highly nonlinear decision boundary by patching together many simple, axis-aligned splits. Whereas a single tree corresponds to a small set of such partitions, an ensemble can carve out increasingly intricate decision regions, often achieving state-of-the-art performance on tabular data.

Chapter 8 Unsupervised Learning

Introduction

- *Dimensionality reduction*
- *Cluster*
- *Latent-variable models and the EM algorithm*

All methods we have discussed so far fall under the category of **supervised learning**, where the training process relies on labeled data and the model is explicitly guided by pairs (x, y). In contrast, **unsupervised learning** operates without labels: the goal is to learn the underlying structure of the data and to approximate the distribution of datapoints x itself.

Definition 8.1 (Unsupervised Learning)

Let $\mathcal{X}$ be an unlabeled dataset. Unsupervised learning aims to learn a mapping $f : \mathcal{X} \to \mathcal{Z}$ or a generative model $p_\theta(x)$ that captures the intrinsic structure of the underlying data distribution $p(x)$. Typical objectives include clustering, dimensionality reduction, density estimation, and representation learning. Importantly, no labeled pairs (x, y) are observed during training. ♣

Unsupervised learning can be divided into several tasks:

1. Dimensionality reduction: datapoints are often lie in high-dimensional manifold, while the dimension that really matter only contribute as a small part. Dimensionality reduction is the process to train the model to find these crutial dimension from enormous amount of dimensions.
2. Clustering. Train model to group similar data togather.
3. Generate models. Various generative model, like SoRA, LLM/GPT, are trained to learn how to generate contents like text or image by learning the distribution of the target context.

8.1 Dimensionality reduction

8.1.1 Principal Component Analysis (PCA)

Theorem 8.1 (Principle of PCA)

PCA seeks a direction w (with $\|w\| = 1$) along which the projected data $w^\top x$ achieves the maximum possible variance. *Formally, the goal is to maximize*

$$\mathrm{Var}(w^\top X). \tag{8.1}$$

♡

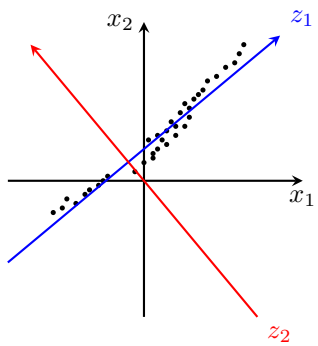
Let the dataset be

$$X = \begin{pmatrix} x_1^\top \\ x_2^\top \\ \vdots \\ x_n^\top \end{pmatrix} \in \mathbb{R}^{n \times d}, \tag{8.2}$$

with sample mean

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i. \tag{8.3}$$

Each $x_i \in \mathbb{R}^d$ is an observation.



The empirical covariance matrix is defined as

$$\Sigma = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(x_i - \bar{x})^\top \in \mathbb{R}^{d\times d}. \tag{8.4}$$

Entrywise,

$$\Sigma_{jk} = \frac{1}{n}\sum_{i=1}^{n}(x_i^{(j)} - \bar{x}^{(j)})(x_i^{(k)} - \bar{x}^{(k)}). \tag{8.5}$$

Remark

The covariance matrix defined with the factor $\frac{1}{n}$ is technically a biased estimator of the true population covariance. The unbiased version uses $\frac{1}{n-1}$ instead. However, when n is large, the difference between $\frac{1}{n}$ and $\frac{1}{n-1}$ is negligible:

$$\frac{1}{n} = \frac{1}{n-1}\left(1 - \frac{1}{n}\right), \tag{8.6}$$

and thus affects the covariance only by a vanishing scalar factor.
Moreover, PCA depends solely on the eigenvectors *of the covariance matrix, i.e., the principal directions. Multiplying Σ by any positive constant does not change its eigenvectors. Therefore, for PCA, the choice between $\frac{1}{n}$ and $\frac{1}{n-1}$ has no effect on the principal components.*

Lemma 8.1 (Variance of a Linear Projection)

Let w be any unit vector in $\mathbb{R}^d$. The projection of a data point x_i onto w is the scalar

$$z_i = w^\top x_i. \tag{8.7}$$

The sample mean of the projected data is

$$\bar{z} = \frac{1}{n}\sum_{i=1}^{n} z_i = \frac{1}{n}\sum_{i=1}^{n} w^\top x_i = w^\top \bar{x}. \tag{8.8}$$

Thus

$$z_i - \bar{z} = w^\top(x_i - \bar{x}). \tag{8.9}$$

The sample variance of the projected data is

$$\mathrm{Var}(w^\top X) = \frac{1}{n}\sum_{i=1}^{n}(z_i - \bar{z})^2 = \frac{1}{n}\sum_{i=1}^{n}\left(w^\top(x_i - \bar{x})\right)^2. \tag{8.10}$$

Using the identity

$$(w^\top a)^2 = w^\top(aa^\top)w,$$

we obtain

$$\mathrm{Var}(w^\top X) = w^\top\left(\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(x_i - \bar{x})^\top\right)w = w^\top \Sigma w. \tag{8.11}$$

♡

Let u_1 be the unit projection vector on $\mathbb{R}^d$ to project each x_i onto a 1-D space, then PCA is equivalent to solve the below problem

$$\underset{\|u_1\|=1}{\operatorname{argmax}} \; u_1^\top \Sigma u_1. \tag{8.12}$$

By introducing a Lagrange multiplier λ for the constraint $u_1^\top u_1 = 1$, we consider the Lagrangian

$$\mathcal{L}(w, \lambda) = u_1^\top \Sigma u_1 - \lambda\big(u_1^\top u_1 - 1\big). \tag{8.13}$$

Taking the derivative with respect to u_1 and setting it to zero gives

$$\frac{\partial \mathcal{L}}{\partial u_1} = 2\Sigma u_1 - 2\lambda u_1 = 0, \tag{8.14}$$

hence

$$\Sigma u_1 = \lambda u_1. \tag{8.15}$$

The derivative with respect to λ enforces the constraint

$$\frac{\partial \mathcal{L}}{\partial \lambda} = -\big(u_1^\top u_1 - 1\big) = 0 \quad \Longrightarrow \quad u_1^\top u_1 = 1. \tag{8.16}$$

Therefore, any maximizer u_1 **must be an eigenvector of** Σ, with λ equal to the corresponding eigenvalue. Since

$$u_1^\top \Sigma u_1 = \lambda \tag{8.17}$$

for any unit eigenvector u_1, the optimization problem is solved by choosing u_1 as the eigenvector associated with the **largest eigenvalue of** Σ. This u_1 is the first principal component.

After obtaining the first principal component u_1, the second principal component u_2 is defined as the direction that maximizes the projected variance subject to two constraints:

$$\underset{u_2}{\operatorname{argmax}} \; u_2^\top \Sigma u_2 \tag{8.18}$$

subject to

$$u_2^\top u_2 = 1, \qquad u_2^\top u_1 = 0. \tag{8.19}$$

The first condition enforces unit length; the second ensures orthogonality to u_1.

Introduce Lagrange multipliers λ_2 and α. Consider the Lagrangian

$$\mathcal{L}(u_2, \lambda_2, \alpha) = u_2^\top \Sigma u_2 + \lambda_2(1 - u_2^\top u_2) + \alpha\, u_2^\top u_1. \tag{8.20}$$

Taking the derivative with respect to u_2 and setting it to zero yields

$$\frac{\partial \mathcal{L}}{\partial u_2} = 2\Sigma u_2 - 2\lambda_2 u_2 + \alpha u_1 = 0. \tag{8.21}$$

Rearranging the stationarity condition gives

$$\Sigma u_2 = \lambda_2 u_2 - \frac{\alpha}{2} u_1. \tag{8.22}$$

Using the orthogonality constraint $u_1^\top u_2 = 0$ and the fact that

$$\Sigma u_1 = \lambda_1 u_1, \tag{8.23}$$

we obtain

$$u_1^\top \Sigma u_2 = \lambda_2\, u_1^\top u_2 - \frac{\alpha}{2} u_1^\top u_1 = -\frac{\alpha}{2}. \tag{8.24}$$

On the other hand,

$$u_1^\top \Sigma u_2 = (\Sigma u_1)^\top u_2 = \lambda_1 u_1^\top u_2 = 0. \tag{8.25}$$

Thus $\alpha = 0$, and the stationarity condition reduces to the eigenvalue equation

$$\Sigma u_2 = \lambda_2 u_2. \tag{8.26}$$

Therefore u_2 must be an eigenvector of Σ associated with the second largest eigenvalue.

By induction:

Theorem 8.2

The first K principal components are exactly the eigenvectors of the covariance matrix Σ corresponding to its K largest eigenvalues.

♡

Denote these eigenvectors by

$$U_{1:K} = (u_1, u_2, \cdots, u_K) \in \mathbb{R}^{d\times K}. \tag{8.27}$$

Each u_i satisfies$\Sigma u_i = \lambda_i u_i$, $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_K$.

Given the matrix of the top K principal directions $U_{1:K}$, the projection of a data point $x \in \mathbb{R}^d$ onto the K-dimensional PCA subspace is $z = U_{1:K}^\top x \in \mathbb{R}^{n\times K}$. For a dataset $X \in \mathbb{R}^{n\times d}$, the projected data matrix is

$$XU_{1:K} \in \mathbb{R}^{n\times K}. \tag{8.28}$$

This reduces the dimensionality from d to K while retaining the directions of maximal variance.

Definition 8.2 (Centered Data Matrix)

Let the centered data matrix be

$$\hat{X} = \begin{pmatrix} x_1^\top - \bar{x}^\top \\ x_2^\top - \bar{x}^\top \\ \vdots \\ x_n^\top - \bar{x}^\top \end{pmatrix} \in \mathbb{R}^{n\times d}. \tag{8.29}$$

Each row of $\hat{X}$ is a mean-subtracted data point.

♣

The empirical covariance matrix can be written compactly as

$$\Sigma = \frac{1}{n}\hat{X}^\top \hat{X}. \tag{8.30}$$

Theorem 8.3 (Eigen-Decomposition of the Covariance)

Because Σ is real symmetric, it admits an orthogonal eigendecomposition:

$$\Sigma = U\Lambda U^\top, \tag{8.31}$$

where U is an orthogonal matrix whose columns are eigenvectors and Λ is a diagonal matrix containing the eigenvalues of Σ.

♡

In practice, numerical libraries such as `numpy.linalg.eig` or `numpy.linalg.eigh` compute the eigen-decomposition of Σ directly. The eigenvectors $\{u_1, \ldots, u_d\}$ form the principal directions of the data.

8.1.2 Singular Value Decomposition (SVD)

Consider the singular value decomposition of the centered data matrix:

$$\hat{X} = U_X S V^\top. \tag{8.32}$$

$$\hat{X}_{\mathbb{R}^{n\times d}} = U_{X\,\mathbb{R}^{n\times r}} \begin{pmatrix} \sigma_1 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & & & & \vdots \\ 0 & \cdots & 0 & \sigma_n & \cdots & 0 \end{pmatrix}_{\mathbb{R}^{r\times r}} V^\top_{\mathbb{R}^{r\times d}}. \tag{8.33}$$

Then

$$\hat{X}^\top \hat{X} = VS^2V^\top, \tag{8.34}$$

which implies that

$$\Sigma = \frac{1}{n} V S^2 V^\top. \tag{8.35}$$

Thus **the right singular vectors of $\hat{X}$ are exactly the eigenvectors of Σ**, and the eigenvalues of Σ are the squared singular values of $\hat{X}$ scaled by $\frac{1}{n}$.

Remark

This relationship provides an alternative way to compute principal components: the top K principal directions are simply the first K right singular vectors of $\hat{X}$. This method is numerically more stable, especially when $n \neq d$ or when d is large.

Note

In practice, computing a full SVD of a $d \times d$ matrix costs $O(d^3)$ time, which is prohibitive for high-dimensional data. Modern numerical linear algebra therefore relies on approximate *SVD methods that dramatically reduce the computational burden while preserving the leading singular components.*

A common approach is the truncated SVD*: instead of computing all singular values, we only approximate the top K components. Techniques such as randomized sketching, subspace iteration, and the Nyström method project the data onto a low-dimensional subspace in which SVD becomes cheap. These methods achieve a cost of roughly*

$$O(ndK) \quad \text{or} \quad O(d^2K),$$

depending on the algorithm, reducing the complexity significantly when $K \ll d$.

This is why real-world PCA implementations (e.g., `sklearn`, `numpy.linalg.svd`*, and randomized PCA algorithms) compute only the leading singular vectors rather than performing a full decomposition.*

8.1.3 t-distributed Stochastic Neighbor Embedding (t-SNE)*

t-SNE is a nonlinear dimensionality reduction method designed for visualization of high-dimensional datasets. It constructs two probability distributions:

- one on the pairwise similarities in the high-dimensional space,
- one on the pairwise similarities in the low-dimensional embedding,

and finds an embedding that makes these two distributions as close as possible.

Definition 8.3 (High-Dimensional Similarity)

For datapoints x_i, x_j in the original space, define a conditional probability

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)}. \tag{8.36}$$

The bandwidth σ_i is chosen such that the perplexity *of the distribution matches a user-specified value. The symmetric joint probability is*

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}. \tag{8.37}$$

♣

Note

It simply applies an RBF kernel to these data points.

Definition 8.4 (Low-Dimensional Similarity)

For embedding points $y_i, y_j \in \mathbb{R}^2$, t-SNE uses a Student-t distribution with one degree of freedom:

$$q_{ij} = \frac{\left(1 + \|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq \ell} \left(1 + \|y_k - y_\ell\|^2\right)^{-1}}. \tag{8.38}$$

The heavy-tailed distribution alleviates the "crowding problem." ♣

Theorem 8.4 (t-SNE Objective)

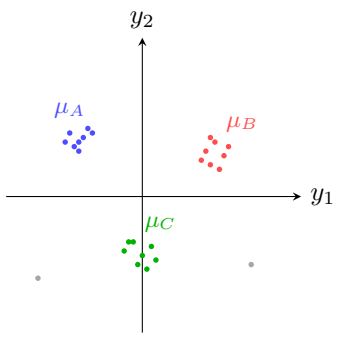
t-SNE seeks an embedding that minimizes the Kullback–Leibler divergence between the high-dimensional and low-dimensional similarity distributions:

$$\mathcal{L}(Y) = \mathrm{KL}(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}. \tag{8.39}$$

Gradient descent on $\mathcal{L}$ yields the final visualization. ♡

t-SNE preserves local neighborhood structure. Pairs with large p_{ij} are forced to satisfy large q_{ij}, keeping nearby points close. The heavy-tailed t-distribution allows distant points to be modeled far apart, preventing clusters from collapsing together.

t-SNE is excellent for visualization but not for general-purpose embedding: its geometry is not globally meaningful, and repeated runs may differ due to initialization and stochasticity.



8.2 Clustering

8.2.1 K-means Clustering Algorithm

Clustering is a fundamental task in unsupervised learning. Given a dataset $D = \{x_1, x_2, \ldots, x_n\} \subset \mathbb{R}^d$, our goal is to divide these points into K meaningful groups. Here K is a user-chosen hyperparameter, reflecting how many clusters we expect to find.

To formalize this idea, assume that each cluster $k \in [K]$ is represented by a **center** $m_k \in \mathbb{R}^d$, and each data point chooses exactly one cluster. We denote the assignment using an indicator variable $r_{ik} \in \{0, 1\}$:

$$r_{ik} = 1 \quad \Longleftrightarrow \quad x_i \text{ belongs to cluster } k. \tag{8.40}$$

Because every point must join one and only one cluster, we enforce

$$\sum_{k=1}^{K} r_{ik} = 1, \qquad \forall i \in [n]. \tag{8.41}$$

The intuitive goal of K-means is simple: each point should be close to the center of the cluster it joins. This leads to

the objective function

Theorem 8.5

$$L = \sum_{i=1}^{n} \sum_{k=1}^{K} r_{ik} \, \|x_i - m_k\|^2, \tag{8.42}$$

which measures the total within-cluster variance.

At first glance, minimizing L over both the assignments r and the centers m seems challenging, because the problem is non-convex and the discrete variables r_{ik} make the search space combinatorial. However, K-means becomes surprisingly tractable once we notice a key structural property of its objective. Although optimizing both the assignments r and the centers m jointly is difficult, each subproblem becomes very simple when the other variable is held fixed:

- **Fixing the centers $\{m_k\}$:** assigning each point x_i reduces to choosing the closest center.
- **Fixing the assignments $\{r_{ik}\}$:** updating each center becomes a simple averaging step, since the optimal m_k is just the mean of the points assigned to cluster k.

This observation directly motivates the alternating minimization strategy used by the K-means algorithm: we first update the assignments, then update the centers, and repeat the process until convergence. The objective value decreases at every iteration, and although the algorithm is not guaranteed to reach the global optimum, it converges to a stable **local minimum** in practice.

Assignment step (fix the centers). For each datapoint x_i, we assign it to the nearest cluster center:

$$r_{ik} = \begin{cases} 1, & k = \operatorname*{argmin}_{j \in [K]} \|x_i - m_j\|^2, \\ 0, & \text{otherwise.} \end{cases} \tag{8.43}$$

Update step (fix the assignments). To update m_k, we minimize the partial objective

$$\sum_{i=1}^{n} r_{ik} \|x_i - m_k\|^2 \tag{8.44}$$

with respect to m_k. Taking the derivative and setting it to zero,

$$\frac{\partial}{\partial m_k} \sum_{i=1}^{n} r_{ik} \|x_i - m_k\|^2 = -2 \sum_{i=1}^{n} r_{ik} (x_i - m_k) = 0, \tag{8.45}$$

which gives the closed-form mean update:

$$m_k = \frac{\sum_{i=1}^{n} r_{ik} x_i}{\sum_{i=1}^{n} r_{ik}}. \tag{8.46}$$

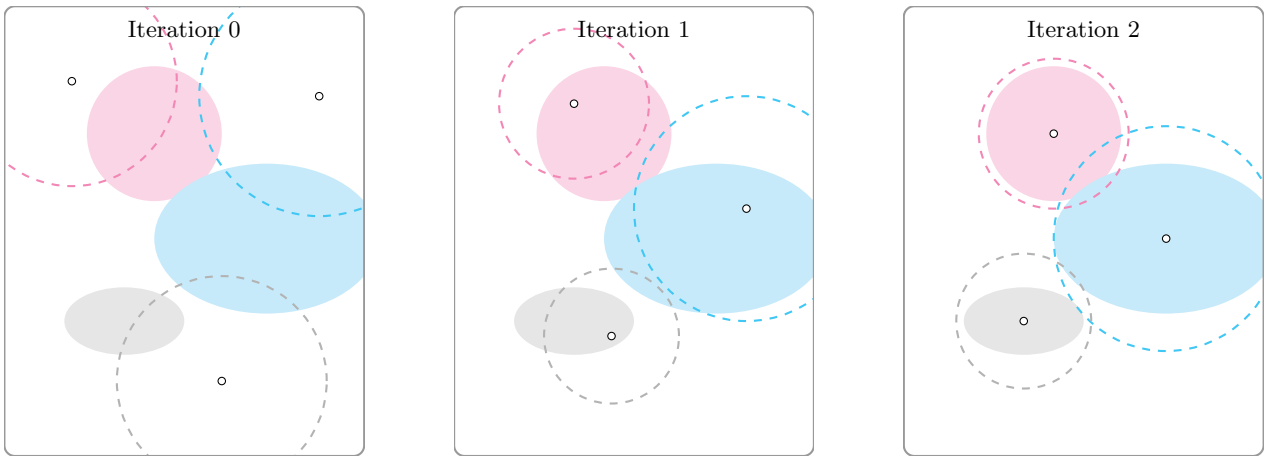
Remark

At each iteration, K-means performs a greedy improvement step: the assignment update minimizes the objective given the centers, and the center update minimizes the objective given the assignments. Hence the objective value is non-increasing and bounded below, which guarantees convergence.

Remark

However, precisely because each step is greedy and only optimizes a partial variable, K-means may converge to a local *minimum rather than the global optimum. Different initializations can therefore lead to different final solutions.*

Together, these two steps define the classical K-means iteration, which alternates between assigning points to their closest centers and recomputing each center as the average of its assigned datapoints.



Note

At each iteration, the K-means algorithm computes the centroids *of the current approximate clusters and then uses these centroids as the new cluster centers. In other words, K-means repeatedly replaces each center with the center of mass of the points currently assigned to it.*

8.2.2 Mixture of Gaussians (MoG)

K-means assumes that points belonging to each cluster are concentrated around a center, but it does not model the *shape* or *spread* of each cluster. A more expressive model is the *Mixture of Gaussians*, which assumes that points assigned to the same cluster are generated from a Gaussian distribution with its own mean and covariance.

Formally, for cluster $k \in [K]$, assume $(x \mid z_k = 1) \sim \mathcal{N}(\mu_k, \Sigma_k)$. Here $z = (z_1, \ldots, z_K)$ is a latent indicator vector with

$$z_k \in \{0, 1\}, \qquad \sum_{k=1}^{K} z_k = 1. \tag{8.47}$$

Thus $z_k = 1$ indicates that x is generated from cluster k.

The prior distribution over latent clusters is modeled using a categorical distribution:

$$P(z_k = 1) = \pi_k, \qquad 0 \le \pi_k \le 1, \qquad \sum_{k=1}^{K} \pi_k = 1. \tag{8.48}$$

Given the latent variable z, the conditional likelihood is

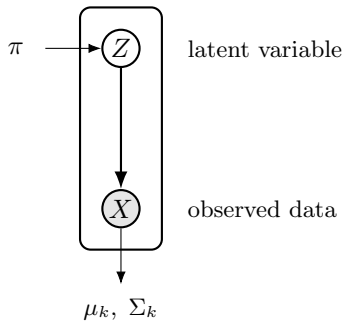
$$P(x \mid z_k = 1) = \mathcal{N}(x \mid \mu_k, \Sigma_k). \tag{8.49}$$

Since MoG is a *generative model*, the marginal density of x is obtained by summing over all latent clusters:

$$P(x) = \sum_{k=1}^{K} P(z_k = 1)\, P(x \mid z_k = 1) = \sum_{k=1}^{K} \pi_k\, \mathcal{N}(x \mid \mu_k, \Sigma_k). \tag{8.50}$$

Note

MoG can be viewed as a soft, probabilistic extension of K-means: instead of forcing each point into exactly one cluster, it allows fractional responsibilities and models not only the center but also the spread and orientation of each cluster.



8.3 Latent Variable Models and the EM Algorithm

MoG is a *latent-variable* model: for each datapoint x_i there is an unobserved cluster indicator. We will use the lighter integer notation

$$G_i \in \{1, 2, \ldots, K\}, \tag{8.51}$$

where $G_i = k$ means x_i is generated from component k. (This is equivalent to one-hot encoding used elsewhere.)

The marginal likelihood for each datapoint is

$$p(x_i; \theta) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(x_i \mid \mu_k, \Sigma_k), \tag{8.52}$$

thus the MLE objective becomes

$$\underset{\theta}{\operatorname{argmax}} \sum_{i=1}^{n} \log\left(\sum_{k=1}^{K} \pi_k \, \mathcal{N}(x_i \mid \mu_k, \Sigma_k)\right), \tag{8.53}$$

where $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}$.

Remark

The difficulty is the ***log of a sum****, which couples all components. Moreover,* $\sum_k \pi_k = 1$ *and each* Σ_k *must be positive definite, making naive unconstrained gradient descent non-trivial.*

To derive EM in a general and reusable way, we introduce KL divergence.

Definition 8.5 (Kullback–Leibler divergence)

For distributions Q, P *on the same variable* Z*,*

$$\mathrm{KL}(Q \| P) := \mathbb{E}_{Z \sim Q}\left[\log \frac{Q(Z)}{P(Z)}\right]. \tag{8.54}$$

♣

Proposition 8.1

$\mathrm{KL}(Q \| P) \geq 0$*, and* $\mathrm{KL}(Q \| P) = 0$ *iff* $Q = P$ *(a.s.).*

♡

Theorem 8.6 (ELBO decomposition)

For any $q(z)$ *and any* θ*,*

$$\log p(x; \theta) = \underbrace{\mathbb{E}_{z \sim q}\left[\log \frac{p(x, z; \theta)}{q(z)}\right]}_{\mathcal{L}(q,\theta)\ \ (ELBO)} + \underbrace{\mathrm{KL}(q(z) \,\|\, p(z \mid x; \theta))}_{\geq 0}. \tag{8.55}$$

Hence $\mathcal{L}(q, \theta) \leq \log p(x; \theta)$.

Remark

ELBO is commonly read as elbow in talks. The bound is tight when $q(z) = p(z \mid x; \theta)$.

EM alternates between optimizing the variational distribution q and the model parameters θ.

E-step (fix θ). Given $\theta^{(t)}$, maximize ELBO in q:

$$q^{(t+1)}(z) = p(z \mid x; \theta^{(t)}), \tag{8.56}$$

which drives the KL gap in (8.55) to 0 at $\theta^{(t)}$.

M-step (fix q). With $q^{(t+1)}$ fixed, update parameters by maximizing ELBO:

$$\theta^{(t+1)} \in \operatorname*{argmax}_{\theta} \ \mathcal{L}\left(q^{(t+1)}, \theta\right). \tag{8.57}$$

Because the $-\mathbb{E}_{q^{(t+1)}}[\log q^{(t+1)}]$ term does not depend on θ, this is equivalent to maximizing the expected complete-data log-likelihood

$$Q(\theta \mid \theta^{(t)}) := \mathbb{E}_{z \sim p(z \mid x; \theta^{(t)})}[\log p(x, z; \theta)]. \tag{8.58}$$

Remark

Repeating E-step → M-step yields a monotone (non-decreasing) data log-likelihood sequence, and EM converges to a stationary point (not necessarily the global optimum).

Proposition 8.2 (Monotonic improvement)

Each EM iteration does not decrease the data log-likelihood:

$$\log p(x; \theta^{(t+1)}) \geq \log p(x; \theta^{(t)}).$$

Proof

In the E-step, we choose $q^{(t+1)}(z) = p(z \mid x; \theta^{(t)})$, *hence the KL gap in* (8.55) *becomes* 0 *at* $\theta^{(t)}$ *and*

$$\mathcal{L}(q^{(t+1)}, \theta^{(t)}) = \log p(x; \theta^{(t)}).$$

In the M-step, we maximize ELBO in θ, *so*

$$\mathcal{L}(q^{(t+1)}, \theta^{(t+1)}) \geq \mathcal{L}(q^{(t+1)}, \theta^{(t)}) = \log p(x; \theta^{(t)}).$$

Since ELBO is always a lower bound on $\log p(x; \theta)$, *we have*

$$\log p(x; \theta^{(t+1)}) \geq \mathcal{L}(q^{(t+1)}, \theta^{(t+1)}),$$

which together proves the claim. ■

For MoG, the complete-data likelihood for one datapoint is

$$p(x_i, G_i = k; \theta) = \pi_k \, \mathcal{N}(x_i \mid \mu_k, \Sigma_k). \tag{8.59}$$

E-step. Compute posterior responsibilities

$$\gamma_{ik} := p(G_i = k \mid x_i; \theta^{(t)}) = \frac{\pi_k^{(t)} \, \mathcal{N}(x_i \mid \mu_k^{(t)}, \Sigma_k^{(t)})}{\sum_{j=1}^{K} \pi_j^{(t)} \, \mathcal{N}(x_i \mid \mu_j^{(t)}, \Sigma_j^{(t)})}. \tag{8.60}$$

M-step. Let $N_k := \sum_{i=1}^{n} \gamma_{ik}$. Then the maximizer has closed-form updates:

$$\pi_k = \frac{N_k}{n}, \tag{8.61}$$

$$\mu_k = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} x_i, \tag{8.62}$$

$$\Sigma_k = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top. \tag{8.63}$$

Remark

*These are **soft-assignment weighted** empirical mean and covariance. When responsibilities collapse to hard assignments, MoG-EM reduces to the familiar K-means updates.*

Assume an isotropic shared covariance $\Sigma_k = \sigma^2 I$ for all k. Then

$$\gamma_{ik} \propto \pi_k \exp\left(-\frac{\|x_i - \mu_k\|^2}{2\sigma^2}\right). \tag{8.64}$$

As $\sigma \to 0$, the soft assignment concentrates on the closest mean:

$$\gamma_{ik} \to \begin{cases} 1, & k = \operatorname{argmin}_j \|x_i - \mu_j\|^2, \\ 0, & \text{otherwise}. \end{cases} \tag{8.65}$$

This recovers the hard assignment step of K-means, and with these hard assignments the M-step mean update reduces to the K-means centroid update.

Chapter 9 Variational Autoencoder (VAE)

Introduction

- *Deep generative models*
- *Autoencoder (AE)*
- *Latent-variable generative modeling*
- *Variational inference and ELBO*
- *VAE objective*
- *Reparameterization trick*

In the previous chapter, we have already seen latent-variable models such as Mixture of Gaussians (MoG), whose generative story is explicit and simple. However, realistic data distributions (e.g. natural images) are far more complex:

- Images live in an extremely high-dimensional space (e.g. a 256×256 RGB image is a vector in $\mathbb{R}^{256 \cdot 256 \cdot 3}$);
- The distribution contains rich multi-scale structures (low-frequency layout, high-frequency texture, object semantics, etc.);
- A mixture of a small number of simple components (e.g. Gaussians) is usually not expressive enough.

This motivates *deep generative models*, which use neural networks to build highly expressive families of distributions.

Even if a distribution admits a closed-form density, sampling can still be hard. In practice, most sampling procedures ultimately rely on generating pseudo-random numbers (often starting from a nearly-uniform source on $[0, 1]$) and applying transformations. For complex high-dimensional distributions, sampling often requires Monte Carlo / MCMC methods, which may have large variance and long burn-in.

To alleviate the difficulty of directly sampling from complex high-dimensional distributions, we seek to learn a lower-dimensional representation in which the data distribution becomes easier to model and manipulate. This motivates the use of autoencoders.

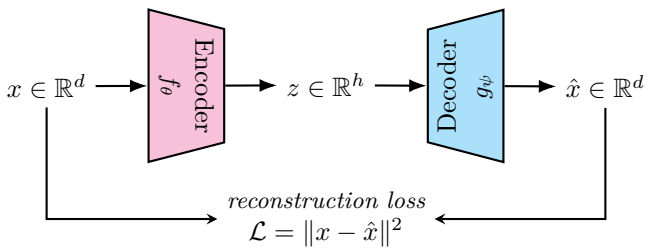
9.1 Autoencoders (AE)

An *autoencoder* maps a high-dimensional data point $x \in \mathbb{R}^d$ into a low-dimensional representation $z \in \mathbb{R}^h$ and reconstructs x from z, typically via an encoder–decoder pair:

$$z = f_\theta(x), \qquad \hat{x} = g_\psi(z). \tag{9.1}$$

Here $f_\theta : \mathbb{R}^d \to \mathbb{R}^h$ is called the **encoder** and $g_\psi : \mathbb{R}^h \to \mathbb{R}^d$ the **decoder**. The training objective minimizes a reconstruction error, e.g. mean squared error:

$$\operatorname*{argmin}_{\theta,\psi} \sum_{i=1}^{n} \|x_i - \hat{x}_i\|^2, \qquad \hat{x}_i = g_\psi(f_\theta(x_i)). \tag{9.2}$$



The learned representation z can be used for downstream tasks such as dimensionality reduction, clustering, or as a feature input to a classifier. **However, an AE is *not* a generative model**: it provides no principled distribution on z to sample from, so generating a new realistic x is not guaranteed.

9.2 Latent-Variable Generative Modeling

We now move from representation learning to generative modeling. Let $\mathcal{D} = \{x_i\}_{i=1}^n$ be an unlabeled dataset (e.g. images). Our goal is to learn a model that can generate *new* data similar to the training distribution. That is, we want to learn(or equivalently, approximate) a model $p_\theta(x)$ that can generate new data similar to the training distribution.

The key idea is to introduce a latent variable z with a simple prior $p(z)$, and generate x via a conditional distribution $p_\theta(x \mid z)$ whose parameters are produced by a neural network.

Definition 9.1 (Latent-variable generative model)

Choose a prior $p(z)$ (easy to sample), and a conditional likelihood $p_\theta(x \mid z)$ (expressive). The marginal model is

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, \mathrm{d}z. \tag{9.3}$$

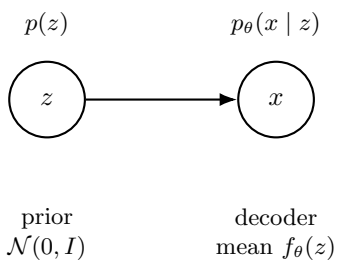
♣

Note

That is, $p_\theta(x)$ is the integral of $p_\theta(x \mid z)$ with measure $p(z)$.

A canonical choice is the standard Gaussian prior $p(z) = \mathcal{N}(0, I)$, and an isotropic Gaussian likelihood whose mean is a neural network:

$$p_\theta(x \mid z) = \mathcal{N}\big(x \mid f_\theta(z),\, \sigma^2 I\big). \tag{9.4}$$



Note

It is absolutely not necessary to use a Gaussian likelihood. Any distribution that is not ill-conditioned is suitable. Here we use a Gaussian likelihood only for simplicity.

Remark

If f_θ were linear, then x would remain Gaussian. A nonlinear and expressive f_θ enables highly non-Gaussian and complex distributions over x.

The maximum-likelihood objective is

$$\operatorname*{argmax}_\theta \sum_{i=1}^n \log p_\theta(x_i). \tag{9.5}$$

But expanding (9.3) gives

$$\log p_\theta(x) = \log \int p_\theta(x \mid z)\, p(z)\, \mathrm{d}z, \tag{9.6}$$

where the integral is typically intractable when $p_\theta(x \mid z)$ is defined by a deep neural network. One may try the Monte

Carlo approximation

$$p_\theta(x) \approx \frac{1}{K}\sum_{k=1}^{K} p_\theta\big(x \mid z^{(k)}\big), \qquad z^{(k)} \sim p(z), \tag{9.7}$$

i.e., estimate the marginal likelihood by sampling latent codes from the prior and averaging the corresponding conditional likelihoods. In principle this is valid, but in high dimensions most prior samples $z^{(k)}$ will fall in regions that are unlikely to explain a specific datapoint x. Consequently, the terms $p_\theta(x \mid z^{(k)})$ are typically extremely small for most samples, and the average is dominated by rare lucky valid samples. This leads to very large variance, so obtaining a reliable estimate would require a large K for each datapoint x_i, which quickly becomes computationally prohibitive.

Since this is a latent-variable model, one may also consider the EM algorithm. Recall that EM alternates between (i) inferring the latent variable distribution given the current parameters (E-step) and (ii) maximizing the expected complete-data log-likelihood under that inferred distribution (M-step). Concretely, the E-step requires the posterior

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z)p(z)}{p_\theta(x)} \propto p_\theta(x \mid z)p(z), \tag{9.8}$$

because the M-step depends on expectations taken with respect to $p_\theta(z \mid x)$. However, the normalization constant is exactly the marginal likelihood $p_\theta(x) = \int p_\theta(x \mid z)p(z)\,dz$, which is the same intractable integral as in (9.6). In classical mixture models (e.g., MoG), the latent variable is discrete, so the corresponding normalization is a finite sum and the posterior can be computed exactly; in VAE, z is continuous and high-dimensional, so the required integrals are generally intractable.

The essential bottleneck, therefore, is that we cannot evaluate or sample from the true posterior $p_\theta(z \mid x)$ efficiently. Variational inference addresses this by replacing the intractable posterior with a tractable approximation

$$q_\phi(z \mid x) \approx p_\theta(z \mid x), \tag{9.9}$$

where q_ϕ is chosen from a convenient family (e.g., Gaussians with diagonal covariance) and its parameters are produced by another neural network, called the *encoder*. Intuitively, instead of performing an exact E-step, we learn an inference model that takes a datapoint x as input and outputs a distribution over plausible latent codes z that could have generated x under the decoder.

Remark

Actually, sampling a random number itself is not easy. Even producing a single sample that is close to uniform on the interval $[0, 1]$ already relies on carefully designed pseudo-random number generators and deterministic transformations. Sampling from a more complex distribution, especially one defined on a high-dimensional space or on a complicated manifold, is therefore substantially more challenging.

Theorem 9.1 (ELBO decomposition)

For any distribution $q_\phi(z \mid x)$,

$$\log p_\theta(x) = \underbrace{\mathbb{E}_{z\sim q_\phi(\cdot\mid x)}\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}\big(q_\phi(z \mid x) \,\big\|\, p(z)\big)}_{\mathcal{L}_{\mathrm{ELBO}}(x;\theta,\phi)} + \underbrace{\mathrm{KL}\big(q_\phi(z \mid x) \,\big\|\, p_\theta(z \mid x)\big)}_{\geq 0}. \tag{9.10}$$

Hence $\mathcal{L}_{\mathrm{ELBO}}(x;\theta,\phi) \leq \log p_\theta(x)$.

♡

Remark

In (9.10), the two KL divergences are different and should not be confused. The KL term inside the ELBO compares $q_\phi(z \mid x)$ with the prior $p(z)$, while the remaining KL term measures the gap between $q_\phi(z \mid x)$ and the true posterior $p_\theta(z \mid x)$.

Remark

The KL term $\mathrm{KL}(q_\phi(z \mid x) \| p(z))$ appears inside *ELBO and acts as a regularizer toward the prior. The KL term* $\mathrm{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x))$ *is the* gap *between ELBO and the true log-evidence* $\log p_\theta(x)$.

During optimization, we optimize $\log p_\theta(x)$ indirectly by maximizing a tractable lower bound (the ELBO). (Because the gap is intractable)

Although the gap term $\mathrm{KL}\big(q_\phi(z \mid x) \,\big\|\, p_\theta(z \mid x)\big)$ cannot be optimized explicitly, both q_ϕ and p_θ depend on the parameters (ϕ, θ) and are updated jointly when maximizing the ELBO. As a result, the gap is implicitly affected during training and does not remain fixed or uncontrolled.

The expectation in $\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]$ still involves an integral, but the sampling distribution is now data-dependent. This is a major improvement: the most costly part in practice is drawing informative samples, and sampling from the prior is inefficient. By replacing prior sampling with sampling conditioned on the observed data, the computation is focused on latent regions that actually explain x. Instead of drawing samples from the broad prior $p(z) = \mathcal{N}(0, I)$, we sample from the posterior $q_\phi(z \mid x)$, which typically concentrates its mass in a narrow region relevant to the given data point. As a result, Monte Carlo estimates of the expectation can have significantly lower variance.

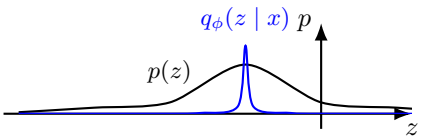


Figure 9.1: A narrow $q_\phi(z \mid x)$ concentrates its mass in a relatively narrow region, while a broad $p(z)$ has a wide distribution.

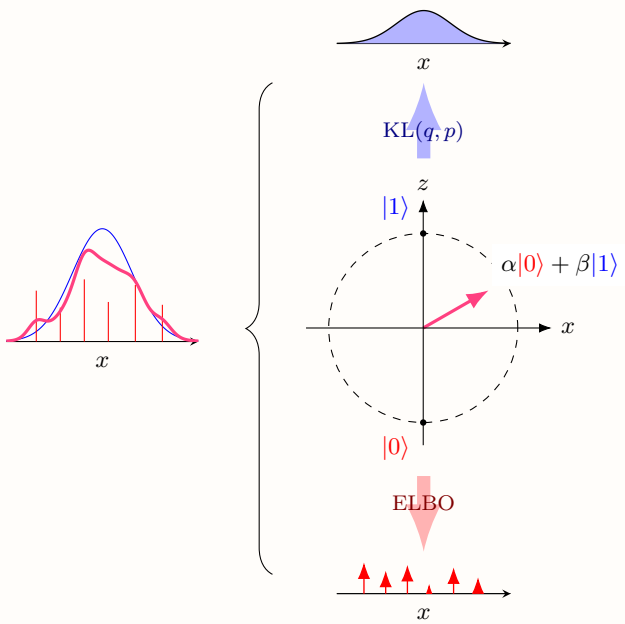
Note

From the perspective of information theory, the condition x brings additional information about z, which increase the success probability of sampling a valid, meaningful z.

Note

The prior $p(z)$ *does not participate in training by being sampled in the forward optimization path. Instead, it enters training through the KL regularization term, which constrains and aligns the overall shape of* $q_\phi(z \mid x)$. *Meanwhile, by sampling* $z \sim q_\phi(z \mid x)$ *and reconstructing x, the model is forced during self-optimization to maintain stable decoding within a neighborhood around* $\mu_\phi(x)$, *thereby endowing the latent representation with semantic consistency and smoothness.*

The weight of the KL term must be balanced. If the KL term is too small, $q_\phi(z \mid x)$ *tends toward deterministic encoding* ($\sigma \to 0$), *and the latent space lacks a well-structured global organization, leading to poor interpolation and unreliable sampling from the prior. If the KL term is too large,* $q_\phi(z \mid x)$ *is pushed close to* $p(z)$, *and the latent variables carry insufficient information about x, making it difficult for the decoder to recover meaningful samples. These two cases correspond respectively to an irregular latent space and information collapse.*



Training VAE maximizes the ELBO over the dataset:

$$\underset{\theta,\phi}{\operatorname{argmax}} \sum_{i=1}^{n} \mathcal{L}_{\mathrm{ELBO}}(x_i;\theta,\phi). \tag{9.11}$$

Assume the decoder likelihood (9.4). Then, up to an additive constant independent of θ,

$$\log p_\theta(x \mid z) = -\frac{1}{2\sigma^2}\left\|x - f_\theta(z)\right\|^2 + C. \tag{9.12}$$

Plugging this into (9.10), maximizing ELBO is equivalent to minimizing (ignoring constants)

$$\underset{\theta,\phi}{\operatorname{argmin}} \sum_{i=1}^{n} \left[\mathbb{E}_{z\sim q_\phi(\cdot\mid x_i)}\left\|x_i - f_\theta(z)\right\|^2 + \beta\,\mathrm{KL}\big(q_\phi(z \mid x_i)\,\big\|\,p(z)\big)\right], \tag{9.13}$$

where $\beta = 2\sigma^2$ (or we may treat β as a tunable weight, leading to β-VAE).

Remark

If we remove the KL regularization term, (9.13) becomes a standard autoencoder objective: it reconstructs well but does not enforce a tractable sampling distribution in latent space, hence is not a principled generative model.

The KL regularizer encourages $q_\phi(z \mid x)$ to be close to the fixed prior $p(z) = \mathcal{N}(0, I)$. After training, to *generate* a new sample, we do not need the encoder:

$$z \sim p(z), \qquad x \sim p_\theta(x \mid z). \tag{9.14}$$

Because $p(z)$ is simple to sample and the decoder is learned, we can generate new x.

Note

In modern diffusion systems, VAE is often used as a latent autoencoder*, diffusion operates in the latent space, then the VAE decoder maps latent variables back to pixel space.*

9.3 Reparameterization Trick

To optimize (9.13) with backpropagation, we must be able to compute gradients with respect to the encoder parameters ϕ. However, if we sample directly by $z \sim q_\phi(z \mid x)$, then z is produced by a stochastic sampling operation rather than an explicit differentiable function of ϕ. In other words, small changes in ϕ change the distribution $q_\phi(z \mid x)$, but the random draw z itself does not provide a differentiable path for gradients to flow back to ϕ, which breaks end-to-end training.

A standard choice is to let the encoder output a Gaussian distribution

$$q_\phi(z \mid x) = \mathcal{N}\big(z \mid \mu_\phi(x),\, \Sigma_\phi(x)\big), \tag{9.15}$$

often with diagonal covariance

$$\Sigma_\phi(x) = \operatorname{diag}\big(\sigma_\phi^2(x)\big). \tag{9.16}$$

Sample instead from a parameter-free noise distribution:

$$\varepsilon \sim \mathcal{N}(0, I), \qquad z = \mu_\phi(x) + L_\phi(x)\,\varepsilon, \tag{9.17}$$

where $L_\phi(x) L_\phi(x)^\top = \Sigma_\phi(x)$ (for diagonal covariance, $L_\phi(x) = \operatorname{diag}(\sigma_\phi(x))$).

Proposition 9.1 (Correctness of reparameterization)

If $\varepsilon \sim \mathcal{N}(0, I)$ and $z = \mu + L\varepsilon$, then $z \sim \mathcal{N}(\mu, LL^\top)$.

Proof

The affine transformation of a Gaussian remains Gaussian. Moreover,

$$\mathbb{E}[z] = \mu + L\,\mathbb{E}[\varepsilon] = \mu, \tag{9.18}$$

and

$$\operatorname{Cov}(z) = L \operatorname{Cov}(\varepsilon)\, L^\top = LIL^\top = LL^\top. \tag{9.19}$$

The sampling step now occurs in ε, which has no learnable parameters; gradients only need to flow through differentiable operations producing $\mu_\phi(x)$ and $L_\phi(x)$.

Note

In a deterministic autoencoder, each data point x is mapped to a single latent code z, and the reconstruction loss only enforces correctness at that particular point. The decoder is therefore unconstrained in the neighborhood around z.

In contrast, a VAE represents each data point by a Gaussian distribution $q_\phi(z \mid x) = \mathcal{N}(\mu(x), \sigma^2(x))$. During training, latent variables are sampled from this distribution. As a result, the decoder is required to reconstruct x not just at a single latent code, but throughout a neighborhood around $\mu(x)$. This stochasticity induces a smooth latent space.

The KL term in (9.13) is tractable when both distributions are Gaussian.

Theorem 9.2 (Gaussian–Gaussian KL)

Let $q = \mathcal{N}(\mu, \Sigma)$ and $p = \mathcal{N}(0, I)$. Then

$$\mathrm{KL}(q \| p) = \frac{1}{2}\Big(\operatorname{tr}(\Sigma) + \mu^\top \mu - d - \log\det(\Sigma)\Big). \tag{9.20}$$

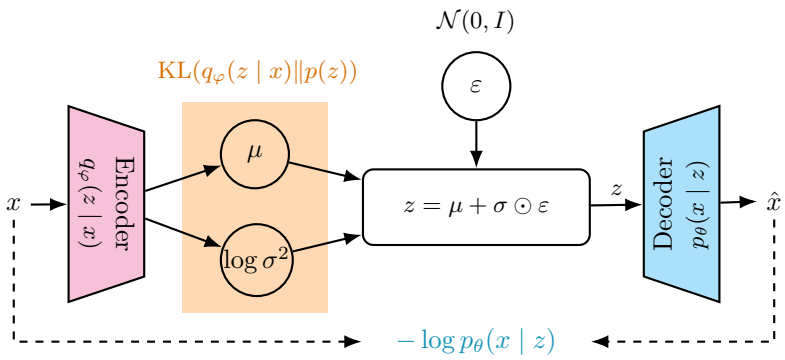


Figure 9.2: The structure of a VAE, in which orange KL loss is regularized towards the prior $p(z)$ which is trained for sampling during generation, and the cyan ELBO loss is regularized towards the decoder $p_\theta(x \mid z)$. The source of randomness is shifted from sampling z itself to sampling an auxiliary variable ε, so that z becomes a deterministic function of ε and the parameters. As a result, the gradient paths through the functions of z in both p_θ and q_ϕ are preserved.

In the diagonal case $\Sigma = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_d^2)$*, this becomes*

$$\mathrm{KL}(q \| p) = \frac{1}{2} \sum_{j=1}^{d} \left(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2 \right). \tag{9.21}$$

Remark

Therefore, VAE training typically uses:

- *a decoder network* f_θ *for* $p_\theta(x \mid z)$*;*
- *an encoder network producing two output heads:* $\mu_\phi(x)$ *and* $\log \sigma_\phi^2(x)$*;*
- *one (or a few) Monte Carlo samples per datapoint for the reconstruction expectation.*

Chapter 10 Diffusion Model

Introduction

- *Diffusion as iterative generation*
- *Markov forward noising process*
- *Reverse denoising model*
- *DDPM*

10.1 From a Physical Picture to Diffusion Models*

⊛ *This section is included out of personal interest and is not part of the course material. Readers who wish to focus solely on the course content may safely skip to the next subsection.*

10.1.1 Langevin Dynamics

Imagine a particle whose position is $x(t) \in \mathbb{R}^d$. Two effects act on it:

1. **Deterministic force (drift).** The particle feels a force pushing it toward low energy. If the energy is described by a scalar potential $U(x)$, then a standard choice of force is $-\nabla U(x)$.
2. **Random perturbation (noise).** The surrounding environment (a heat bath) keeps giving it tiny random kicks.

If there were no noise, we would have an ODE

$$\frac{\mathrm{d}x}{\mathrm{d}t} = -\nabla U(x). \tag{10.1}$$

This is a gradient descent flow: $x(t)$ moves downhill on the energy landscape $U(x)$.

To model the random kicks, we add a noise term. The clean mathematical model is an SDE (stochastic differential equation), called *overdamped Langevin dynamics*:

Theorem 10.1 (overdamped Langevin dynamics)

$$\mathrm{d}x_t = -\nabla U(x_t)\,\mathrm{d}t + \sqrt{2T}\,\mathrm{d}W_t, \tag{10.2}$$

where $T > 0$ is a constant controlling the noise strength (temperature), and W_t is Brownian motion. Brownian motion behaves like a cumulative sum of many tiny independent random steps, and its typical size over a short time Δt scales like $\sqrt{\Delta t}$, not Δt.

Remark

A clean way to see the $\sqrt{t}$ scaling is to start from Gaussian additivity. If $\xi_1, \ldots, \xi_n$ are i.i.d. $\mathcal{N}(0, \sigma^2)$, then

$$\sum_{k=1}^{n} \xi_k \sim \mathcal{N}(0, n\sigma^2), \tag{10.3}$$

so the typical size (standard deviation) is $\sqrt{n}\,\sigma$.
Now discretize time: $t = n\Delta t$. Brownian motion is defined to have increments with variance equal to elapsed time,

$$W_{(k+1)\Delta t} - W_{k\Delta t} \sim \mathcal{N}(0, \Delta t). \tag{10.4}$$

To realize such increments using standard Gaussians, take $\xi_k \sim \mathcal{N}(0,1)$ and set

$$\Delta W_k := \sqrt{\Delta t}\,\xi_k, \tag{10.5}$$

because then $\Delta W_k \sim \mathcal{N}(0,\Delta t)$ as required. Summing n steps gives

$$W_t \approx \sum_{k=1}^{n} \Delta W_k = \sum_{k=1}^{n} \sqrt{\Delta t}\,\xi_k \sim \mathcal{N}(0, n\Delta t) = \mathcal{N}(0,t) \quad \Rightarrow \quad \mathrm{d}W_t \propto \sqrt{\mathrm{d}t} \tag{10.6}$$

Instead of tracking a single random trajectory, we can track the *probability density* $p_t(x)$ of the random variable x_t. The Langevin dynamics (10.2) induces a PDE for p_t, called the *Fokker–Planck equation*:

$$\partial_t p_t(x) = \nabla \cdot \big(p_t(x)\,\nabla U(x)\big) + T\,\Delta p_t(x). \tag{10.7}$$

We can interpret the two terms as follows.

- The term $\nabla \cdot (p_t \nabla U)$ is a *transport* term: probability mass flows along the vector field $-\nabla U$.
- The term $T\Delta p_t$ is a *diffusion* term: it spreads out the density, like heat diffusion in PDE.

A classic fact is that (10.7) has a stationary solution of the Boltzmann form

$$p_\infty(x) \propto \exp\big(-U(x)/T\big). \tag{10.8}$$

This matches physical intuition, that the higher-energy states (large U) share less probability.

10.1.2 Score Matching

Rewrite (10.8) as $U(x) = -T \log p_\infty(x) + \text{const}$. Taking gradients gives

$$-\nabla U(x) = T\,\nabla \log p_\infty(x). \tag{10.9}$$

The vector field $\nabla \log p(x)$ is called the *score* of the density.

Definition 10.1 (Score)

The score function $\psi(x)$ of a density $p(x)$ is the gradient of the log-density:

$$\psi(x) := \nabla \log p(x). \tag{10.10}$$

♣

Notice that $-\nabla U(x)$ always refers to a reservoir force field, we can say that matching a score function, i.e. matching the gradient of the log-density, is equivalent to matching a kind of force (zero-curl vector) field.

This is a key conceptual link to diffusion models: if we can estimate $\nabla \log p_t(x)$, we can use it to guide a noisy dynamics back toward a desired distribution. Suppose the data distribution is $p_{\text{data}}(x)$ on $\mathbb{R}^d$. We build a model $p_\theta(x)$ (possibly *unnormalized*), and we want to learn its score $\psi_\theta(x) := \nabla_x \log p_\theta(x)$. A naive idea is to minimize the squared distance between score fields:

Theorem 10.2 (Explicit Score Matching)

$$\mathcal{L}_{\text{ESM}}(\theta) := \frac{1}{2}\mathbb{E}_{x\sim p_{\text{data}}}\Big[\|\psi_\theta(x) - \psi_{\text{data}}(x)\|^2\Big], \qquad s_{\text{data}}(x) := \nabla_x \log p_{\text{data}}(x). \tag{10.11}$$

♡

This objective is conceptually perfect: we are literally matching forces. However, it hides an impossible term: we do not know $\psi_{\text{data}}(x)$. The miracle of *explicit score matching* (ESM) is that the unknown score can be eliminated by integration by parts, leaving an objective that only depends on ψ_θ and its divergence.

To solve the problem the ESM holds, we further expand (10.11):

$$\mathcal{L}_{\text{ESM}}(\theta) = \frac{1}{2}\mathbb{E}_{p_{\text{data}}}\big[\|\psi_\theta(x)\|^2\big] + \frac{1}{2}\mathbb{E}_{p_{\text{data}}}\big[\|\psi_{\text{data}}(x)\|^2\big] - \mathbb{E}_{p_{\text{data}}}\big[\psi_\theta(x)^\top \psi_{\text{data}}(x)\big]. \tag{10.12}$$

The middle term does not depend on θ, so we only need to handle the cross term $\mathbb{E}_{p_{\text{data}}}[\psi_\theta^\top \psi_{\text{data}}]$. Write $p = p_{\text{data}}$ and

$\psi_{\text{data}} = \nabla \log p$. Then

$$\mathbb{E}_p\left[\psi_\theta(x)^\top \psi_{\text{data}}(x)\right] = \int \psi_\theta(x)^\top \nabla \log p(x)\, p(x)\, dx \tag{10.13}$$

$$= \int \psi_\theta(x)^\top \nabla p(x)\, dx. \tag{10.14}$$

Integration by parts with boundary conditions gives

$$\int \psi_\theta(x)^\top \nabla p(x)\, dx = -\int p(x)\, \nabla \cdot \psi_\theta(x)\, dx = -\mathbb{E}_p\left[\nabla \cdot \psi_\theta(x)\right], \tag{10.15}$$

where $\nabla \cdot \psi_\theta = \sum_{i=1}^d \partial_{x_i} \psi_{\theta,i}$ is the divergence of the score function.

Plugging (10.15) back, we obtain an equivalent objective (up to a constant):

Theorem 10.3 (Implicit Score Matching)

$$\mathcal{J}_{\text{ISM}}(\theta) := \mathbb{E}_{x\sim p_{\text{data}}}\left[\frac{1}{2}\|\psi_\theta(x)\|^2 + \nabla \cdot \psi_\theta(x)\right]. \tag{10.16}$$

♡

This is the classical score matching objective: the Implicit Score Matching (ISM). It never evaluates $p_\theta(x)$, and it does not need the normalization constant of p_θ. In other words, ISM can learn an *energy-based model* by only looking at forces.

So far, we were matching the score of the *data distribution* p_{data}. But in diffusion models, we care about scores of *noisy* distributions p_t. Surprisingly, it actually makes learning dramatically easier.

Let $q_\sigma(\tilde{x} \mid x)$ be a Gaussian corruption process:

$$\tilde{x} = x + \sigma\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I). \tag{10.17}$$

This defines a noisy marginal

$$p_\sigma(\tilde{x}) = \int q_\sigma(\tilde{x} \mid x)\, p_{\text{data}}(x)\, dx. \tag{10.18}$$

Denoising score matching (DSM) learns the score of $p_\sigma(\tilde{x})$, but it uses a training target that is *explicit* and *local*.

A crucial identity is the score of the Gaussian kernel:

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) = -\frac{\tilde{x} - x}{\sigma^2}. \tag{10.19}$$

Vincent's denoising formulation shows that minimizing

$$\mathcal{L}_{\text{DSM}}(\theta) := \frac{1}{2}\mathbb{E}_{x\sim p_{\text{data}}}\, \mathbb{E}_{\tilde{x}\sim q_\sigma(\cdot|x)}\left[\left\|\psi_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)\right\|^2\right] \tag{10.20}$$

is (up to a constant independent of θ) equivalent to score matching on the noisy marginal p_σ:

$$\mathcal{L}_{\text{DSM}}(\theta) \equiv \frac{1}{2}\mathbb{E}_{\tilde{x}\sim p_\sigma}\left[\|\psi_\theta(\tilde{x}) - \frac{\tilde{\varepsilon}}{\sigma^2}\|^2\right] + \text{const}. \tag{10.21}$$

where $\tilde{\varepsilon} := \tilde{x} - x$ is the noise injected by the corruption kernel. In other words, DSM replaces the unknown score of p_σ by a computable teacher score from the corruption kernel (10.19).

Remark

This formula is intuitive. Under Gaussian corruption $\tilde{x} = x + \sigma\varepsilon$, the kernel $q_\sigma(\tilde{x} \mid x)$ corresponds to a quadratic potential centered at x. Its score

$$F = -\nabla(U(x + \delta x) - U(x)) = -\nabla^2 U|_x \delta x \quad \Leftrightarrow \quad -\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) = -\frac{\tilde{x} - x}{\sigma^2} \tag{10.22}$$

is exactly a restoring force pointing from $\tilde{x}$ back to x. Since the noisy marginal p_σ is a local mixture of such Gaussians, its score near high-density regions is well approximated by this restoring direction.

The most extraordinary part comes.

Interpret the noise level σ as a time parameter. If we set a schedule $\sigma = \sigma(t)$ and define the forward corruption

$$x_t = x_0 + \sigma(t)\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I), \tag{10.23}$$

then the marginal distribution of x_t is exactly the noisy marginal we wrote before:

$$p_t(x) \equiv p_{\sigma(t)}(x). \tag{10.24}$$

Therefore, training a network $\psi_\theta(x, \sigma)$ on many noise levels is the same as learning the time-dependent score field $\psi_\theta(x, t) \approx \nabla_x \log p_t(x)$ along the entire forward diffusion trajectory. This is precisely the information needed to run the dynamics backward and generate samples.

10.1.3 Diffusion Models

Up to this point, we have only used a *static* picture of noise: for each noise level σ, we define a corrupted distribution p_σ and learn its score. The key insight of diffusion models is to turn this family $\{p_\sigma\}_\sigma$ into a *continuous-time evolution*.

In other words, instead of viewing σ merely as a hyperparameter, we regard it as a time variable and explicitly define a *forward stochastic dynamics* that gradually transforms data into noise. This dynamics should satisfy three basic requirements:

1. it is easy to simulate;
2. it progressively destroys structure in the data;
3. its time marginals p_t are exactly the noisy distributions whose scores we have learned via denoising score matching.

From the physical perspective, the most natural way to inject noise continuously in time is through Brownian motion. This leads us back to stochastic differential equations and Langevin dynamics.

A particularly convenient choice is the *variance-preserving* (VP) diffusion:

$$\mathrm{d}x_t = -\frac{1}{2}\beta(t)\,x_t\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}W_t, \tag{10.25}$$

where $\beta(t) \geq 0$ is a noise schedule chosen by us.

A crucial advantage of (10.25) is that it has an explicit conditional distribution. Define

$$\alpha(t) = \exp\Big(-\frac{1}{2}\int_0^t \beta(s)\,\mathrm{d}s\Big). \tag{10.26}$$

Then one can show

Theorem 10.4 (Variance-preserving diffusion)

$$x_t \mid x_0 \sim \mathcal{N}\big(\alpha(t)\,x_0,\ (1-\alpha(t)^2)\,I\big). \tag{10.27}$$

Proof

Lemma 10.1 (Itô isometry)

Let W_t be 1D Brownian motion and let $f(t)$ be an adapted process such that $\mathbb{E}\int_0^T f(t)^2\,\mathrm{d}t < \infty$. Then

$$\mathbb{E}\left[\left(\int_0^T f(t)\,\mathrm{d}W_t\right)^2\right] = \mathbb{E}\left[\int_0^T f(t)^2\,\mathrm{d}t\right]. \tag{10.28}$$

In particular, $\mathbb{E}\Big[\int_0^T f(t)\,\mathrm{d}W_t\Big] = 0$.

Define

$$\alpha(t) := \exp\Big(-\frac{1}{2}\int_0^t \beta(s)\,\mathrm{d}s\Big), \tag{10.29}$$

and consider $y_t := \alpha(t)^{-1}x_t$. *Since* $\alpha(t)$ *is deterministic:*

$$\mathrm{d}(\alpha^{-1}x_t) = \alpha^{-1}\,\mathrm{d}x_t + x_t\,\mathrm{d}(\alpha^{-1}). \tag{10.30}$$

Compute $\mathrm{d}(\alpha^{-1})$ *from* (10.29)*:*

$$\frac{\mathrm{d}}{\mathrm{d}t}\alpha(t) = -\frac{1}{2}\beta(t)\alpha(t) \quad \Rightarrow \quad \frac{\mathrm{d}}{\mathrm{d}t}\alpha(t)^{-1} = \frac{1}{2}\beta(t)\alpha(t)^{-1}, \tag{10.31}$$

That is:

$$\mathrm{d}(\alpha^{-1}) = \frac{1}{2}\beta(t)\alpha(t)^{-1}\,\mathrm{d}t. \tag{10.32}$$

Then:

$$\mathrm{d}y_t = \alpha^{-1}\Big(-\frac{1}{2}\beta(t)x_t\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}W_t\Big) + x_t\Big(\frac{1}{2}\beta(t)\alpha^{-1}\,\mathrm{d}t\Big) \tag{10.33}$$

$$= \alpha(t)^{-1}\sqrt{\beta(t)}\,\mathrm{d}W_t. \tag{10.34}$$

So the drift cancels completely. Then we integrate from 0 *to* t*:*

$$y_t = y_0 + \int_0^t \alpha(s)^{-1}\sqrt{\beta(s)}\,\mathrm{d}W_s \tag{10.35}$$

hence

$$x_t = \alpha(t)x_0 + \alpha(t)\int_0^t \alpha(s)^{-1}\sqrt{\beta(s)}\,\mathrm{d}W_s. \tag{10.36}$$

The Itô integral has zero mean given x_0*, so*

$$\mathbb{E}[x_t \mid x_0] = \alpha(t)x_0. \tag{10.37}$$

Let

$$Z_t := \alpha(t)\int_0^t \alpha(s)^{-1}\sqrt{\beta(s)}\,\mathrm{d}W_s. \tag{10.38}$$

Then Z_t *is Gaussian with mean* 0*, and by Itô isometry (coordinate-wise),*

$$\mathrm{Cov}(Z_t \mid x_0) = \alpha(t)^2\int_0^t \beta(s)\alpha(s)^{-2}\,\mathrm{d}s \cdot I. \tag{10.39}$$

Now observe a neat identity:

$$\frac{\mathrm{d}}{\mathrm{d}s}\big(\alpha(s)^{-2}\big) = \beta(s)\alpha(s)^{-2}, \tag{10.40}$$

so

$$\int_0^t \beta(s)\alpha(s)^{-2}\,\mathrm{d}s = \alpha(t)^{-2} - 1. \tag{10.41}$$

Plugging into (10.39) *gives*

$$\mathrm{Cov}(Z_t \mid x_0) = \alpha(t)^2(\alpha(t)^{-2} - 1)\,I = (1 - \alpha(t)^2)\,I. \tag{10.42}$$

From (10.36)*,* $x_t \mid x_0$ *is an affine transform of a Gaussian Itô integral, hence Gaussian with the mean and covariance computed above:*

$$x_t \mid x_0 \sim \mathcal{N}\big(\alpha(t)\,x_0,\ (1 - \alpha(t)^2)\,I\big), \tag{10.43}$$

which is exactly (10.27)*.* ■

When t is large and $\alpha(t) \approx 0$, x_t is close to a standard Gaussian.

The forward process maps complicated data distributions to simple Gaussian noise. To generate new samples, we need to run the process *backward*: starting from $x_T \sim \mathcal{N}(0, I)$, we want to move toward x_0.

Here is the key mathematical statement (we use it as a fact): if the forward dynamics is

$$\mathrm{d}x_t = f(x_t, t)\,\mathrm{d}t + g(t)\,\mathrm{d}W_t, \tag{10.44}$$

then the reverse-time dynamics has the form

$$\mathrm{d}x_t = \Big(f(x_t,t) - g(t)^2\,\nabla_x \log p_t(x_t)\Big)\mathrm{d}t + g(t)\,\mathrm{d}\bar{W}_t, \tag{10.45}$$

where p_t is the density of x_t and $\bar{W}_t$ is Brownian motion in reverse time.

The important part is the extra term $-g(t)^2\nabla_x \log p_t(x)$. It says that the reverse dynamics needs the score of the time-t density. For the VP choice (10.25), we have $f(x,t) = -\frac{1}{2}\beta(t)x$ and $g(t)^2 = \beta(t)$, so (10.45) becomes

$$\mathrm{d}x_t = \Big(-\frac{1}{2}\beta(t)\,x_t - \beta(t)\,\nabla_x \log p_t(x_t)\Big)\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\bar{W}_t. \tag{10.46}$$

10.1.4 Discrete DDPM

Equation (10.46) is the punchline: *to reverse the diffusion, we need the score* $\nabla_x \log p_t(x)$. A DDPM is essentially a practical, step-by-step way to approximate this reverse-time evolution.

Remark

In continuous time we defined

$$\alpha(t) = \exp\Big(-\frac{1}{2}\int_0^t \beta(s)\,ds\Big), \tag{10.47}$$

so the VP marginal has mean factor $\alpha(t)$ *and variance factor* $1-\alpha(t)^2$. *In discrete DDPM it is standard to write* $\alpha_t = 1-\beta_t$ *and* $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, *so the marginal mean factor is* $\sqrt{\bar{\alpha}_t}$. *Thus the correspondence is simply*

$$\alpha(t)^2 \approx \bar{\alpha}_t, \qquad \alpha(t) \approx \sqrt{\bar{\alpha}_t}. \tag{10.48}$$

Forward chain. Choose a small step size and define a discrete Markov chain

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{\alpha_t}\,x_{t-1},\ (1-\alpha_t)I\big), \qquad \alpha_t := 1-\beta_t. \tag{10.49}$$

Let $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. By multiplying Gaussians (or by induction), one gets the closed form

$$q(x_t \mid x_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_t}\,x_0,\ (1-\bar{\alpha}_t)I\big), \tag{10.50}$$

equivalently,

Theorem 10.5

$$x_t = \sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0,I). \tag{10.51}$$

So at each time t, the data signal is shrunk by $\sqrt{\bar{\alpha}_t}$ and the rest is Gaussian noise.

Then the question come up with: what should the reverse step look like? The true reverse transition $q(x_{t-1} \mid x_t)$ is generally intractable because it depends on the unknown data distribution. DDPMs therefore introduce a learned reverse kernel

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t,t),\ \Sigma_t\big), \tag{10.52}$$

where Σ_t is usually fixed by the schedule, and the main learning problem is the mean μ_θ.

Now comes the key bridge from (10.46) to DDPM: the reverse drift contains $-\beta(t)\nabla_x \log p_t(x)$, so *the only missing ingredient is the score*. In discrete time, the analogous object is the score of the marginal $q(x_t)$. Therefore, if we can approximate $\nabla_{x_t} \log q(x_t)$, we can write down a sensible reverse mean.

Note

Be careful with the placement of the time index t.

From (10.50), the conditional density $q(x_t \mid x_0)$ is Gaussian, whose score is explicit:

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}\, x_0}{1 - \bar{\alpha}_t}. \tag{10.53}$$

Using the reparameterization (10.51), we have

$$\varepsilon = \frac{x_t - \sqrt{\bar{\alpha}_t}\, x_0}{\sqrt{1 - \bar{\alpha}_t}}, \tag{10.54}$$

so (10.53) can be rewritten as

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{1}{\sqrt{1 - \bar{\alpha}_t}}\, \varepsilon. \tag{10.55}$$

This equation explains a famous design choice in DDPM: *instead of predicting the score directly, we predict the noise ε.* (Just like what we did in DSM.) If a network $\varepsilon_\theta(x_t, t)$ approximates the true ε in (10.51), then it also gives an approximation of the score field at time t via

$$\nabla_{x_t} \log q(x_t) \;\approx\; \psi_\theta(x_t, t) := -\frac{1}{\sqrt{1 - \bar{\alpha}_t}}\, \varepsilon_\theta(x_t, t). \tag{10.56}$$

Solving (10.54) for x_0 yields an estimator

$$\hat{x}_0(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}}\Big(x_t - \sqrt{1 - \bar{\alpha}_t}\, \varepsilon_\theta(x_t, t)\Big). \tag{10.57}$$

Plugging $\hat{x}_0$ into the Gaussian algebra for $q(x_{t-1} \mid x_t, x_0)$ gives the standard DDPM mean:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\Big(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}}\, \varepsilon_\theta(x_t, t)\Big). \tag{10.58}$$

Proof

From the forward chain,

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}\, x_{t-1},\, \beta_t I), \qquad \beta_t := 1 - \alpha_t, \tag{10.59}$$

and from the closed form,

$$q(x_{t-1} \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_{t-1}}\, x_0,\, (1 - \bar{\alpha}_{t-1}) I). \tag{10.60}$$

Conditioned on x_0, the pair (x_{t-1}, x_t) is jointly Gaussian, hence

$$q(x_{t-1} \mid x_t, x_0) \;\propto\; q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0) \tag{10.61}$$

is also Gaussian. Since both factors have isotropic covariance, the mean can be obtained by completing the square:

$$\begin{aligned}
\log q(x_{t-1} \mid x_t, x_0) &= -\frac{1}{2\beta_t}\Big(\alpha_t \|x_{t-1}\|^2 - 2\sqrt{\alpha_t}\, x_t^\top x_{t-1}\Big) \\
&\quad - \frac{1}{2(1 - \bar{\alpha}_{t-1})}\Big(\|x_{t-1}\|^2 - 2\sqrt{\bar{\alpha}_{t-1}}\, x_0^\top x_{t-1}\Big) + \text{const} \\
&= -\frac{1}{2}\left[\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}\right)\|x_{t-1}\|^2 - 2\left(\frac{\sqrt{\alpha_t}}{\beta_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} x_0\right)^\top x_{t-1}\right] + \text{const} \\
&= -\frac{A}{2}\left[\|x_{t-1}\|^2 - 2\left(\frac{b}{A}\right)^\top x_{t-1}\right] + \text{const} \\
&= -\frac{A}{2}\left[\left\|x_{t-1} - \frac{b}{A}\right\|^2 - \left\|\frac{b}{A}\right\|^2\right] + \text{const} \\
&= -\frac{1}{2\tilde{\beta}_t}\, \|x_{t-1} - \tilde{\mu}_t(x_t, x_0)\|^2 + \text{const},
\end{aligned} \tag{10.62}$$

where:

$$
\begin{aligned}
A &:= \frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}} = \frac{\alpha_t(1-\bar{\alpha}_{t-1}) + \beta_t}{\beta_t(1-\bar{\alpha}_{t-1})} = \frac{1-\bar{\alpha}_t}{\beta_t(1-\bar{\alpha}_{t-1})}, \\
b &:= \frac{\sqrt{\alpha_t}}{\beta_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}} x_0, \\
\tilde{\beta}_t &:= \frac{1}{A} = \frac{\beta_t(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}, \\
\tilde{\mu}_t(x_t, x_0) &:= \frac{b}{A} = \frac{\beta_t(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} \left(\frac{\sqrt{\alpha_t}}{\beta_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}} x_0 \right) \\
&= \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t} x_0.
\end{aligned} \tag{10.63}
$$

where we used $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$ and $\beta_t = 1 - \alpha_t$.

Now DDPM replaces the unknown x_0 by the estimator $\hat{x}_0(x_t, t)$:

$$
\mu_\theta(x_t, t) := \tilde{\mu}(x_t, \hat{x}_0(x_t, t)). \tag{10.64}
$$

Substituting

$$
\hat{x}_0(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \sqrt{1-\bar{\alpha}_t}\, \varepsilon_\theta(x_t, t) \right) \tag{10.65}
$$

and simplifying yields the standard DDPM mean

$$
\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\, \varepsilon_\theta(x_t, t) \right). \tag{10.66}
$$

■

Then a sampling step is simply

$$
x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z, \qquad z \sim \mathcal{N}(0, I), \tag{10.67}
$$

with a chosen variance σ_t^2 (often related to β_t).

Because (10.51) provides the true noise ε, we can train ε_θ by regression:

$$
\mathcal{L}(\theta) = \mathbb{E}_t\, \mathbb{E}_{x_0 \sim p_{\text{data}}}\, \mathbb{E}_{\varepsilon \sim \mathcal{N}(0,I)} \left[\|\varepsilon - \tilde{\varepsilon}_\theta(x_t, t)\|^2 \right], \tag{10.68}
$$

where x_t is constructed from x_0, ε by (10.51). Comparing (10.68) with (10.56), we see that DDPM training is just a denoising-score-matching objective in disguise.

Therefore, the reverse SDE (10.46) tells us *what information is needed* (the score) and DDPM shows *how to obtain it in practice*.

Although this section is intended only as a physical and conceptual supplement, it has in fact walked through a nearly complete derivation of diffusion models from first principles. Readers who are less interested in the mathematical or physical details may safely skip the derivations and focus on the final conclusions in the following sections, which are all that is needed for practical understanding and use.

10.2 Denoising Diffusion Probabilistic Models (DDPM)

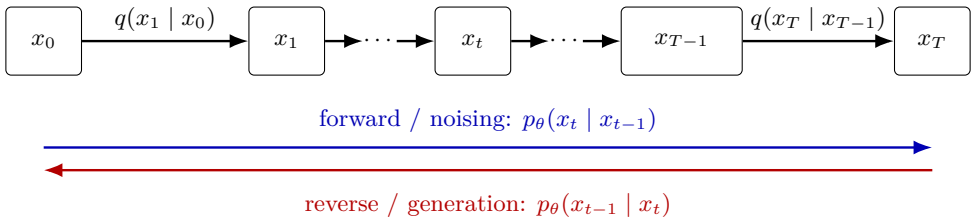
Unlike VAE-style *one-shot* generation, where one samples a low-dimensional latent variable $z \sim \mathcal{N}(0, I)$ and decodes it in a single step, diffusion models generate samples through a sequence of *many* small transformations.

Specifically, we choose a large time horizon T (e.g. $T = 1000$) and begin from pure Gaussian noise $x_T \sim \mathcal{N}(0, I)$, then progressively denoise:

$$
x_T \to x_{T-1} \to \cdots \to x_1 \to x_0.
$$

The final state x_0 lies in the *same* space as the data (e.g. an image vector in $\mathbb{R}^d$), while all intermediate states are unobserved

and serve as latent variables that mediate this gradual transformation.



Remark

In the diffusion literature, the forward *process refers to the gradual noising transformation $x_0 \mapsto x_T$, whereas the* reverse *process refers to the denoising transformation $x_T \mapsto x_0$. Importantly, the reverse process is the actual* generative *procedure.*

Remark

It is remarkable that meaningful content,ranging from natural images to molecular structures, can be generated starting from pure noise.

Motivated by this perspective, we model diffusion as a latent-variable probabilistic model defined by a Markov chain

$$x_0, x_1, \ldots, x_T \in \mathbb{R}^d,$$

where only x_0 is observed (the training data) and all $x_1, \ldots, x_T$ are latent. The conditional-independence structure is a simple chain.

Note

This is another latent-variable maximum-likelihood problem (similar in spirit to VAE), but with many *latent variables. Directly computing $p_\theta(x_0)$ requires marginalizing all $x_{1:T}$ and is intractable.*

We model the reverse denoising chain as

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t), \tag{10.69}$$

with the prior $p(x_T) = \mathcal{N}(0, I)$. For continuous-valued variables, we typically parameterize each reverse transition as Gaussian:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t),\, \Sigma_\theta(x_t, t)\big). \tag{10.70}$$

In practice, both μ_θ and Σ_θ are produced by neural networks that take x_t as input; it is also common to explicitly feed the time index t (or a time embedding) so that the network can distinguish different denoising steps.

We want to maximize the data log-likelihood $\operatorname{argmax}_\theta \log p_\theta(x_0)$, which is intractable due to latent-variable marginalization. Introduce a variational distribution $q(x_{1:T} \mid x_0)$, to approximate the true posterior $p_\theta(x_{1:T} \mid x_0)$. By the standard variational identity we mentioned last chapter,

$$\log p_\theta(x_0) = \mathcal{L}_{\text{ELBO}}(x_0; \theta) + \mathrm{KL}(q(x_{1:T} \mid x_0) \,\|\, p_\theta(x_{1:T} \mid x_0)), \tag{10.71}$$

where the evidence lower bound (ELBO) is

$$\mathcal{L}_{\text{ELBO}}(x_0; \theta) := \mathbb{E}_{q(x_{1:T}|x_0)}\left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)}\right]. \tag{10.72}$$

Maximizing ELBO is equivalent to maximizing $\log p_\theta(x_0)$ when the variational posterior matches the true posterior.

In DDPM, the variational distribution q is chosen as a **fixed** Markov noising process:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}), \tag{10.73}$$

with

Theorem 10.6

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(\sqrt{\alpha_t}\, x_{t-1},\ (1-\alpha_t)I\right), \qquad \alpha_t \in (0,1). \tag{10.74}$$

Remark

Here we write the forward step as

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, I), \tag{10.75}$$

rather than using α_t and $(1-\alpha_t)$ directly, for a simple variance reason. If x_{t-1} has (approximately) unit covariance and is independent of ε_t, then

$$\mathrm{Cov}(x_t) = \alpha_t \,\mathrm{Cov}(x_{t-1}) + (1-\alpha_t)\,\mathrm{Cov}(\varepsilon_t) \approx \alpha_t I + (1-\alpha_t)I = I. \tag{10.76}$$

So α_t represents the fraction of variance *preserved from the previous state, while $1-\alpha_t$ is the* fraction of variance *injected as fresh Gaussian noise. Using the square-roots makes the coefficients act on standard deviations, ensuring the variances add linearly and the overall scale of x_t stays stable across many steps.*

The sequence $\{\alpha_t\}$ is a **noise schedule** (hyperparameters); typically α_t gradually decreases so that later steps inject more noise.

Remark

Unlike the VAE encoder $q_\phi(z \mid x)$, the forward chain (10.74) is pre-specified *and contains no learnable parameters. This guarantees that, with a suitable schedule, x_T becomes (approximately) standard Gaussian without needing an explicit KL regularizer.*

Define

$$\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s, \qquad \bar{\alpha}_0 := 1. \tag{10.77}$$

Then one can show by recursion (using Gaussian reparameterization and Gaussian-additivity) that

$$q(x_t \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)I\right), \tag{10.78}$$

equivalently,

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I). \tag{10.79}$$

Remark

eq:ddpm-reparam implies we can sample x_t from x_0 in one *shot without simulating the entire chain. This is crucial for efficient training.*

From (10.78), if $\bar{\alpha}_T \approx 0$ (achieved by large T and a decreasing schedule), then

$$q(x_T \mid x_0) \approx \mathcal{N}(0, I),$$

so the end state is close to an isotropic Gaussian noise, matching the reverse-process prior $p(x_T)$.

The ELBO contains terms involving both q and p_θ. A key trick is to work with the forward posterior **conditioned on** x_0: $q(x_{t-1} \mid x_t, x_0)$. Although $q(x_{t-1} \mid x_t)$ is generally intractable, conditioning on x_0 makes it tractable because all

relevant terms admit closed forms: $q(x_t \mid x_{t-1})$, $q(x_{t-1} \mid x_0)$, and $q(x_t \mid x_0)$. By Bayes' rule,

$$q(x_{t-1} \mid x_t, x_0) = \frac{q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)} \tag{10.80}$$

Theorem 10.7 (DDPM posterior is Gaussian)

Under the Gaussian forward process (10.74), the conditional distribution $q(x_{t-1} \mid x_t, x_0)$ is Gaussian:

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(\tilde{\mu}_t(x_t, x_0),\, \tilde{\beta}_t I\big), \tag{10.81}$$

where writing $\beta_t := 1 - \alpha_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$,

$$\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\, \beta_t, \tag{10.82}$$

and the posterior mean can be expressed as

$$\tilde{\mu}_t(x_t, x_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\, x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\, x_t. \tag{10.83}$$

A detailed derivation can be found in the supplementary section earlier in this note.

Remark

$$\tilde{\mu}_t(x_t, \varepsilon) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}}\, \varepsilon \right), \tag{10.84}$$

Then we can apply reparameterization trick.

Expand the ELBO using the Markov factorizations of p_θ and q:

$$\mathcal{L}_{\text{ELBO}}(x_0; \theta) = \mathbb{E}_{q(x_{1:T} \mid x_0)} \left[\log p(x_T) + \sum_{t=1}^{T} \log p_\theta(x_{t-1} \mid x_t) - \sum_{t=1}^{T} \log q(x_t \mid x_{t-1}) \right]. \tag{10.85}$$

Rearranging yields a standard DDPM form:

$$\mathcal{L}_{\text{ELBO}}(x_0; \theta) = -\mathrm{KL}(q(x_T \mid x_0) \,\|\, p(x_T)) + \mathbb{E}_q[\log p_\theta(x_0 \mid x_1)] - \sum_{t=2}^{T} \mathbb{E}_q[\mathrm{KL}(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t))]. \tag{10.86}$$

Proof

Use the identity

$$q(x_{1:T} \mid x_0) = q(x_T \mid x_0) \prod_{t=2}^{T} q(x_{t-1} \mid x_t, x_0), \tag{10.87}$$

which implies

$$\sum_{t=1}^{T} \log q(x_t \mid x_{t-1}) = \log q(x_T \mid x_0) + \sum_{t=2}^{T} \log q(x_{t-1} \mid x_t, x_0). \tag{10.88}$$

Substitute (10.88) into the ELBO:

$$\mathcal{L}_{\text{ELBO}}(x_0; \theta) = \mathbb{E}_q[\log p(x_T) - \log q(x_T \mid x_0)] + \mathbb{E}_q[\log p_\theta(x_0 \mid x_1)] \tag{10.89}$$

$$+ \sum_{t=2}^{T} \mathbb{E}_q[\log p_\theta(x_{t-1} \mid x_t) - \log q(x_{t-1} \mid x_t, x_0)]. \tag{10.90}$$

Now recognize KL divergences. The first expectation depends only on $x_T \sim q(x_T \mid x_0)$:

$$\mathbb{E}_q[\log p(x_T) - \log q(x_T \mid x_0)] = -\mathrm{KL}(q(x_T \mid x_0) \,\|\, p(x_T)). \tag{10.91}$$

For each $t \geq 2$, conditioning on (x_t, x_0), we have

$$\mathbb{E}_q[\log p_\theta(x_{t-1} \mid x_t) - \log q(x_{t-1} \mid x_t, x_0)] = -\mathbb{E}_q[\mathrm{KL}(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t))]. \tag{10.92}$$

Putting the terms together yields (10.86). ■

Remark

- *The prior KL term is usually treated as a constant (it contains no learnable parameters).*
- *The $t = 1$ reconstruction term $\mathbb{E}_q[\log p_\theta(x_0 \mid x_1)]$ is sometimes ignored or folded into the $t = 1$ case.*
- *The main training signal comes from the sum of KL terms for $t = 2, \ldots, T$.*

We choose $p_\theta(x_{t-1} \mid x_t)$ as Gaussian and parameterize its mean to mirror (10.84):

$$\mu_\theta(x_t, t) := \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right), \tag{10.93}$$

where $\varepsilon_\theta(x_t, t)$ is a neural network (typically U-Net) that predicts the injected noise. For the variance, a common simplification is to fix

$$\Sigma_\theta(x_t, t) = \sigma_t^2 I, \tag{10.94}$$

with σ_t set by a schedule (often related to $\beta_t = 1 - \alpha_t$), rather than learned.

Remark

Why not learn the variance? Learning variances introduces log-determinants and matrix inverses in the Gaussian KL closed form, making optimization harder. Empirically, matching the mean *(or equivalently the denoising direction) is most important.*

Because both $q(x_{t-1} \mid x_t, x_0)$ and $p_\theta(x_{t-1} \mid x_t)$ are Gaussians, each KL term in (10.86) has a closed form. Under the parameterization (10.93) and with fixed variances, each KL term in (10.86) reduces to a squared difference between the posterior mean $\tilde{\mu}_t(x_t, x_0)$ and the model mean $\mu_\theta(x_t, t)$. Since both means can be written as linear functions of the injected noise ε, minimizing the KL is equivalent (up to a time-dependent weight) to matching noises:

$$\mathcal{L}_{\text{simple}}(\theta) := \mathbb{E}_{x_0 \sim p_{\text{data}}} \mathbb{E}_{t \sim \text{Unif}(\{1,\ldots,T\})} \mathbb{E}_{\varepsilon \sim \mathcal{N}(0,I)} \left[\left\| \varepsilon - \varepsilon_\theta\left(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon,\ t \right) \right\|_2^2 \right]. \tag{10.95}$$

The original derivation yields a per-step scaling weight; many implementations drop the weight and still obtain strong performance.

Algorithm 1 DDPM training (noise prediction)

Require: dataset $\{x_0\}$, noise schedule $\{\alpha_t\}_{t=1}^T$, model $\varepsilon_\theta(\cdot, \cdot)$

```
repeat
    Sample a data point x_0 ~ p_data
    Sample a timestep t ~ Unif({1, ..., T})
    Sample noise ε ~ N(0, I)
    Form x_t = √ᾱ_t x_0 + √(1 − ᾱ_t) ε
    Take a gradient step on ‖ε − ε_θ(x_t, t)‖_2^2
until converged
```

To generate a sample, initialize $x_T \sim \mathcal{N}(0, I)$ and iterate backward:

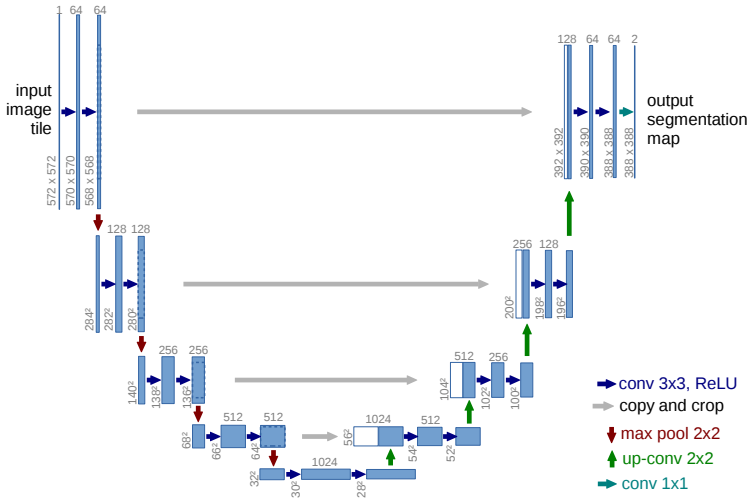
$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z, \qquad z \sim \mathcal{N}(0, I), \tag{10.96}$$

where σ_t^2 is typically chosen to match the forward variance, e.g. $\sigma_t^2 = \beta_t$ or $\tilde{\beta}_t$, for $t = T, T-1, \ldots, 1$, with the convention $z = 0$ at $t = 1$ (do not re-inject noise at the last step). Substitute (10.93) for μ_θ to obtain the commonly used update:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z. \tag{10.97}$$

10.3 Practical Implementation of Diffusion Models

- The noise network $\varepsilon_\theta(x_t, t)$ maps an input tensor to an output tensor of the *same shape*, which naturally matches **U-Net** (encoder–decoder with skip connections):



- Recent diffusion systems also use transformer backbones (Diffusion Transformer, **DiT**) as the denoiser. The probabilistic formulation remains similar; the main difference is the architecture used to represent ε_θ.
- **Latent diffusion**: DDPM operates in pixel space, which can be costly for high-resolution images. Latent diffusion uses a pretrained autoencoder (e.g. a VAE):
  - Encode x to a latent $z = \mathrm{Enc}(x)$.
  - Run diffusion in latent space to model $p(z)$ or $p(z \mid y)$.
  - Decode $\hat{x} = \mathrm{Dec}(z_0)$ back to pixel space.

  This reduces the dimensionality of the denoiser input/output and accelerates training and sampling.
- **Conditional generation**: To control what is generated (e.g. "cat" vs. "dog"), one trains a conditional model $p_\theta(x \mid y)$ by providing paired data (x_0, y) and feeding y (or a text prompt embedding) into the denoiser. Practical systems often use techniques such as classifier-free guidance.
- **Flow matching / rectified flow** Diffusion can be viewed as learning a trajectory from noise to data. Flow matching methods aim to learn a simpler transport, often a straight-line path, by predicting a velocity field. At a high level, sample an interpolation point between noise g and data x and learn to predict the direction (velocity) pointing from g to x.